Universidad Católica de Córdoba

Facultad de Ingeniería

Proyecto: NexusAI



Asistente académico con inteligencia artificial integrado en Moodle



Informe Final de Grado

Alumnos:

- Santiago Tricherri
- Marcos Bugliotti
- Delfina Salinas

Directores:

- Ignacio Luciano Carreño
- Federico Eduardo Porrini
- Leandro Juarez

21 de octubre de 2026

Córdoba — Argentina

Índice

1 Resumen
- 1.1 Abstract

2 Presentación del tema

3 Glosario

4 Diagnóstico (problemática)
- 4.1 Estado actual (contexto)
- 4.2 Impacto sobre los interesados
- 4.3 Oportunidades

5 Relevamiento y metodología de investigación
- 5.1 Metodología
- 5.2 Encuesta a estudiantes (N = 166)
- 5.3 Encuesta a docentes (N = 17)
- 5.4 Contexto institucional relevado
- 5.5 Síntesis

6 Objetivos
- 6.1 Objetivo general
- 6.2 Objetivos específicos
- 6.3 Tabla de trazabilidad

7 Marco teórico
- 7.1 Contexto general del problema
- 7.2 Técnicas investigadas
- 7.3 Tecnologías investigadas
  - 7.3.1 Base de datos vectorial
  - 7.3.2 Modelos de lenguaje y embeddings
  - 7.3.3 Arquitectura de plugin Moodle
  - 7.3.4 Extracción y fragmentación de documentos
  - 7.3.5 Estilos de arquitectura del backend
  - 7.3.6 Autenticación servicio-a-servicio
  - 7.3.7 Transporte de respuestas en tiempo real
  - 7.3.8 Infraestructura de despliegue
  - 7.3.9 Validación de datos en los límites del sistema
  - 7.3.10 Control de consumo y moderación de contenido
  - 7.3.11 Frameworks de desarrollo web y empaquetado
  - 7.3.12 Almacenamiento en memoria para estado efímero
  - 7.3.13 Proxy inverso, certificados TLS y monitoreo
  - 7.3.14 Estrategias de prueba e integración continua
  - 7.3.15 Seguridad de aplicaciones web y de modelos de lenguaje
  - 7.3.16 Protección de datos personales y control de acceso
  - 7.3.17 Reconocimiento de voz

7.3.18 Licencias de software libre
7.4 Opciones similares en el mercado
7.4.1 Asistentes académicos generales
7.4.2 Plugins de Moodle con inteligencia artificial
8 Propuesta de solución
8.1 Alcance funcional
8.2 Diseño
8.2.1 Arquitectura del software
8.2.2 Patrones de diseño
8.2.3 Interfaz de usuario
8.2.4 Proceso de diseño de la interfaz
8.2.5 Justificación de las decisiones frente a las alternativas
8.3 Implementación
8.3.1 Funcionalidades post-MVP
8.4 Arquitectura técnica del backend
8.4.1 Autenticación HMAC en tres capas
8.4.2 Esquema de base de datos: alcance vigente y migración de dimensión
8.4.3 Streaming de respuestas: Server-Sent Events
8.4.4 Contratos de datos: schemas Pydantic
8.4.5 Ciclo de vida y estado de la aplicación
8.5 Arquitectura técnica del plugin Moodle
8.5.1 Inyección del widget: Hook API y callback legado
8.5.2 Capabilities y control de acceso
8.5.3 Web services: el patrón de tres métodos
8.5.4 React como módulo AMD
8.5.5 Empaquetado con Webpack
8.6 Pruebas
8.6.1 Pruebas unitarias y de integración (backend)
8.6.2 Pruebas automatizadas del frontend y del plugin
8.6.3 Evaluación de calidad del asistente: dos campañas
8.6.4 Pruebas de aceptación, usuarios reales y rendimiento
8.6.5 Síntesis de resultados frente a los objetivos
9 Seguridad y cumplimiento
9.1 Auditoría de seguridad: OWASP Top 10 y análisis de dependencias
9.2 Estrategia de privacidad y cumplimiento normativo
9.3 Prácticas de seguridad del plugin
10 Despliegue e infraestructura
10.1 Topología: del plan original a las dos VMs reales
10.2 El proceso de despliegue
10.3 Operación, mantenimiento y brechas pendientes
11 Publicación en Moodle Marketplace

11.1 Por qué separar el repositorio
11.2 Proceso de envío y primera revisión
11.3 Correcciones de la primera ronda
11.4 Auditoría propia y cierre de los últimos gaps
11.5 Sincronización automática y una lección operativa
11.6 Estado actual
12 Gestión del proyecto y metodología ágil
12.1 Metodología adoptada
12.2 Cierre de Fase 1 — Setup e Investigación
12.3 Cierre de Sprint 2 — RAG completo
12.4 Un pivote real de infraestructura
12.5 Cierre
13 Beneficios post-implementación
14 Impacto económico (estudio de costos)
14.1 Costos de operación para la institución
14.2 Costos de personal (valoración teórica)
14.3 Análisis costo-beneficio para la institución
14.4 Alternativa: clave de IA propia del alumno
15 Impacto social
16 Impacto medioambiental
17 Conclusión
17.1 Reflexiones personales del equipo
18 Bibliografía / Referencias
19 Anexos
19.1 Anexo A — Requerimientos detallados
19.1.1 Requerimientos funcionales
19.1.2 Requerimientos no funcionales
19.2 Anexo B — Modelo de datos
19.3 Anexo C — Métricas del proyecto
19.4 Anexo D — Repositorio y evidencias
19.5 Anexo E — Registro de decisiones de arquitectura (ADRs)
19.6 Anexo F — Documentación técnica del repositorio

1 Resumen

El presente informe documenta el desarrollo de **NexusAI**, un asistente académico basado en inteligencia artificial integrado como plugin en la plataforma Moodle.

Las plataformas de gestión del aprendizaje (LMS) como Moodle se utilizan mayoritariamente como repositorios documentales estáticos, mientras que los estudiantes recurren a asistentes de IA generales —ajenos al material de cada materia — para resolver sus dudas. Esto produce respuestas genéricas, dispersión de la información y pérdida de visibilidad del docente sobre las consultas del alumnado.

Para abordar esta situación se diseñó e implementó un plugin de tipo `local` para Moodle, acompañado de un backend que aplica la técnica de *Retrieval-Augmented Generation* (RAG) sobre el material indexado de cada curso, almacenado en una base de datos vectorial. La solución recupera los fragmentos más relevantes del material y los provee como contexto a un modelo de lenguaje de gran escala, que responde citando explícitamente la fuente.

El producto mínimo viable (MVP), entregado en junio de 2026, alcanzó siete funcionalidades en producción: buscador semántico, chat multicurso, respuesta progresiva, citas trazables, historial, generador de cuestionarios y detección de vacíos de contenido. Posteriormente se incorporaron foros asistidos por IA, analítica docente, generación de exámenes, un plan de estudio personalizado y nuevos formatos de material.

Los resultados muestran que es posible ofrecer asistencia académica contextualizada y auditable dentro del LMS institucional: la búsqueda ubicó el material pertinente entre los cinco primeros resultados en todas las preguntas con respuesta en el material, el 95 % de las respuestas completas demoró menos de 4,4 segundos, la cobertura de pruebas automatizadas alcanzó el 83,6 % y el plugin fue aprobado y publicado en el directorio oficial de Moodle.

Palabras clave: Moodle, inteligencia artificial, RAG, asistente académico, educación, soberanía de datos.

1.1 Abstract

This report documents the development of **NexusAI**, an artificial-intelligence academic assistant integrated as a plugin into the Moodle platform.

Learning Management Systems (LMS) such as Moodle are mostly used as static document repositories, while students rely on general-purpose AI assistants — unaware of each subject's material— to solve their doubts. This leads to generic answers, information dispersion, and a loss of teacher visibility over student queries.

To address this, a Moodle `local` plugin was designed and implemented, together with a backend that applies Retrieval-Augmented Generation (RAG) over each course's indexed material, stored in a vector database. The solution retrieves the most relevant fragments and provides them as context to a large language model, which answers by explicitly citing its source.

The minimum viable product (MVP), delivered in June 2026, reached seven production features: semantic search, multi-course chat, progressive responses, traceable citations, history, quiz generation, and content-gap detection. Later iterations added AI-assisted forums, teacher analytics, exam generation, a personalized study plan, and new material formats.

Results show that contextualized and auditable academic assistance can be delivered within the institutional LMS: retrieval placed the relevant material among the top five results for every evaluation question answerable from the material, 95% of complete answers took less than 4.4 seconds, automated test coverage reached 83.6%, and the plugin was approved and published in the official Moodle directory.

Keywords: Moodle, artificial intelligence, RAG, academic assistant, education, data sovereignty.

2 Presentación del tema

La adopción de la inteligencia artificial generativa en la educación universitaria avanza más rápido que su integración formal en las instituciones y en sus plataformas educativas (UNESCO, 2023). Mientras los estudiantes utilizan asistentes generales para resolver dudas académicas, las plataformas LMS institucionales como Moodle continúan funcionando como repositorios documentales, sin integración con la inteligencia artificial.

El proyecto NexusAI surge de esta brecha. Su propósito es incorporar, dentro del propio campus virtual, un asistente académico capaz de responder consultas en lenguaje natural sobre el material real de cada materia, generar ejercicios de práctica y proveer información accionable al docente. La relevancia del tema radica en que la solución no compite con el docente ni reemplaza a la plataforma existente: agrega una capa de inteligencia sobre Moodle, manteniendo a los estudiantes en el entorno donde reside el material académico validado.

El problema que motiva el proyecto se desarrolla en detalle en la sección *Diagnóstico*, y constituye el eje central sobre el cual se articula el resto del informe. El relevamiento formal realizado para este proyecto (166 estudiantes y 17 docentes de la UCC) confirma la brecha: el 80,7 % de los estudiantes percibe dispersión de la información entre el campus virtual, las aplicaciones de mensajería y los servicios de almacenamiento personales, y los 88 estudiantes que respondieron la pregunta sobre IA declararon usar un asistente generalista para estudiar o hacer trabajos (con la salvedad metodológica de la sección 5.2). NexusAI responde a esa brecha con tres capacidades concretas dentro del propio Moodle: un asistente conversacional que cita la fuente exacta del material, un generador de ejercicios de práctica a partir de ese mismo material y la detección automática de los vacíos de contenido que quedan sin responder.

3 Glosario

Se definen a continuación los términos específicos del dominio del proyecto, necesarios para comprender el informe por parte de un lector no especializado en el área.

- **LMS (Learning Management System):** plataforma de gestión del aprendizaje que centraliza materiales, actividades y comunicación de un curso. Moodle es el LMS que utiliza la UCC para su campus virtual.
- **Moodle:** sistema de gestión del aprendizaje de código abierto, con más de 148.000 sitios registrados en 237 países (Moodle, s.f.-d), utilizado por instituciones educativas para administrar cursos en línea.
- **Plugin:** componente de software que extiende las funcionalidades de una plataforma existente sin modificar su núcleo. NexusAI se distribuye como un plugin de tipo `local` para Moodle.
- **Inteligencia artificial generativa:** rama de la IA capaz de producir contenido nuevo —texto, imágenes, código— a partir de patrones aprendidos de grandes volúmenes de datos.
- **Modelo de lenguaje de gran escala (LLM):** modelo de inteligencia artificial entrenado sobre grandes cantidades de texto, capaz de comprender y generar lenguaje natural.
- **RAG (Retrieval-Augmented Generation):** técnica que combina la búsqueda de información relevante en una fuente de datos propia con la generación de texto de un LLM, permitiendo respuestas fundamentadas en material específico.
- **Embedding:** representación numérica (vector) de un fragmento de texto que captura su significado semántico, permitiendo comparar la similitud entre textos.
- **Base de datos vectorial:** sistema de almacenamiento optimizado para guardar y buscar *embeddings* por similitud semántica.

- **Alucinación (de un LLM):** fenómeno por el cual un modelo de lenguaje genera información plausible pero falsa o no fundamentada en una fuente real.
- **Chunk (fragmento):** porción de texto en la que se divide un documento para su indexación y recuperación.
- **Streaming:** entrega progresiva de la respuesta del modelo, token por token, para reducir la latencia percibida por el usuario.
- **Capability (Moodle):** permiso con nombre (por ejemplo, `local/nexusai:use`) que Moodle asigna a los roles en un contexto determinado: sitio, categoría, curso o actividad.
- **Similitud coseno:** medida de semejanza entre dos vectores basada en el coseno del ángulo que forman; es la métrica habitual para comparar *embeddings*.
- **HNSW (Hierarchical Navigable Small World):** estructura de índice basada en grafos jerárquicos para la búsqueda aproximada de vecinos más cercanos.
- **Recall@k, Precision@k y MRR:** métricas de recuperación de información; miden, respectivamente, qué proporción de los fragmentos relevantes aparece entre los *k* primeros resultados, qué proporción de esos *k* resultados es relevante y en qué posición aparece, en promedio, el primer resultado relevante.
- **GIFT:** formato de texto de Moodle para importar bancos de preguntas.

4 Diagnóstico (problemática)

La problemática que da origen al proyecto no constituye únicamente un problema en sentido estricto, sino también una oportunidad de mejora dentro del proceso educativo universitario mediado por plataformas digitales.

4.1 Estado actual (contexto)

Las plataformas LMS como Moodle son fundamentales en la educación universitaria, pero en la práctica se utilizan como repositorios estáticos: los estudiantes descargan archivos y entregan trabajos, sin un acompañamiento real al proceso de aprendizaje. A partir de la experiencia directa del equipo del proyecto como estudiantes de la Universidad Católica de Córdoba (UCC), se identificaron de manera preliminar tres problemáticas recurrentes en el uso del campus virtual, que luego se validaron mediante un relevamiento formal a 166 estudiantes y 17 docentes de la institución:

- **Dispersión de la información** entre el campus virtual, aplicaciones de mensajería y servicios de almacenamiento personales, que dificulta encontrar el material correcto en el momento de estudiar. El 80,7% de los estudiantes encuestados percibe esta dispersión al menos “a veces”, y un 43,4% la califica como alta o frecuente.
- **Dificultad para organizar el estudio** y planificar repasos antes de las evaluaciones, en un contexto donde solo el 34% de los estudiantes describe la mayoría de sus materias como bien organizadas dentro del campus.
- **Uso pasivo del campus virtual** frente al material académico: aunque el 72,3% de los estudiantes ingresa al campus a diario o varias veces por semana, el 100% de quienes fueron consultados al respecto ya recurre a herramientas de IA generalistas —ajenas al material de la materia— para estudiar o resolver trabajos, lo que evidencia una necesidad no cubierta por la plataforma institucional.

El fenómeno no es local. La encuesta global del Digital Education Council (2024), con 3.839 respuestas de estudiantes de grado y posgrado de 16 países, encontró que el 86 % utiliza herramientas de IA en sus estudios —la más utilizada es ChatGPT—, y la

guía de la UNESCO (2023) advierte que la adopción de la IA generativa avanza más rápido que su regulación e integración por parte de las instituciones educativas.

4.2 Impacto sobre los interesados

La situación afecta a dos grupos principales. Por un lado, los estudiantes reciben respuestas genéricas de modelos entrenados sobre internet abierto —no sobre el material específico de su curso—, con riesgo de respuestas plausibles pero incorrectas en el contexto de esa materia particular. Por otro lado, los docentes pierden visibilidad sobre qué consultan los estudiantes acerca de su material, porque dichas consultas suceden fuera del LMS.

4.3 Oportunidades

A partir de este diagnóstico se identifican tres oportunidades de mejora que el proyecto busca aprovechar:

- Mantener a los estudiantes dentro del campus virtual, donde reside el material validado por el docente.
- Ofrecer respuestas fundamentadas en el material específico del curso, y no en fuentes genéricas de internet.
- Generar datos accionables para el docente sobre las consultas y los vacíos de contenido detectados.

Los datos citados en esta sección provienen del relevamiento propio del equipo: encuesta a 166 estudiantes y encuesta a 17 docentes de la UCC, realizadas entre marzo y junio de 2026. El detalle de preguntas, metodología y resultados completos se desarrolla en la sección *Relevamiento y metodología de investigación*, a continuación.

5 Relevamiento y metodología de investigación

Esta sección documenta en detalle el relevamiento mencionado en el Diagnóstico: diseño, proceso de recolección, preguntas completas y resultados de las dos encuestas propias del equipo, junto con el contexto institucional relevado en paralelo.

5.1 Metodología

Ambas encuestas se diseñaron como formularios de Google, difundidas entre marzo y junio de 2026, y comparten una particularidad metodológica que se documenta explícitamente para no inflar artificialmente la muestra: el formulario se amplió mientras ya se estaban recolectando respuestas, de modo que las primeras respuestas solo vieron el bloque inicial de preguntas y las posteriores vieron el formulario completo. Por esta razón, cada resultado reportado a continuación indica el **N real** de quienes vieron esa pregunta en particular, en lugar de asumir el total de respondentes para toda pregunta — algunos resultados tienen como denominador el total de la encuesta y otros un subconjunto, según corresponda.

5.2 Encuesta a estudiantes (N = 166)

Relevamiento cerrado entre el 31 de marzo y junio de 2026, con 166 respuestas mayoritariamente de la Facultad de Ingeniería, con representación también de Ciencias Económicas y de Administración, Ciencias de la Salud, Derecho y Ciencias Sociales, Ciencias Políticas y RRII, y Ciencias Agropecuarias. El cuestionario cubrió cinco bloques: perfil y uso del campus, experiencia y organización de materias, dispersión de información y comunicación, validación de funcionalidades de NexusAI, y una pregunta abierta de cierre.

Frecuencia de uso del campus virtual (n=166): el 72,3% (120/166) lo usa todos los días o varias veces por semana. El campus es, en la práctica, parte de la rutina de la mayoría — la problemática que motiva el proyecto no es la falta de uso, sino qué tan bien ese uso frecuente se traduce en organización real del estudio.

Dispersión de la información (n=166): para poder combinar las dos versiones del formulario (que ofrecían escalas de respuesta distintas para esta pregunta), los resultados se agruparon en tres categorías equivalentes. El 43,4% (72/166) reporta dispersión alta o frecuente, el 37,3% (62/166) reporta dispersión intermedia y el 19,3% (32/166) reporta dispersión baja o nula. En conjunto, **el 80,7% de los estudiantes**

percibe dispersión de información al menos "a veces", confirmando con datos formales la problemática que el equipo había identificado inicialmente de forma informal.

Experiencia y organización de materias (n=88): el 80,7% califica su experiencia con el campus como buena o mejor, pero la razón dominante ("funciona bien aunque le faltan cosas") describe un uso intensivo y a la vez insatisfecho, no un rechazo a la plataforma. Sobre la organización de sus materias, solo el 34% ve "la mayoría" o "todas" bien organizadas; el 54,5% las describe como "mitad y mitad" — la organización depende fuertemente de cada docente, no de la plataforma en sí.

Uso de herramientas de IA para estudiar (n=88): el 100% de quienes vieron esta pregunta ya usa alguna herramienta de IA generalista para estudiar, hacer trabajos, o ambas cosas. Es un dato a interpretar con cautela porque el formulario no ofrecía ninguna opción negativa, pero es consistente con el resto del relevamiento: la IA generalista ya es una práctica de estudio extendida, mientras el campus virtual no ofrece nada equivalente integrado.

Corrección automática de ejercicios (n=88): el 95,5% (84/88) ve valor en que una IA corrija ejercicios con explicación — 43,2% la usaría frecuentemente, 37,5% como complemento a la corrección del docente y 14,8% "tal vez, según la materia". Solo el 4,5% prefiere que la corrección quede exclusivamente en manos del docente.

Dos respuestas abiertas ilustran el patrón encontrado:

> "Mejoraría el hecho de que no esté tan dispersa la información, también que esté más organizado."

> "El valor del aula virtual depende mucho de cómo la preparen los profesores. Bien organizada, mejora mucho la experiencia."

5.3 Encuesta a docentes (N = 17)

Relevamiento cerrado entre el 1 de abril y el 19 de junio de 2026, con 17 respuestas de docentes mayoritariamente de la Facultad de Ingeniería. La encuesta se revisó previamente con Leandro Juarez, director del proyecto y docente de la cátedra de Proyecto Integrador, antes de su difusión.

Uso actual de IA en la práctica docente (n=17): el 64,7% (11/17) ya usa alguna herramienta de IA; ningún docente la rechaza de plano.

Delegación de tareas a una IA integrada al campus (n=17): el 88,2% (15/17) delegaría al menos parcialmente tareas a una IA integrada —el 47,1% condicionado a poder revisar el resultado antes de publicarlo, el 23,5% "según la tarea" y el 17,6% sin condición. Ningún docente respondió que no delegaría nada. La condición de supervisión antes de publicar, repetida en la mayoría de las respuestas, valida directamente el patrón de diseño "asistir, no reemplazar" adoptado en NexusAI.

Estructura inicial del curso armada por IA (n=15): el 86,7% (13/15) ve valor en que una IA arme la estructura inicial del curso, y 12 de esos 13 docentes se reparten en partes iguales entre hacerlo con revisión posterior (40%) o en paralelo, decidiendo cada paso junto a la IA (40%). Solo el 13,3% prefiere armarlo íntegramente solo.

Herramientas de evaluación actuales (n=15): un tercio no las usa y no puede opinar; del resto, las opiniones se dividen entre "básicas pero sirven", "me faltan funcionalidades" y "cubren lo que necesito" en proporciones similares. Las sugerencias abiertas más repetidas piden corrección automática de trabajos y rúbricas inteligentes.

Dos respuestas abiertas:

> "Transformarlo en una herramienta de gestión real y no un reservorio de materiales."
>
> "Alertas tempranas sobre temas docentes a resolver (contenido, clases, asistencias, notas, condiciones)."

La tabla siguiente resume cómo los resultados confirmaron o descartaron decisiones de alcance tomadas antes de tener los datos:

Tabla 1. *Decisiones de alcance confrontadas con los resultados de la encuesta a docentes*

Condición evaluada	Resultado real
>50% de interés en generador de evaluaciones / delegar tareas a una IA → adelantar esa épica al Sprint 3	**Confirmado (88,2%)** — consistente con incluir Herramientas para Docentes en el alcance del MVP.
Bajo interés en analytics → bajar prioridad post-MVP	**Descartado** — alto interés: la mayoría de las respuestas abiertas pide ver temas con más dudas y alumnos en riesgo, lo que mantuvo Analytics dentro del alcance.
>70% prioriza citar fuentes / objeciones fuertes a que los datos salgan a un proveedor externo de IA	No se preguntó explícitamente en la versión final del formulario — el requisito de citación ya estaba definido desde el diseño técnico, y no hay evidencia de objeción en las respuestas abiertas relevadas.

5.4 Contexto institucional relevado

En paralelo a las encuestas, el equipo relevó los requisitos técnicos y políticos que impone la UCC para instalar un plugin de terceros en su instancia institucional de Moodle (versión y SO del servidor, proceso formal de aprobación, restricciones de red, marco de privacidad aplicable). La mayoría de esas preguntas —dirigidas al área técnica de Moodle de la universidad— quedó sin respuesta confirmada dentro del plazo del proyecto. Por eso, desde una etapa temprana, el equipo priorizó una estrategia de despliegue que no dependiera de esa aprobación institucional: una demo autohospedada, independiente de la infraestructura de la UCC, que es efectivamente la que se usa durante el desarrollo y la que se describe en el Anexo D de este informe (backend permanente en Oracle Cloud; cada integrante corre Moodle localmente apuntando a ese backend). La sumisión del plugin al Moodle Plugins Directory, descripta en la sección *Publicación en Moodle Marketplace*, es en los hechos la vía que terminó reemplazando a la aprobación institucional directa: un plugin publicado y revisado por Moodle HQ es más fácil de autorizar para un área de IT que uno sin ningún tipo de validación externa.

5.5 Síntesis

Tabla 2. *Síntesis de la evidencia del relevamiento por problemática*

Problemática del Diagnóstico	Evidencia de la encuesta a estudiantes	Evidencia de la encuesta a docentes
Dispersión de la información	80,7% percibe dispersión al menos “a veces”; 43,4% la percibe alta o frecuente.	—
Uso pasivo / desorganizado del campus virtual	Solo 34% ve sus materias bien organizadas; depende del docente más que de la plataforma.	33% no usa las herramientas de evaluación del campus.
Recurso a IA generalista ajena al material del curso	100% de quienes vieron la pregunta ya usa IA para estudiar y/o trabajos; 95,5% quiere corrección automática con explicación.	64,7% ya usa IA en su práctica docente; 88,2% delegaría tareas a una IA integrada al campus.

6 Objetivos

6.1 Objetivo general

Desarrollar un plugin para Moodle que incorpore un asistente académico basado en inteligencia artificial, capaz de responder consultas en lenguaje natural sobre el contenido real de cada materia, generar ejercicios de práctica personalizados y proveer herramientas de *analytics* al docente, mejorando la experiencia educativa dentro del aula virtual de la UCC.

6.2 Objetivos específicos

- Identificar y documentar las problemáticas del uso actual del campus virtual de la UCC mediante una encuesta a estudiantes y otra a docentes, y cuantificar el porcentaje de encuestados afectados por cada problemática identificada.
- Diseñar e implementar un plugin de tipo `local` para Moodle que se integre al entorno existente sin modificar su núcleo y que sea instalable, sin dependencias externas, en las versiones de Moodle soportadas, verificado mediante integración continua (RF-15 a RF-18, RNF-10).
- Desarrollar un asistente conversacional basado en RAG que recupere el material pertinente con un Recall@5 de al menos 0,85 y un MRR de al menos 0,70, y que responda con un percentil 95 (p95) de latencia menor a 8 segundos (RF-01 a RF-07, RNF-07).
- Implementar un generador de cuestionarios de práctica de opción múltiple, con cuatro opciones por pregunta, retroalimentación inmediata y aviso cuando el tema pedido no figura en el material (RF-10 a RF-14).
- Proveer al docente un panel de analítica y un reporte de vacíos de contenido por curso, ordenado por frecuencia y filtrable por períodos de 7, 30, 90 y 365 días (RF-20 a RF-22).
- Evaluar la solución con pruebas automatizadas que cubran al menos el 70 % del código (RNF-16), pruebas de carga contra los acuerdos de latencia (RNF-06 a RNF-09) y una prueba de uso con estudiantes reales.

6.3 Tabla de trazabilidad

La siguiente tabla evidencia el cumplimiento de cada objetivo específico vinculándolo con los requerimientos implementados y la prueba que lo valida.

Tabla 3. *Trazabilidad entre objetivos específicos, requerimientos y evidencia de cumplimiento*

Objetivo específico	Requerimiento(s)	Evidencia de cumplimiento
Relevamiento de problemáticas	—	Encuestas a 166 estudiantes y 17 docentes de la UCC (sección 5): el 80,7 % de los estudiantes percibe dispersión de la información; el 88,2 % de los docentes delegaría tareas a una IA integrada al campus.
Plugin `local` integrado a Moodle	RF-15 a RF-18; RNF-10	Plugin instalable en Moodle 4.5 LTS a 5.3; CI sobre 7 combinaciones de versión y motor de base de datos (PostgreSQL y MySQL); 257 pruebas PHPUnit; aprobado y publicado en el Moodle Plugins Directory (versión 0.19.0).
Asistente conversacional RAG	RF-01 a RF-07	Recall@5 = 1,00 y MRR = 0,94 (sección 8.6.3); latencia p95 de 4,39 s con el LLM real (sección 8.6.4); pruebas `test_retriever` y `test_pipeline`.
Generación de cuestionarios	RF-10 a RF-14	Cuestionarios de opción múltiple con cuatro opciones y retroalimentación inmediata; aviso cuando el tema no figura en el material (RF-12); salida del LLM validada contra esquemas Pydantic.
Analytics y detección de vacíos	RF-20 a RF-22	Reporte de vacíos por curso con filtros de 7, 30, 90 y 365 días y panel de analítica; pruebas automatizadas del registro de vacíos.
Evaluación de la solución	RNF-06 a RNF-09, RNF-16	Cobertura del 83,6 % (sección 8.6.1); pruebas de carga con p95 = 4,39 s y 1.008 consultas sostenidas sin errores (sección 8.6.4); prueba de uso con 3 a 5 estudiantes (TEST-06), de alcance limitado.

La medición de la latencia hasta el primer *token* (RNF-06) y una evaluación de usabilidad con una muestra amplia de estudiantes quedan como trabajo futuro (sección 17).

7 Marco teórico

El presente marco teórico proporciona el fundamento conceptual necesario para comprender la problemática, sin plantear todavía una solución específica. El análisis abarca el contexto del problema, las técnicas disponibles y las alternativas existentes en el mercado.

7.1 Contexto general del problema

La incorporación de inteligencia artificial generativa a los entornos educativos ha sido señalada como una de las transformaciones más relevantes para la educación superior (UNESCO, 2023). Los modelos de lenguaje de gran escala (LLM) son capaces de sostener conversaciones en lenguaje natural, pero su conocimiento proviene de los datos con los que fueron entrenados, por lo que desconocen el material particular de una materia universitaria. Al consultarlos sobre dicho material, tienden a producir respuestas genéricas o, en el peor caso, **alucinaciones**: información plausible pero incorrecta o no fundamentada (Ji et al., 2023).

7.2 Técnicas investigadas

Para que un asistente responda sobre contenido específico de una materia existen tres estrategias principales:

Tabla 4. *Estrategias para que un asistente responda sobre contenido específico*

Estrategia	Costo	Flexibilidad	Limitación principal
Prompting puro	Bajo	Baja	El modelo no conoce el material; inventa o se rehúsa
Fine-tuning	Muy alto	Alta	Requiere reentrenar el modelo por cada curso
RAG	Medio	Media-alta	Requiere infraestructura de indexación y recuperación

Fuente: elaboración propia a partir de Lewis et al. (2020) y Gao et al. (2023).

La técnica de **Retrieval-Augmented Generation** (RAG) combina la búsqueda semántica sobre documentos propios con la generación de texto de un LLM (Lewis et al., 2020). Permite incorporar material nuevo sin reentrenar el modelo, recuperando los fragmentos más relevantes y proveyéndolos como contexto. Diversos trabajos posteriores han sistematizado sus variantes y su aplicación a tareas intensivas en conocimiento (Gao et al., 2023). La calidad del sistema depende de decisiones de

diseño como la estrategia de fragmentación (*chunking*), el modelo de *embeddings* y la métrica de similitud empleada.

En la etapa de recuperación, tanto la pregunta *q* como cada fragmento *d* se representan como vectores densos (*embeddings*), y la afinidad semántica entre ambos se estima con la similitud coseno (ecuación 1):

$$\text{sim}(\mathbf{q}, \mathbf{d}) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\| \, \|\mathbf{d}\|} \qquad \text{distancia}(\mathbf{q}, \mathbf{d}) = 1 - \text{sim}(\mathbf{q}, \mathbf{d}) \tag{1}$$

La extensión pgvector expone la distancia coseno mediante el operador `<=>`. Comparar la pregunta contra todos los vectores de un curso tiene un costo lineal en la cantidad de fragmentos; los índices de búsqueda aproximada de vecinos más cercanos reducen ese costo. Entre ellos, HNSW organiza los vectores en un grafo jerárquico de "mundo pequeño navegable" cuya búsqueda escala de forma aproximadamente logarítmica (Malkov y Yashunin, 2020); sus parámetros *m* (vecinos por nodo) y *ef_construction* (amplitud de la búsqueda durante la construcción) regulan el equilibrio entre exhaustividad, memoria y tiempo de indexación. El tamaño del índice depende de la dimensión de los vectores: con números de punto flotante de 4 bytes, un vector de 768 dimensiones ocupa unos 3 KB y uno de 1.536 dimensiones, unos 6 KB.

La búsqueda semántica puede complementarse con una puntuación léxica, como `ts_rank` de la búsqueda de texto completo de PostgreSQL (PostgreSQL Global Development Group, s.f.), que favorece las coincidencias exactas de siglas, nombres propios o términos técnicos poco frecuentes. Las estrategias híbridas combinan ambas señales mediante una suma ponderada, cuyos pesos son un parámetro de diseño.

Para evaluar la recuperación se utilizan métricas clásicas de recuperación de información (Manning et al., 2008). Para un conjunto *Q* de preguntas, siendo *R* el conjunto de fragmentos relevantes de una pregunta y *top-k* los *k* primeros resultados devueltos (ecuación 2):

$$\text{Recall@}k = \frac{|R \cap \text{top}_k|}{|R|} \qquad \text{Precision@}k = \frac{|R \cap \text{top}_k|}{k} \qquad \text{MRR} = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{\text{rank}_i} \tag{2}$$

donde $rank_i$ es la posición del primer resultado relevante para la pregunta *i*. La calidad de la respuesta generada se evalúa con criterios como la fidelidad (*faithfulness*), es

decir, que cada afirmación esté respaldada por el contexto recuperado; para medirla, la literatura propone tanto rúbricas humanas como la evaluación automática con un segundo modelo de lenguaje que actúa como juez (Es et al., 2024; Zheng et al., 2023).

Para la práctica de estudio, la repetición espaciada programa cada repaso cuando el recuerdo comienza a debilitarse. El sistema de Leitner (1972) agrupa las tarjetas en cajas con intervalos fijos y crecientes, mientras que el algoritmo SM-2 (Woźniak, 1990) ajusta para cada tarjeta un factor de facilidad según la calidad de la respuesta y calcula el próximo intervalo multiplicando el anterior por ese factor. El primero es más simple de implementar; el segundo adapta mejor los intervalos a la dificultad de cada contenido.

7.3 Tecnologías investigadas

Además de la técnica general (RAG), se investigó la pila tecnológica necesaria para implementarla, contrastando alternativas para cada componente.

7.3.1 Base de datos vectorial

Tabla 5. *Comparación de bases de datos vectoriales*

Alternativa	Características	Encaje con la escala del proyecto
pgvector (extensión de PostgreSQL)	Un solo sistema para datos relacionales y vectoriales; permite combinar filtros SQL y búsqueda vectorial en una sola consulta.	Pensado para integrarse a un backend que de todas formas necesita una base relacional, sin sumar una pieza de infraestructura más.
ChromaDB (ChromaDB, s.f.)	Sistema separado de PostgreSQL: las consultas relacionales y vectoriales requieren dos llamadas y lógica de coordinación adicional.	Agrega un segundo sistema a operar y mantener disponible.
Pinecone / Weaviate / Qdrant	Bases de datos vectoriales dedicadas, diseñadas para escalar a grandes volúmenes de vectores con alta concurrencia.	La escala estimada para la UCC (~240.000 vectores, hasta ~4,8 millones en una expansión a varias instituciones) queda por debajo de los volúmenes para los que se diseñan.

Fuente: elaboración propia a partir de pgvector (s.f.), ChromaDB (s.f.), Pinecone (s.f.), Qdrant (s.f.) y Weaviate (s.f.).

7.3.2 Modelos de lenguaje y *embeddings*

Se compararon el costo y las prestaciones de los modelos de lenguaje candidatos: Gemini 2.5 Flash ofrece una capa gratuita suficiente para un prototipo, pero sin garantías de nivel de servicio (Google, s.f.-b); GPT-4o-mini es significativamente más

económico que GPT-4o y suficiente para responder preguntas sobre un contexto dado, mientras que GPT-4o se reserva para razonamientos más complejos (OpenAI, s.f.-c). Prácticamente todos los proveedores relevantes, incluido Gemini, exponen una interfaz compatible con el SDK de OpenAI (Google, s.f.-b), lo que permite una abstracción común sin atarse a un proveedor específico; la alternativa es un *framework* de orquestación como LangChain (LangChain, s.f.), que ofrece componentes listos a cambio de una dependencia adicional con su propio ciclo de versiones.

En cuanto a los modelos de *embeddings*, `gemini-embedding-001` (Google, s.f.-a) y `text-embedding-3-small` (OpenAI, s.f.-b) permiten reducir la dimensión del vector de salida: el primero recomienda 768, 1.536 o 3.072 dimensiones y el segundo genera 1.536 por defecto. La dimensión incide en el almacenamiento y en el costo de cada comparación, pero los vectores de modelos distintos no son comparables entre sí: cambiar de modelo obliga a recalcular todos los *embeddings* del índice.

7.3.3 Arquitectura de plugin Moodle

Moodle admite varios tipos de plugin (`local`, `block`, `mod`, `filter`, `auth`, entre otros), cada uno con su propio ciclo de vida y nivel de integración con el núcleo (Moodle HQ, 2024). Un `block` se agrega curso por curso y, con el tema Boost de Moodle 4.x, queda oculto en un panel lateral, lo que limita su uso para un asistente que debe estar disponible en todo el curso. El subsistema de IA nativo, disponible desde Moodle 4.5, organiza la integración en *Placements* (dónde aparece la IA en la interfaz), *Providers* (conexión con APIs externas) y un *Manager* que media entre ambos, y expone tres acciones —`generate_text`, `generate_image` y `summarise_text`— sin una acción de conversación con historial persistente (Moodle HQ, s.f.-b). Un plugin de tipo `local`, en cambio, puede definir funciones externas (*web services*), tareas programadas y *hooks* propios sin requerir que el docente lo agregue a cada curso.

Para extender la interfaz y reaccionar al ciclo de vida de la plataforma, Moodle ofrece dos mecanismos: los *callbacks* por convención de nombre, históricos, y la API de *hooks* tipada incorporada en Moodle 4.3, en la que un plugin registra *listeners* en `db/hooks.php` (Moodle HQ, s.f.-d). Los *callbacks* se deprecan progresivamente, por lo que un plugin que soporta varias versiones de Moodle puede necesitar ambos mecanismos durante la transición.

Una cuestión adicional es cómo garantizar que un asistente use solo el material que cada usuario tiene permitido ver. Una alternativa es replicar en el servicio externo las reglas de visibilidad de la plataforma (actividades ocultas, restricciones de acceso, grupos), con el riesgo de que ambas copias diverjan; la otra es consultar a Moodle en cada solicitud —por ejemplo, con `cm_info::uservisible`— y enviar al servicio externo la lista de actividades visibles, a costa de un cálculo adicional por solicitud.

7.3.4 Extracción y fragmentación de documentos

Para la extracción de texto de los materiales se compararon pypdf, con extracción deficiente en PDFs de diseño complejo (pypdf, s.f.); pdfminer.six, de interfaz más compleja de integrar (pdfminer.six, s.f.); PyMuPDF, distribuido bajo licencia AGPL, que obliga a publicar bajo esa misma licencia el código de los servicios que lo usan (Artifex, s.f.); y pdfplumber, con mejor manejo nativo de tablas y diseño de página (pdfplumber, s.f.). Para la fragmentación (*chunking*), el tamaño del fragmento es en sí mismo un *trade-off*: los fragmentos pequeños mejoran la precisión de la recuperación semántica, mientras que cierto grado de superposición entre fragmentos consecutivos mitiga la pérdida de contexto cuando uno de ellos corta una oración o una idea a la mitad.

El tamaño de los fragmentos suele medirse en *tokens* del tokenizador del modelo (por ejemplo, la codificación `cl100k_base`) y no en caracteres, para acotar el costo y la longitud del contexto. Para documentos escaneados, que carecen de capa de texto, se requiere reconocimiento óptico de caracteres: Tesseract es un motor de OCR de código abierto con modelos para español e inglés (Smith, 2007), mientras que los servicios de OCR en la nube ofrecen mayor precisión en imágenes de baja calidad a cambio de enviar el documento a un tercero.

7.3.5 Estilos de arquitectura del backend

Para un backend que debe ejecutar varias responsabilidades (conversación, indexación de material, analítica) existen, en términos generales, dos estilos: un monolito organizado internamente por dominios o una arquitectura de microservicios, con un proceso independiente por responsabilidad (Lewis y Fowler, 2014). Los microservicios ofrecen escalado y despliegue independientes por componente, a costa de una complejidad operativa considerable: cada servicio necesita su propio *pipeline* de CI/CD, descubrimiento de servicios y manejo de fallas de red entre procesos que antes eran simples llamadas de función. En equipos reducidos y con

infraestructura acotada, esa complejidad compite con el tiempo disponible para desarrollar funcionalidad; un monolito organizado por dominios internos ofrece gran parte del orden que los microservicios buscan imponer, sin la sobrecarga de coordinar múltiples procesos desplegados por separado.

7.3.6 Autenticación servicio-a-servicio

Cuando dos servicios propios —no un usuario y un servicio— necesitan autenticarse entre sí sin mantener estado, las alternativas más comunes son HMAC (código de autenticación de mensajes basado en una función *hash* y una clave secreta compartida; Krawczyk et al., 1997) con marca de tiempo y *nonce*, JWT (Jones et al., 2015) y mTLS, la autenticación mutua con certificados sobre TLS (Rescorla, 2018). JWT está pensado sobre todo para delegar la identidad de un usuario entre sistemas —por ejemplo, para que un *frontend* pruebe ante un backend quién es el usuario autenticado—, un problema distinto al de dos servicios propios que ya confían en el canal que los conecta; HMAC resuelve la integridad y la autenticidad del mensaje con un secreto compartido, sin necesidad de un *token* con información de identidad. mTLS ofrece una garantía más fuerte a nivel de transporte, pero implica distribuir y rotar certificados en ambos extremos, una carga operativa comparable a la de los microservicios en equipos e infraestructuras pequeños.

Más allá de qué esquema de autenticación usar entre dos servicios propios, hay una pregunta previa: si el navegador del usuario final debe hablar directamente con el backend que orquesta el modelo de lenguaje, o si un intermediario ya autenticado por la plataforma —que actúa como *proxy*— es quien reenvía esa comunicación. Exponer el backend directo al navegador requiere resolver CORS y, sobre todo, decidir dónde vive la clave del proveedor de IA: si viaja al cliente, cualquier vulnerabilidad de *scripting* del lado del navegador puede filtrarla; un *proxy* autenticado evita ese riesgo porque la clave nunca sale del lado servidor, a costa de un salto de red adicional y de que el intermediario quede en el camino crítico de cada solicitud.

7.3.7 Transporte de respuestas en tiempo real

Para entregar la respuesta de un modelo de lenguaje *token* por *token*, en lugar de esperar a que esté completa, las opciones principales son *Server-Sent Events* (SSE; WHATWG, s.f.) y WebSockets (Fette y Melnikov, 2011); el *long-polling* agrega latencia en cada respuesta. WebSockets ofrece un canal bidireccional persistente, pero

requiere un cambio de protocolo HTTP (*upgrade*) que varios *proxies* y *firewalls* corporativos o universitarios bloquean o manejan de forma inconsistente. SSE, en cambio, es unidireccional (servidor → cliente) y corre sobre HTTP sin cambio de protocolo, lo que resulta suficiente cuando el cliente solo necesita recibir texto a medida que se genera, sin enviar datos por el mismo canal mientras la respuesta está en curso.

7.3.8 Infraestructura de despliegue

Para alojar un backend de este tipo pueden considerarse tres categorías de infraestructura: un PaaS gestionado (por ejemplo, Railway o Fly.io; Railway, s.f.; Fly.io, s.f.), que simplifica el despliegue a cambio de un costo recurrente y de menos control sobre el entorno; una IaaS con nivel gratuito permanente (por ejemplo, Oracle Cloud *Always Free*), que exige administrar el sistema operativo y los contenedores, pero no tiene costo mientras se respeten los límites del nivel gratuito (Oracle, s.f.); y la infraestructura propia de la institución, que elimina el costo de alojamiento, pero depende de un proceso de aprobación administrado por un tercero. En el caso de la UCC, ese proceso no pudo confirmarse durante el relevamiento institucional (sección 5.4), un factor de riesgo que las dos primeras alternativas no presentan.

7.3.9 Validación de datos en los límites del sistema

Para los datos que llegan a una API desde un cliente externo existen, en términos generales, dos posturas. Una validación permisiva acepta y procesa la solicitud aunque lleguen campos no declarados en el contrato —el comportamiento por defecto de herramientas como Pydantic (Pydantic, s.f.)—, lo que la hace más tolerante a cambios del lado del cliente, pero deja pasar en silencio parámetros inesperados, sea por un error de integración o por un intento deliberado de colar un campo no previsto. Una validación estricta, en cambio, rechaza con un error cualquier campo fuera del esquema declarado: sacrifica algo de flexibilidad a cambio de que cualquier desvío del contrato se note de inmediato, en el momento en que ocurre, en lugar de ejecutarse sin aviso.

Un problema análogo aparece en la salida de los modelos de lenguaje cuando se espera un formato estructurado, por ejemplo, un cuestionario en JSON. Pueden distinguirse tres niveles: pedir el formato solo en la instrucción, sin garantías; el modo JSON, que asegura un JSON sintácticamente válido pero no que respete el esquema

esperado; y la salida estructurada (*Structured Outputs*), que restringe la decodificación al esquema declarado (OpenAI, s.f.-e). Los dos primeros requieren validar la respuesta contra el esquema en el servidor; el tercero no está disponible de la misma forma en todos los proveedores compatibles con la API de OpenAI.

7.3.10 Control de consumo y moderación de contenido

Un servicio que invoca modelos de lenguaje cobrados por *token* necesita acotar el consumo por usuario. Los algoritmos de limitación de tasa más comunes son la ventana fija (un contador por intervalo, simple pero permisivo en los bordes de la ventana), la ventana deslizante (más precisa, con mayor costo de almacenamiento) y el balde de *tokens* (*token bucket*), que admite ráfagas acotadas y proviene del control de tráfico en redes (Tanenbaum y Wetherall, 2011). Un límite por cantidad de solicitudes no distingue entre una pregunta breve y una que consume miles de *tokens*; por eso suele complementarse con presupuestos de *tokens* por usuario y por período, que deben reservarse de forma atómica antes de invocar al proveedor para no excederse con solicitudes concurrentes.

Para filtrar contenido inapropiado existen dos enfoques: una API de moderación especializada, rápida y de bajo costo (OpenAI, s.f.-d), o la clasificación con el propio modelo de lenguaje mediante una instrucción específica, que no depende de un proveedor particular pero agrega latencia y costo. Ante una falla del moderador, el sistema puede bloquear la solicitud (*fail-closed*, más seguro) o dejarla pasar (*fail-open*, más disponible); la elección depende del riesgo aceptado en cada contexto.

Respecto de quién asume el costo del consumo, existen dos modelos. Con una clave de API institucional compartida, la organización centraliza el gasto, los límites y el control del uso. Con la clave propia de cada usuario (*bring your own key*, BYOK), cada persona conecta su cuenta con el proveedor y paga su propio consumo, que se factura por *token* (OpenAI, s.f.-c); el costo se traslada al usuario, pero el servicio debe almacenar cada clave de forma cifrada y separar los controles institucionales —límites, moderación— del consumo personal.

7.3.11 Frameworks de desarrollo web y empaquetado

Para el backend de un servicio que orquesta modelos de lenguaje, las alternativas habituales difieren en lenguaje, modelo de concurrencia y ecosistema. Django, en Python, ofrece una plataforma completa con ORM, administración y plantillas,

orientada a aplicaciones con interfaz propia (Django Software Foundation, s.f.); FastAPI, también en Python, es un *framework* asíncrono centrado en APIs que valida las solicitudes a partir de anotaciones de tipos y genera la especificación OpenAPI de forma automática (FastAPI, s.f.); y Express, sobre Node.js, es un *framework* mínimo de JavaScript con un amplio ecosistema de *middleware* (OpenJS Foundation, s.f.). El lenguaje incide en el acceso a las librerías de IA y de procesamiento de documentos, mayoritariamente desarrolladas en Python, y el soporte asíncrono, en la capacidad de atender muchas respuestas en *streaming* simultáneas.

Del lado del cliente, Moodle construye su interfaz con plantillas Mustache y módulos JavaScript AMD cargados mediante RequireJS (Moodle HQ, s.f.-g; Moodle HQ, s.f.-e). Una interfaz con mucho estado local, como un chat con historial y respuestas incrementales, puede construirse con esas herramientas nativas o con una librería de componentes como React (React, s.f.), que simplifica el manejo del estado, pero exige empaquetar el código —con herramientas como Webpack (Webpack, s.f.)— en un formato compatible con el cargador AMD de Moodle.

La accesibilidad de una interfaz web se evalúa contra las Pautas de Accesibilidad para el Contenido Web (WCAG) 2.2, que exigen, entre otros criterios, roles y nombres accesibles para los controles, operación completa por teclado, foco visible y respeto por la preferencia de movimiento reducido (World Wide Web Consortium, 2023).

7.3.12 Almacenamiento en memoria para estado efímero

Mecanismos como los *nonces* de un solo uso, los contadores de límites de uso o las cachés requieren guardar datos de corta vida con expiración automática. Pueden mantenerse en la memoria del propio proceso, lo que es rápido pero no se comparte entre procesos ni sobrevive a un reinicio; en la base de datos relacional, que agrega latencia y carga de escritura; o en un almacén clave-valor en memoria como Redis, que ofrece expiración por clave y operaciones atómicas, como `SET` con las opciones `NX` y `EX`, útiles para registrar un valor solo si todavía no existe (Redis, s.f.).

7.3.13 Proxy inverso, certificados TLS y monitoreo

Exponer un servicio por HTTPS requiere un *proxy* inverso que termine la conexión TLS y gestione los certificados. Las alternativas incluyen servidores como Nginx, que necesitan una herramienta adicional como Certbot para obtener y renovar certificados de Let's Encrypt (NGINX, s.f.; Electronic Frontier Foundation, s.f.), y servidores como

Caddy, que los obtienen y renuevan automáticamente mediante el protocolo ACME (Caddy, s.f.; Let's Encrypt, s.f.). Para la operación, el monitoreo puede resolverse con una pila de observabilidad completa —por ejemplo, Prometheus (Prometheus Authors, s.f.)—, con servicios externos de monitoreo o con alertas mínimas sobre eventos críticos; las primeras ofrecen más visibilidad a cambio de consumir recursos que, en servidores pequeños, compiten con la propia aplicación.

7.3.14 Estrategias de prueba e integración continua

La pirámide de pruebas propone una base amplia de pruebas unitarias, rápidas y aisladas, una capa intermedia de pruebas de integración entre componentes y una cantidad menor de pruebas de extremo a extremo y de aceptación con usuarios (Cohn, 2009). En sistemas con requisitos de latencia se suman las pruebas de carga, que simulan usuarios concurrentes y comparan percentiles de latencia —por ejemplo, el percentil 95 (p95), valor por debajo del cual se ubica el 95 % de las mediciones— contra un acuerdo de nivel de servicio; herramientas como k6 permiten declarar esos umbrales en el propio *script* (Grafana Labs, s.f.). La cobertura de código mide qué proporción del código ejecutan las pruebas y suele usarse como compuerta de calidad junto con el análisis estático (SonarSource, s.f.). La integración continua ejecuta estas verificaciones ante cada cambio del repositorio, y el despliegue continuo extiende la automatización hasta instalar la versión en un ambiente y verificarla con pruebas de humo (Humble y Farley, 2010).

Para diseñar casos de prueba de caja negra se usan técnicas como la partición en clases de equivalencia, el análisis de valores límite y las tablas de decisión; desde la caja blanca, la cobertura de sentencia y la de decisión miden qué parte del código ejecutan esos casos (Myers et al., 2011). El plan de pruebas que organiza estos niveles suele vincularse con un modelo de calidad de producto como ISO/IEC 25010, que define nueve características —entre ellas, adecuación funcional, eficiencia de desempeño, seguridad y mantenibilidad— para especificar y evaluar la calidad del software (International Organization for Standardization, 2023).

7.3.15 Seguridad de aplicaciones web y de modelos de lenguaje

El OWASP Top 10 reúne las categorías de riesgo más frecuentes en aplicaciones web, como el control de acceso roto, las fallas criptográficas, la inyección y el uso de componentes con vulnerabilidades conocidas (OWASP, s.f.-a). Entre ellas, el *cross-*

site scripting (XSS) es relevante cuando una aplicación inserta en la página contenido generado por terceros —por ejemplo, la respuesta de un modelo de lenguaje—, y se mitiga escapando o saneando el HTML con librerías como DOMPurify (Cure53, s.f.). Las aplicaciones basadas en LLM agregan riesgos propios, recopilados en el OWASP Top 10 para aplicaciones de LLM, entre los que se destaca la inyección de *prompt*: instrucciones maliciosas incluidas en la entrada del usuario o en documentos recuperados que intentan modificar el comportamiento del modelo (OWASP, s.f.-b). Las mitigaciones habituales combinan delimitar el contenido no confiable, reforzar las instrucciones de sistema y limitar las acciones que el modelo puede desencadenar.

7.3.16 Protección de datos personales y control de acceso

Un sistema que procesa consultas de estudiantes trata datos personales y queda alcanzado por la Ley 25.326 de Protección de los Datos Personales (Ley N.º 25.326, 2000) y, como referencia internacional, por el Reglamento General de Protección de Datos de la Unión Europea (Reglamento [UE] 2016/679, 2016). Entre sus principios se encuentran la minimización de los datos, la limitación de la finalidad y los derechos de acceso y de supresión. El RGPD distingue la seudonimización —datos que ya no identifican a la persona sin información adicional— de la anonimización; reemplazar un identificador por su *hash* criptográfico es una técnica habitual, pero resulta reversible por fuerza bruta cuando el espacio de identificadores posibles es pequeño, salvo que se combine con un valor secreto. En Moodle, estas obligaciones se canalizan mediante la Privacy API, que exige a cada plugin declarar qué datos personales almacena o envía a servicios externos e implementar su exportación y su borrado (Moodle HQ, s.f.-f).

El control de acceso basado en roles (RBAC) asigna permisos a roles y roles a usuarios, en lugar de asignar permisos individualmente (Sandhu et al., 1996). Moodle implementa este modelo mediante *capabilities*: permisos con nombre que se otorgan a roles como estudiante o profesor en un contexto determinado —sitio, categoría, curso o actividad— y que el código verifica antes de cada operación (Moodle HQ, s.f.-a).

7.3.17 Reconocimiento de voz

Para transcribir consultas habladas existen servicios comerciales en la nube y modelos abiertos. Whisper es un modelo de reconocimiento de voz multilingüe entrenado con supervisión débil a gran escala y distribuido con pesos abiertos

(Radford et al., 2023); puede ejecutarse en infraestructura propia, con requisitos de cómputo considerables, o a través de proveedores que lo ofrecen como API. La elección implica un compromiso entre costo, latencia y el envío del audio a un tercero.

7.3.18 Licencias de software libre

Las licencias de software libre se agrupan en dos familias. Las licencias con *copyleft*, como la GNU General Public License versión 3 (GPL v3), permiten usar, estudiar, modificar y redistribuir el código, a condición de que las versiones derivadas se distribuyan bajo la misma licencia (Free Software Foundation, 2007). Las licencias permisivas, como la licencia MIT, permiten la reutilización —incluso en productos cerrados— con la sola obligación de conservar el aviso de autoría (Open Source Initiative, s.f.). En el ecosistema de Moodle, todo plugin publicado en el directorio oficial debe distribuirse bajo una licencia compatible con la GPL (Moodle HQ, s.f.-h), lo que impide cobrar por el plugin en sí, aunque no por los servicios asociados, como el alojamiento, la instalación o el soporte.

7.4 Opciones similares en el mercado

Se relevaron asistentes académicos basados en IA existentes, con el fin de comprender el panorama actual y las carencias que el proyecto podría atender, distinguiendo dos grupos: asistentes académicos generales, ajenos al ecosistema Moodle, y plugins de Moodle que ya incorporan IA.

7.4.1 Asistentes académicos generales

- **Khanmigo (Khan Academy):** tutor de IA pedagógicamente pulido, con modo socrático y cuidado explícito de la privacidad de menores, pero responde sobre el contenido de Khan Academy —no sobre el material de cada materia— y no se integra con Moodle (Khan Academy, s.f.).
- **Coursera Coach:** asistente ya conectado a los videos y transcripciones del curso, pero cerrado al ecosistema Coursera, sin apertura a contenido externo (Coursera, s.f.).
- **NotebookLM (Google):** ofrece buen RAG con citas precisas, pero requiere que el usuario suba los documentos manualmente, uno por uno, y almacena los datos en servidores de un tercero (Google, s.f.-c).

- **Microsoft Copilot for Education:** fuerte integración con Teams, Word y OneNote, pero centrado en su propio ecosistema, sin integración nativa con Moodle (Microsoft, s.f.).
- **ChatGPT Edu:** máxima potencia del modelo base, pero sin integración con el LMS —el alumno tiene que pegar el contexto manualmente— y sin *analytics* para el docente (OpenAI, s.f.-a).

Tabla 6. *Comparación con asistentes académicos generales*

Producto	Integra con Moodle	RAG sobre material del docente	Autohospedado	Código abierto	Analítica docente
Khanmigo	No	No	No	No	parcial
Coursera Coach	No	sí, en Coursera	No	No	parcial
NotebookLM	No	sí, manual	No	No	No
Microsoft Copilot Edu	No	con plugins	No	No	parcial
ChatGPT Edu	No	No	No	No	No

Fuente: elaboración propia a partir de Khan Academy (s.f.), Coursera (s.f.), Google (s.f.-c), Microsoft (s.f.) y OpenAI (s.f.-a).

Ninguna de las alternativas de este grupo combina integración nativa con Moodle, RAG automático sobre el material del docente y despliegue autohospedado. Entre sus fortalezas se destacan la investigación pedagógica acumulada por Khanmigo, la potencia de los modelos de mayor tamaño en razonamiento complejo —cuya ventaja sobre modelos livianos como GPT-4o-mini pesa menos cuando la respuesta se basa en un contexto recuperado por RAG— y el catálogo de cursos propio de Khan Academy y Coursera.

7.4.2 Plugins de Moodle con inteligencia artificial

El panorama relevante no se limita a los asistentes académicos generales: incluye el ecosistema de plugins de Moodle que ya incorporan IA, por ser el entorno en el que se inserta el problema:

- **block_openai_chat (Bryce Yoder):** ofrece un chat con OpenAI en un bloque lateral y un campo "Source of Truth" donde el docente pega texto manualmente

como contexto, sin RAG sobre el material: para que el asistente conozca 200 páginas de apuntes, el docente debe pegarlas a mano (Moodle, s.f.-b).

- **block_ai_chat + local_ai_manager (BYCS):** arquitectura modular con subplugins para distintos proveedores (OpenAI, Azure, Anthropic) y soporte para varias instituciones, pero sin RAG sobre el material del curso: es un chat genérico conectado a un LLM (Moodle, s.f.-a; Moodle, s.f.-c).
- **block_terusrag:** uno de los pocos plugins de Moodle que implementa RAG sobre el material del curso (Moodle, s.f.-f).
- **local_ai_course_assistant (Saylor Academy):** el más completo de los cuatro —*widget* flotante con *streaming* SSE, soporte para varios proveedores, modo socrático y cumplimiento de GDPR y SOC 2—, pensado para instituciones globales, sin plan de estudio, analítica docente ni generación de evaluaciones (Saylor Academy, s.f.).

Tabla 7. *Comparación con plugins de Moodle que incorporan IA*

Característica	`block_openai_chat`	`block_ai_chat`	`block_terusrag`	`local_ai_course_assistant`
RAG sobre el material del curso	No	No	Sí	parcial
RAG autohospedado (datos en la institución)	No	No	No	parcial
Chat con historial	Sí	Sí	Sí	Sí
Respuesta progresiva (SSE)	parcial	parcial	No	Sí
Widget flotante en todas las páginas	No	No	No	Sí
Indexación automática de material	No	No	Sí	manual
Plan de estudio o cuestionarios con IA	No	No	No	No
Analítica docente	No	No	No	No
Respuesta honesta ante falta de información	No	No	No	parcial
Independencia del proveedor de LLM	No	Sí	No	Sí

Fuente: elaboración propia a partir de Moodle (s.f.-a, s.f.-b, s.f.-c, s.f.-f) y Saylor Academy (s.f.).

Ninguno de los cuatro combina indexación automática del material del curso, RAG autohospedado y herramientas de analítica para el docente en una misma solución.

8 Propuesta de solución

La propuesta de solución describe cómo se aborda el problema planteado, a partir de lo analizado en el marco teórico. Esta es la sección más técnica del trabajo.

NexusAI es un plugin de Moodle con asistente de IA que combina tres capas: un plugin de tipo `local` (Moodle HQ, 2024) dentro de Moodle, una interfaz de chat embebida y un backend que orquesta el pipeline RAG. La pieza diferencial es el **RAG auténtico**: el material que el docente sube a Moodle se indexa automáticamente y el asistente responde citando la fuente; si la pregunta no puede responderse con el material disponible, el sistema lo admite explícitamente en lugar de inventar.

8.1 Alcance funcional

El sistema delimita claramente lo que entra y lo que queda fuera de su alcance actual. El MVP (junio de 2026) fue el primer hito funcional; el desarrollo continuó después de esa entrega, ampliando el alcance original.

Funcionalidades del MVP (junio 2026):

- Asistente conversacional RAG con respuesta en *streaming* y citas trazables al fragmento de origen.
- Buscador semántico (recuperación pura, sin LLM) sobre el material del curso.
- Chat multi-curso opcional, con citas que indican la materia de cada fragmento.
- Gestión de material por parte del docente: carga de PDF, indexación automática asíncrona, visualización de estado y borrado en cascada.
- Generador de cuestionarios de opción múltiple a partir del material, con feedback inmediato.
- Panel de detección de vacíos de contenido (*gaps*) para el docente.

Funcionalidades incorporadas después del MVP:

- Foros con detección automática de posts similares o duplicados mientras el alumno escribe, resumen de hilos largos y sugerencia de respuesta con RAG para el docente.
- Panel de analítica docente con preguntas frecuentes agrupadas por tema, a partir de un registro anonimizado de las interacciones del chat.
- Generador de exámenes a partir del material del curso, exportable al formato GIFT nativo de Moodle.
- Modo Estudio ampliado: más tipos de pregunta (verdadero/falso, abiertas, flashcards), historial de errores persistido en el backend, y un plan de estudio personalizado que combina errores de práctica, vacíos de contenido detectados en el chat y el próximo examen del calendario del curso.
- Filtros de búsqueda por tipo de material y por unidad/sección del curso.
- Vista de calendario con próximos exámenes y entregas.
- Soporte de material en PPTX, XLSX, CSV, Markdown y HTML además de PDF, con reconocimiento óptico de caracteres (OCR) de respaldo para documentos escaneados.
- Rediseño de la interfaz (agosto de 2026): el chat pasó de un *dropdown* flotante a un panel lateral acoplado, y el panel del docente se unificó en un lenguaje visual consistente de tipo *dashboard*, con paneles de material, vacíos de contenido, preguntas frecuentes, generación de exámenes y búsqueda integrados bajo la misma navegación.
- Consulta por voz: el alumno puede dictar la pregunta, que se transcribe con el modelo Whisper large-v3 a través de Groq.
- Resúmenes automáticos de documentos, almacenados en caché para no regenerarlos en cada consulta.

- Activación de NexusAI por curso (desactivado por defecto) y respuestas restringidas al material que el usuario puede ver en Moodle.
- Controles de consumo: límites de consultas por minuto y por día, presupuestos de *tokens* por rol, moderación de contenido y registro del consumo y del costo de cada llamada a un modelo.

Fuera de alcance (excluido conscientemente): soporte multi-institución nativo, integración con mensajería externa, y contenido multimedia, entre otros (ver detalle en *Anexos*).

Los requerimientos funcionales (RF-01 a RF-22) y no funcionales (RNF-01 a RNF-21) que detallan el alcance del MVP se encuentran documentados en los Anexos.

El alcance del MVP se modeló con historias de usuario, cada una con criterios de aceptación en formato *Dado–Cuando–Entonces*. La tabla siguiente las resume y las vincula con los requerimientos del Anexo A.

Tabla 8. *Historias de usuario del MVP y requerimientos asociados*

HU	Historia de usuario	RF
HU-001	Como alumno, quiero abrir un chat desde mi curso y preguntar en lenguaje natural, para obtener respuestas inmediatas sobre el material de mi materia.	RF-01
HU-002	Como alumno, quiero ver de qué archivo y fragmento proviene cada respuesta, para verificar y profundizar en el material original.	RF-02, RF-04
HU-003	Como alumno, quiero ver la respuesta aparecer mientras se genera, para no esperar varios segundos sin leer nada.	RF-05
HU-010	Como docente, quiero subir archivos al curso desde una página dedicada, para que el asistente los use al responder.	RF-15 a RF-17
HU-011	Como docente, quiero que el sistema detecte si subo dos veces el mismo archivo, para no duplicar material indexado.	RF-19
HU-020	Como alumno, quiero buscar fragmentos del material sin esperar al LLM, para encontrar rápido en qué archivo está un tema.	RF-08, RF-09
HU-030	Como alumno, quiero generar un cuestionario de opción múltiple sobre el material, para repasar antes de un parcial.	RF-10, RF-11, RF-13, RF-14
HU-031	Como alumno, quiero que el sistema me avise si pido un cuestionario sobre un tema que no está en el material, para no recibir uno engañoso.	RF-12
HU-040	Como alumno, quiero ver mis conversaciones anteriores y retomar una, para continuar donde lo dejé.	RF-06
HU-050	Como alumno inscripto en varios cursos, quiero consultar el material de todas mis materias, para encontrar conexiones entre temas.	RF-07
HU-060	Como docente, quiero ver las preguntas que el material no pudo responder, para identificar qué temas agregar al curso.	RF-20 a RF-22

8.2 Diseño

8.2.1 Arquitectura del software

La arquitectura adoptada es **cliente-servidor en tres capas**, con un backend organizado como **monolito modular**. El navegador del estudiante nunca se comunica directamente con el backend: toda llamada pasa por el plugin de Moodle, que actúa como *proxy* autenticado (patrón *Hybrid PHP Proxy*). Este diseño resuelve tres cuestiones de manera simultánea: la clave de la API del LLM nunca llega al navegador, la protección ante CSRF queda cubierta por el mecanismo nativo de Moodle y se evitan los problemas de CORS, al existir una única comunicación de servidor a servidor. Como se observa en la Figura 1, el navegador solo se comunica con el plugin, que reenvía cada solicitud al backend; únicamente este último interactúa con los proveedores de LLM, *embeddings*, moderación y transcripción de voz.

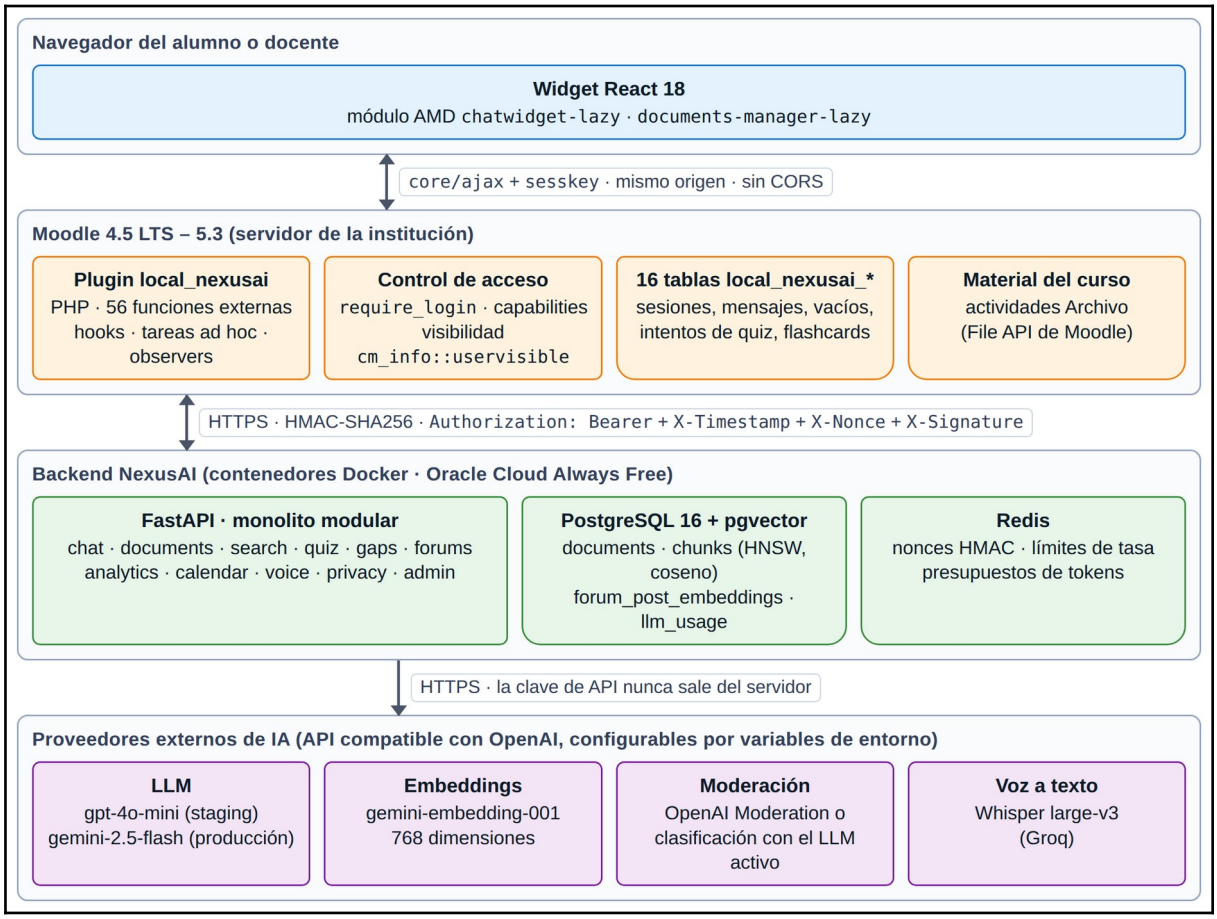


Figura 1. Arquitectura de componentes de NexusAI. *Fuente:* elaboración propia. *Texto alternativo:* cuatro capas apiladas —navegador, Moodle, backend y proveedores de IA— unidas por flechas que indican el protocolo entre cada par: `core/ajax` con `sesskey` entre el *widget* y Moodle, HTTPS con firma HMAC entre Moodle y el backend, y HTTPS con clave de API entre el backend y los proveedores.

8.2.2 Patrones de diseño

La tabla siguiente resume los patrones de diseño aplicados, dónde se implementan y qué problema resuelve cada uno.

Tabla 9. *Patrones de diseño aplicados en NexusAI*

Patrón	Implementación en NexusAI	Problema que resuelve
Proxy (*Hybrid PHP Proxy*)	`backend_client.php` y `chat_stream.php` reenvían al backend toda solicitud del navegador	Aísla la clave del proveedor de IA, reutiliza la sesión y la protección CSRF de Moodle y evita CORS.
Adaptador / Estrategia	`LLMProvider`, `EmbeddingProvider` y `TranscriptionProvider` sobre el SDK compatible con OpenAI	Permite cambiar de proveedor o de modelo con variables de entorno, sin modificar el código.
Inyección de dependencias	`Depends()` de FastAPI: `verify_hmac`, `get_db`, `get_redis`	Aplica autenticación y recursos compartidos de forma declarativa y permite sustituirlos en las pruebas.
Observador	*Observers* de eventos de Moodle (alta, modificación y borrado de material, foros y cursos)	Reacciona a los cambios de Moodle sin modificar su núcleo.
Hook (listener tipado)	`before_footer_html_generation`, `primary_extend`, `before_course_deleted`	Inyecta el *widget* y acompaña el ciclo de vida de Moodle con la API de *hooks* oficial.
Cola de tareas (*ad hoc tasks*)	`sync_material`, `sync_forum_post`, `purge_backend_data`	Saca las llamadas de red del flujo del evento y permite reintentarlas ante fallas.
Cache-aside	`document_summaries` y cachés en Redis	Evita regenerar resúmenes ya calculados, con el consiguiente ahorro de *tokens*.
Reintento con espera exponencial	`async_retry` en las llamadas a proveedores (espera de base · 2^{n-1})	Tolera fallas transitorias del proveedor de IA (HTTP 429 o 5xx).

Como se observa en la Figura 2, el núcleo del backend se organiza en *routers* que dependen del módulo de seguridad, módulos de recuperación e indexación y adaptadores de proveedores de IA, sobre las tablas `documents` y `chunks`.

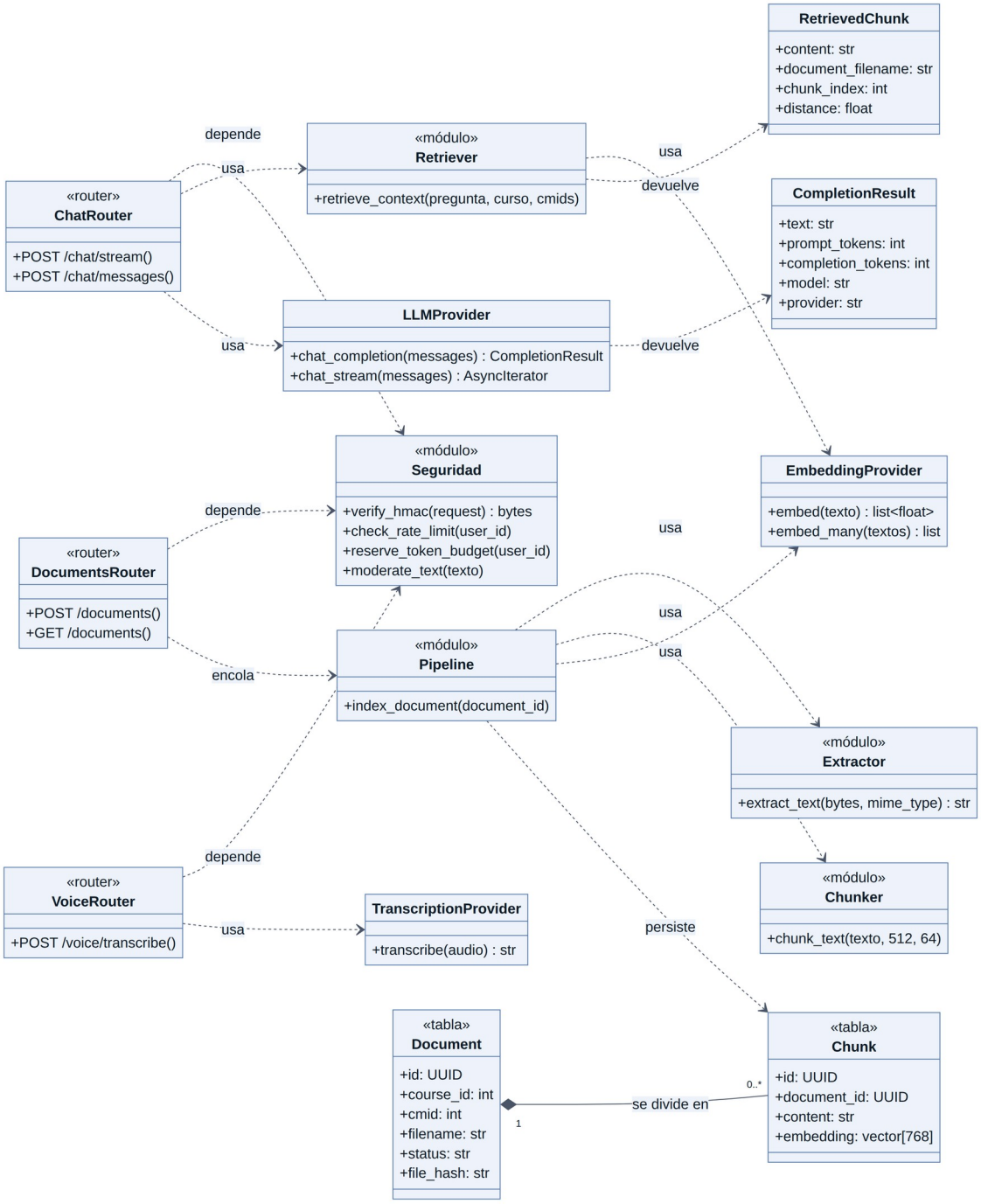


Figura 2. Diagrama de clases UML del núcleo del backend. *Fuente:* elaboración propia a partir del código del backend. *Texto alternativo:* los *routers* de chat, documentos y voz dependen del módulo de seguridad; el *router* de chat usa el recuperador y el proveedor de LLM; el *pipeline* de indexación usa el extractor, el fragmentador y el proveedor de *embeddings*, y persiste en `chunks`, que compone a `documents`.

8.2.3 Interfaz de usuario

La interfaz de chat se dispara desde un ícono en la barra de navegación de Moodle y se despliega como un panel lateral acoplado al borde derecho de la pantalla, de ancho

fijo y alto completo, en lugar del *dropdown* flotante de las primeras versiones. Presenta la respuesta en *streaming*, citas clickeables que expanden el fragmento utilizado con su porcentaje de similitud, e historial de conversaciones por sesión, todo dentro del mismo panel. En dispositivos móviles el panel se comporta como una hoja a pantalla completa (*bottom-sheet*).

Como se observa en la Figura 3a, cada respuesta se muestra junto a la fuente citada.

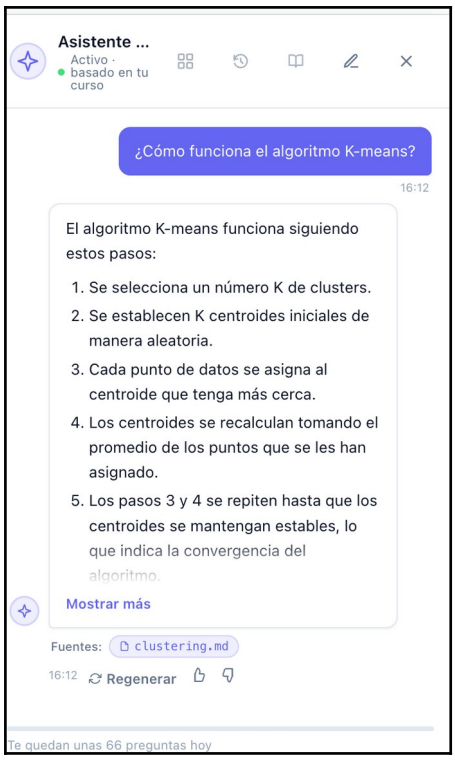


Figura 3a. Widget de chat respondiendo a una pregunta sobre K-means con cita a la fuente. *Fuente:* elaboración propia, captura de la versión actual desplegada localmente (panel lateral post-rediseño de agosto). *Texto alternativo:* una conversación donde el asistente explica el algoritmo K-means en pasos numerados, con un chip “Fuentes: clustering.md” debajo de la respuesta.

El panel docente ofrece, además, un buscador semántico sobre el mismo material indexado, como se observa en la Figura 3b.



Figura 3b. Buscador semántico del panel docente, con el término “K-means” resaltado dentro del resultado. *Fuente:* elaboración propia, captura de la versión actual desplegada localmente. *Texto alternativo:* un campo de búsqueda con filtros por tipo de archivo y unidad, devolviendo un fragmento de texto con dos coincidencias del término buscado resaltadas.

Desde el mismo panel, el docente gestiona el material indexado y su estado, según se muestra en la Figura 3c.

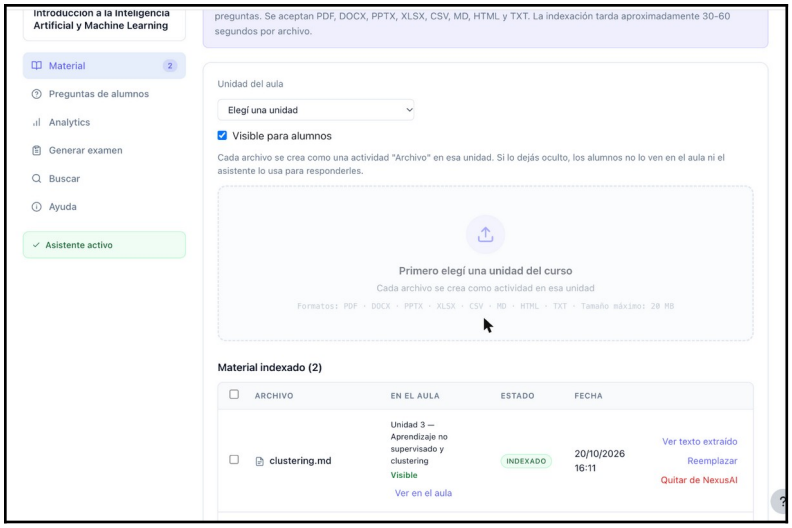


Figura 3c. Panel del docente: gestión de material indexado. *Fuente:* elaboración propia, captura de la versión actual desplegada localmente. *Texto alternativo:* la tabla de materiales subidos, con la unidad del curso asignada, el estado de indexación y si el material está visible para los alumnos.

El panel también expone las preguntas de los alumnos que quedaron sin respuesta, es decir, los vacíos de contenido detectados, como se observa en la Figura 3d.



Figura 3d. Panel del docente: preguntas de alumnos sin responder (vacíos de contenido). *Fuente:* elaboración propia, captura de la versión actual desplegada localmente. *Texto alternativo:* una pregunta real de alumno ("¿Qué es el clustering?") marcada como "sin match", es decir, sin una respuesta bien respaldada por el material indexado en ese momento.

8.2.4 Proceso de diseño de la interfaz

El panel lateral descrito es el resultado de dos iteraciones previas de diseño.

El primer concepto (*issue* FRONT-01/#158, 05/05/2026) fue un *dropdown* flotante disparado por un ícono, en lugar de un modal de pantalla completa, para que el alumno pudiera leer el material del curso y conversar con el asistente al mismo tiempo; se tomaron como referencia los *widgets* de Intercom, Crisp y ChatGPT. El *wireframe* definió cinco estados para la vista del alumno (cerrado, bienvenida, conversación, carga y error, este último con el mensaje real del backend —por ejemplo, un HTTP 503 — y la opción de reintentar) y cuatro para la vista docente (vacía, con una zona para arrastrar y soltar archivos; indexación, con una barra de progreso estimada, ya que el backend solo informa los estados `pending`, `indexing`, `indexed` y `error`; lista de materiales indexados; y configuración del plugin). De las seis decisiones de diseño documentadas con su *trade-off*, dos condicionan el resto del sistema: mostrar las fuentes citadas junto a cada respuesta, coherente con el principio de no inventar información, y la indexación asíncrona del material, que evita bloquear la interfaz del docente durante los 30 a 60 segundos que puede demorar el procesamiento de un PDF.

El panel lateral actual surgió de un rediseño posterior (rds-01 a rds-07, 10 al 12/08/2026, *issue* #400). El *dropdown* original, redimensionable desde sus cuatro bordes, se reemplazó por un panel acoplado al borde derecho y de alto completo, que

reduce el ancho del contenido de la página de Moodle, con un comportamiento de respaldo cuando el tema activo no expone el selector esperado o la ventana es angosta. Se agregó un riel colapsado para reabrirlo y se eliminó el cierre al hacer clic fuera del panel, ya que un panel acoplado convive con el contenido del curso. En la misma semana se rediseñaron, con el mismo criterio, el historial de conversaciones —que pasó de una capa superpuesta a una vista del propio panel— y la navegación del panel docente, de pestañas horizontales a un menú lateral.

En paralelo (28 al 30/07/2026), el equipo documentó una capa de pulido visual sobre componentes de React que ya funcionaban, sin cambios de lógica: instrucciones (*prompts*) para v0.dev, la herramienta de Vercel que genera interfaces a partir de una descripción (v0, s.f.). Los cinco *prompts* trabajados (*widget* de chat, plan de estudio personalizado, *quiz* del modo estudio, buscador semántico y panel docente) comparten un mismo sistema de diseño, tomado de la hoja de estilos del *widget*: color primario `#6366f1`, radios de 4 a 16 píxeles, iconografía de trazo simple sin emojis y referencias estéticas a Linear, Vercel y v0.dev. Esos *prompts* fijaban para el *dropdown* de entonces un tamaño de 400 × 600 píxeles en escritorio y una hoja a pantalla completa en dispositivos móviles; el rediseño de agosto conservó este último comportamiento y reemplazó el tamaño fijo de escritorio por el panel acoplado de alto completo.

Las Figuras 3a a 3d de esta sección son capturas de la versión actual del *widget*, posterior al rediseño de agosto recién descrito.

8.2.5 Justificación de las decisiones frente a las alternativas

Frente a los tipos de plugin y al subsistema de IA nativo descritos en la sección 7.3.3, se adoptó un plugin de tipo `local`. Un `block` obligaría a cada docente a agregarlo curso por curso y queda oculto en el tema Boost, y el subsistema de IA de Moodle 4.5 solo expone acciones atómicas, sin conversación con historial persistente; además, no existía en las versiones que el proyecto debía soportar al momento de diseñarse. Un plugin autocontenido, que no depende de la maduración de ese subsistema, resultó más adecuado para el caso de uso.

La tabla siguiente sitúa a NexusAI frente a las alternativas relevadas en la sección 7.4.

Tabla 10. *Posicionamiento de NexusAI frente a las alternativas relevadas*

Característica	Alternativas relevadas que la ofrecen (Tablas 6 y 7)	NexusAI
Integración nativa con Moodle	Plugins de Moodle de la Tabla 7	Sí
RAG automático sobre el material del curso	Terus RAG; parcial en SOLA	Sí
Base vectorial autohospedada	Parcial en SOLA	Sí
Analítica para el docente	Ninguna	Sí
Cuestionarios y plan de estudio con IA	Ninguna	Sí
Respuesta honesta ante falta de información	Parcial en SOLA	Sí
Independencia del proveedor de LLM	AI Chat (BYCS) y SOLA	Sí

El principal riesgo competitivo es que Moodle HQ incorpore al subsistema nativo una acción de chat con historial. La mitigación considerada es que el diferencial de NexusAI no se limita al chat: incluye el modo estudio, la analítica docente, la generación de exámenes y los foros asistidos por IA, funcionalidades que exceden lo que ese subsistema expone. Al mismo tiempo, NexusAI no supera a todas las alternativas: Khanmigo acumula años de investigación pedagógica que el proyecto recién comienza a construir; los modelos livianos que utiliza NexusAI rinden por debajo de los de mayor tamaño en razonamiento complejo; y las plataformas con catálogo propio no dependen, como NexusAI, de que cada docente suba su material.

8.3 Implementación

La solución se construyó de manera incremental, dividida en módulos lógicos:

Módulo de asistente conversacional (chat): orquesta el pipeline RAG y el *streaming* de respuestas.

Módulo de gestión de documentos: extracción de texto, fragmentación, generación de *embeddings* e indexación asíncrona.

Módulo de búsqueda semántica: recuperación de fragmentos por similitud.

Módulo de cuestionarios (*quiz*): generación de preguntas de opción múltiple.

Módulo de detección de vacíos (*gaps*): registro de consultas no resueltas y reporte agregado para el docente.

Módulo de seguridad: autenticación HMAC SHA-256 en tres capas (clave de API, firma y *nonce* anti-replay).

Después del MVP se sumaron los módulos `forums`, `analytics`, `calendar`, `courses`, `voice`, `privacy`, `admin` y `migration`, hasta completar los trece *routers* que la API expone bajo el prefijo `/api/v1`. El diagrama de clases del núcleo del backend se presenta en la sección 8.2.2 (Figura 2).

El pipeline de indexación procesa los documentos de forma asíncrona: extrae el texto según el formato, lo divide con una ventana deslizante de 512 *tokens* (codificación `cl100k_base`) y un solapamiento de 64 *tokens* (12,5 %), genera un *embedding* por fragmento y lo almacena en PostgreSQL con la extensión **pgvector** (pgvector, s.f.), junto con una columna `tsvector` para la búsqueda léxica. El estado del documento avanza de `pending` a `indexing` y finaliza en `indexed` o `error`; si falla el *embedding* de un fragmento aislado, este se conserva sin vector, y el documento solo se marca con error cuando fallan todos.

Como se observa en la Figura 4, la secuencia completa de una consulta del estudiante atraviesa autenticación, limitación de tasa, búsqueda vectorial y generación de la respuesta en *streaming*.

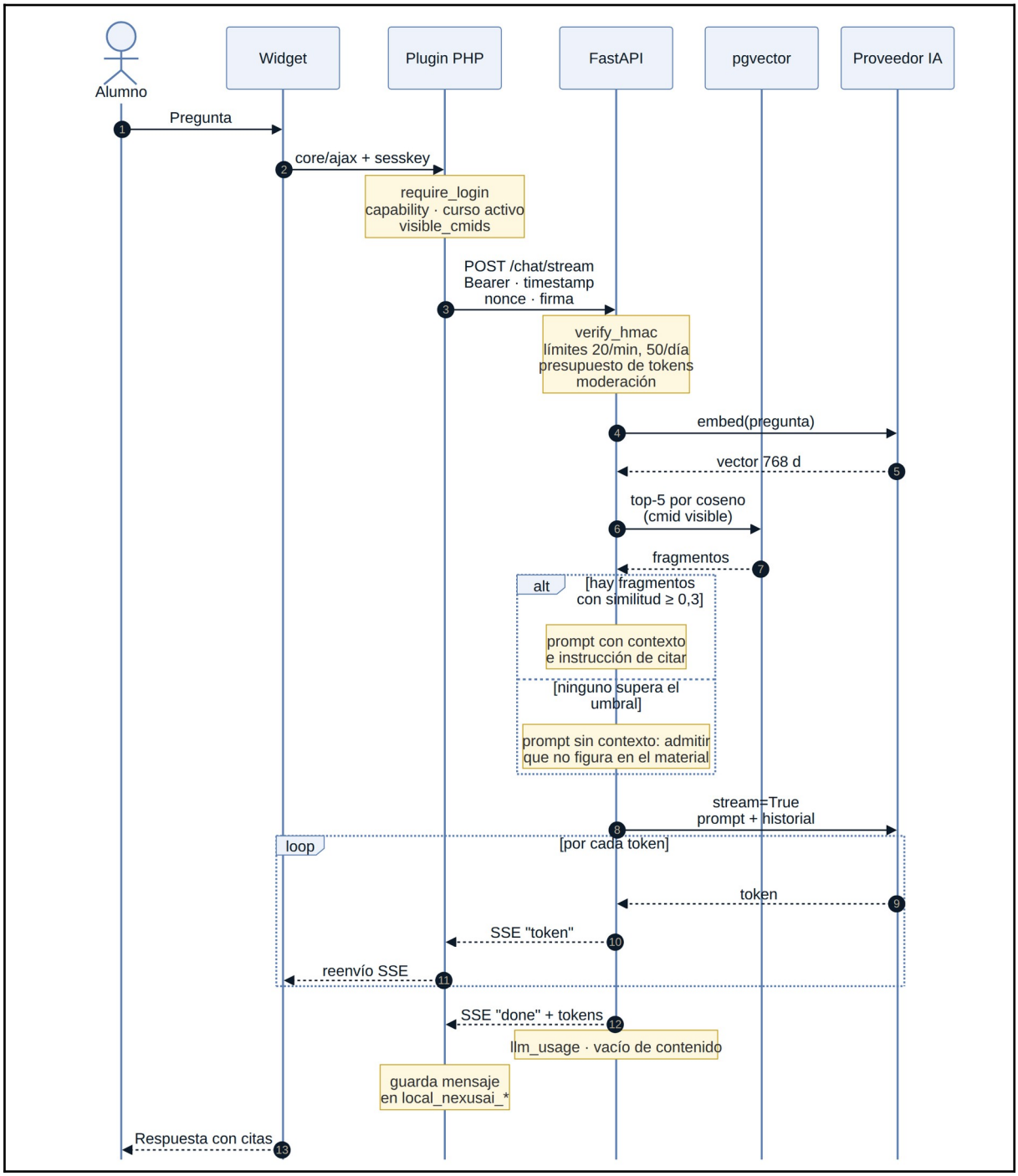


Figura 4. Diagrama de secuencia de una consulta del estudiante en modo *streaming*. *Fuente:* elaboración propia. *Texto alternativo:* secuencia entre el alumno, el *widget*, el plugin PHP, el backend, la base vectorial y el proveedor de IA: validaciones de Moodle, firma HMAC, límites de consumo, *embedding* de la pregunta, búsqueda de los cinco fragmentos más similares, construcción del *prompt* con o sin contexto según el umbral de similitud y generación token por token, reenviada como eventos SSE.

El pipeline de indexación se ejecuta de forma asíncrona y previa a cualquier consulta: la Figura 5 muestra el flujo completo, desde que el docente sube un documento hasta que queda disponible para la búsqueda vectorial.

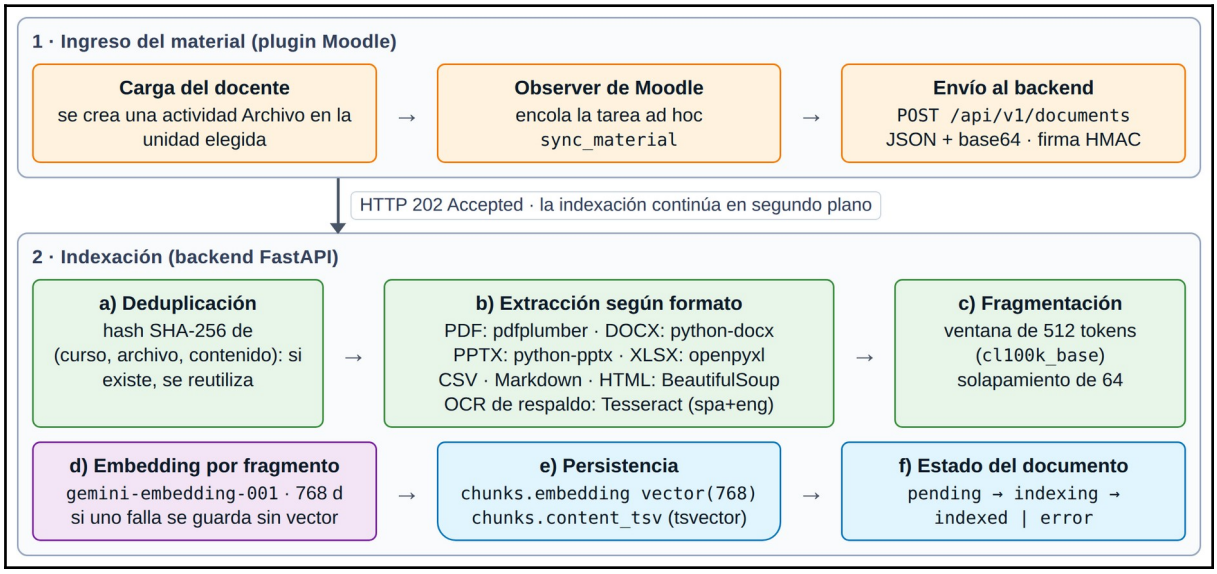


Figura 5. Pipeline de ingreso e indexación del material. *Fuente:* elaboración propia. *Texto alternativo:* dos etapas; en el plugin, la carga del docente crea una actividad Archivo cuyo *observer* encola una tarea que envía el archivo al backend; en el backend, el documento se deduplica por *hash*, se extrae su texto según el formato, se fragmenta en ventanas de 512 *tokens*, se calcula un *embedding* por fragmento y se persiste en la tabla `chunks`.

La tabla siguiente reúne los parámetros efectivos del pipeline RAG, tal como están configurados en el código del backend.

Tabla 11. *Parámetros efectivos del pipeline RAG*

Parámetro	Valor	Efecto
Tokenizador y fragmento	`cl100k_base`; 512 *tokens* con solapamiento de 64	Acota el tamaño del contexto y preserva la continuidad entre fragmentos.
Modelo de *embeddings*	`gemini-embedding-001`, 768 dimensiones (configurable)	Define la columna `vector(768)` de `chunks` y `forum_post_embeddings`.
Índice vectorial	HNSW, `vector_cosine_ops`, *m* = 16, *ef_construction* = 64	Búsqueda aproximada por distancia coseno, con los valores por defecto de pgvector.
Recuperación para el chat	*top-k* = 5; similitud mínima de 0,3 (distancia ≤ 0,7)	Sin fragmentos sobre el umbral, el *prompt* indica admitir que el tema no figura en el material y se registra un vacío de contenido.
Filtro de autorización	curso, `status = indexed` y `cmid` incluido en `visible_cmids`	Solo se usa material de actividades visibles para el usuario.
Buscador semántico	puntaje híbrido (ecuación siguiente); candidatos con similitud ≥ 0,32 o coincidencia léxica	Mejora la búsqueda de siglas y términos exactos.
Límites de uso	20 consultas por minuto y 50 por día por usuario; presupuesto de *tokens* por rol	Contiene el costo y evita abusos (HTTP 429).
Modelo de lenguaje	Configurable por ambiente: `gpt-4o-mini` en *staging* y `gemini-2.5-flash` en producción (22/10/2026)	Verificado con el *endpoint* `/health` de cada ambiente.

El puntaje del buscador semántico combina las dos señales descritas en el marco teórico (ecuación 3):

$$\text{score}(q,d) = 0{,}6 \cdot \text{sim}(q,d) + 0{,}4 \cdot \text{ts_rank}(q,d) \tag{3}$$

8.3.1 Funcionalidades post-MVP

Los seis módulos descritos son los que entregó el MVP de junio de 2026. El desarrollo continuó organizado en épicas, cuyas funcionalidades ya se enumeraron en el alcance funcional; esta sección muestra cómo se ven en uso, con capturas del entorno desplegado localmente, como se observa en las Figuras 6a y 6b.

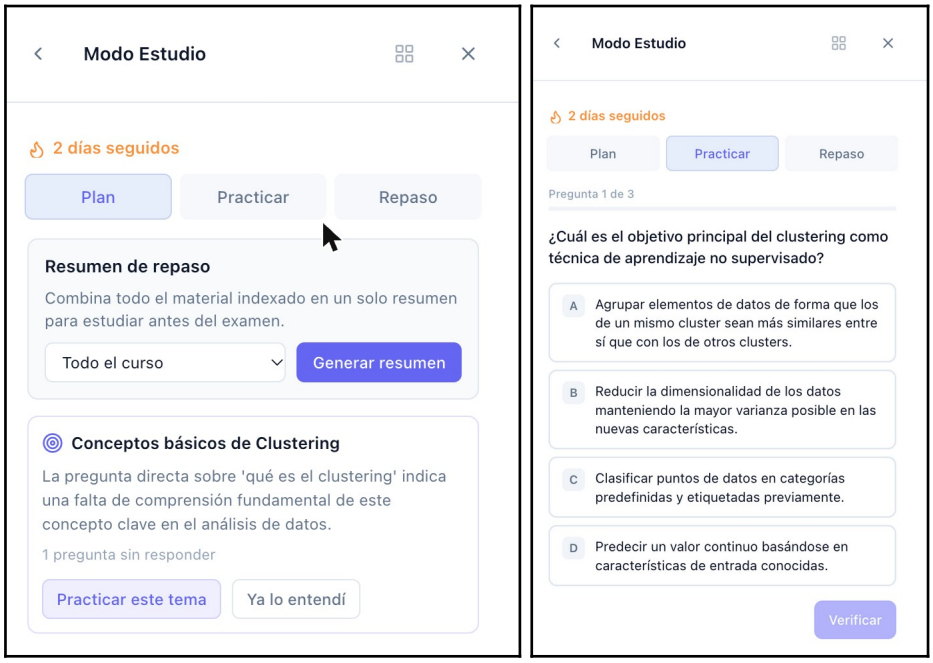


Figura 6a/6b. Modo Estudio: plan de repaso (izquierda) y *quiz* de práctica (derecha). *Fuente:* elaboración propia, capturas de la versión actual desplegada localmente. *Texto alternativo:* a la izquierda, una recomendación de repaso generada a partir de un vacío de contenido detectado, con botones "Practicar este tema" y "Ya lo entendí"; a la derecha, una pregunta de opción múltiple de un *quiz* de práctica, con cuatro alternativas y un botón "Verificar".

El plan de repaso cruza dos fuentes: los vacíos de contenido detectados en el chat y los errores cometidos en *quizzes* anteriores. Sobre el mismo *quiz* de práctica se agregaron luego *flashcards* con repetición espaciada según el algoritmo SM-2 (SP-11/#315, 07/09/2026), que no se muestran aquí porque no tienen una pantalla propia distinta del *quiz*. El calendario y el resumen semanal del foro sí tienen pantalla propia, como se observa en las Figuras 7a y 7b.

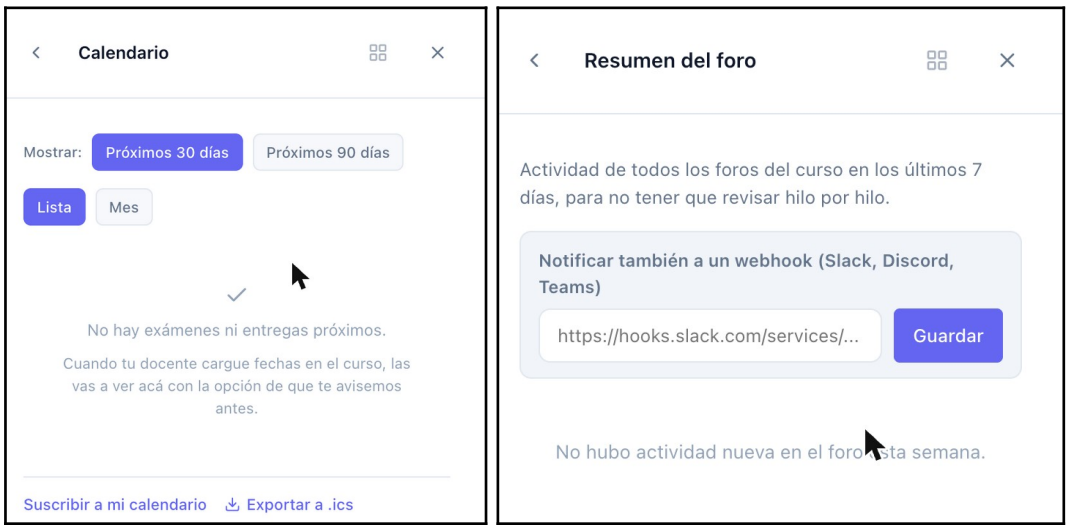


Figura 7a/7b. Calendario de próximas entregas (izquierda) y resumen semanal del foro (derecha). *Fuente:* elaboración propia, capturas de la versión actual desplegada localmente. *Texto alternativo:* a la izquierda, un estado sin exámenes ni entregas próximas, con opciones de suscribirse al calendario o exportarlo como `.ics`; a la derecha, un resumen sin actividad nueva en el foro esa semana, con un campo para notificar también a un *webhook* externo.

El calendario (CAL-02) expone las fechas del curso como un *feed* `.ics` al que el alumno puede suscribirse; la revisión del Moodle Plugins Directory detectó luego que ese *feed* incluía usuarios suspendidos y actividades no visibles, lo que se corrigió en la versión 0.19.0 (sección 11). El resumen del foro surgió junto con los foros asistidos por IA y luego incorporó la opción de reenviar el resumen semanal a un *webhook* de Slack, Discord o Teams (FOR-07/#378, 08/09/2026), para que el docente no necesite ingresar a Moodle para leerlo.

El mismo panel docente agrupa automáticamente las preguntas de los alumnos por tema, como se observa en la Figura 8.

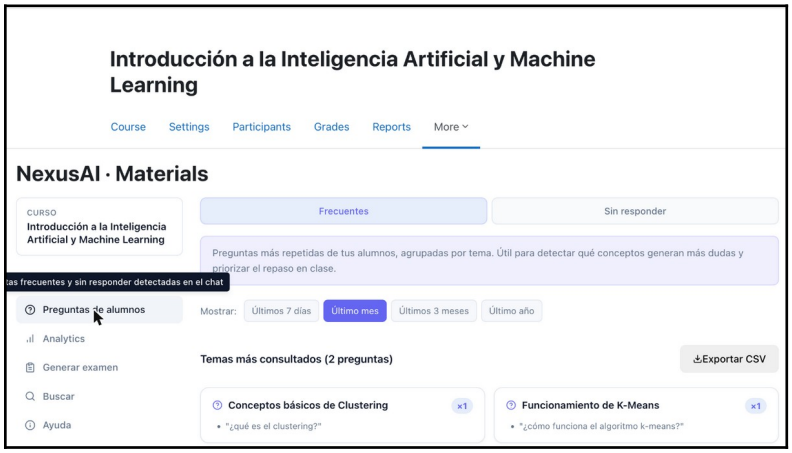


Figura 8. Panel docente: preguntas de alumnos agrupadas por tema. *Fuente:* elaboración propia, captura de la versión actual desplegada localmente. *Texto alternativo:* dos temas detectados a partir de preguntas reales de alumnos, cada uno con la cantidad de veces consultado y un ejemplo de la pregunta original.

El panel de *Analytics* reúne estas y otras métricas de uso del curso en una sola vista, como se observa en la Figura 9.

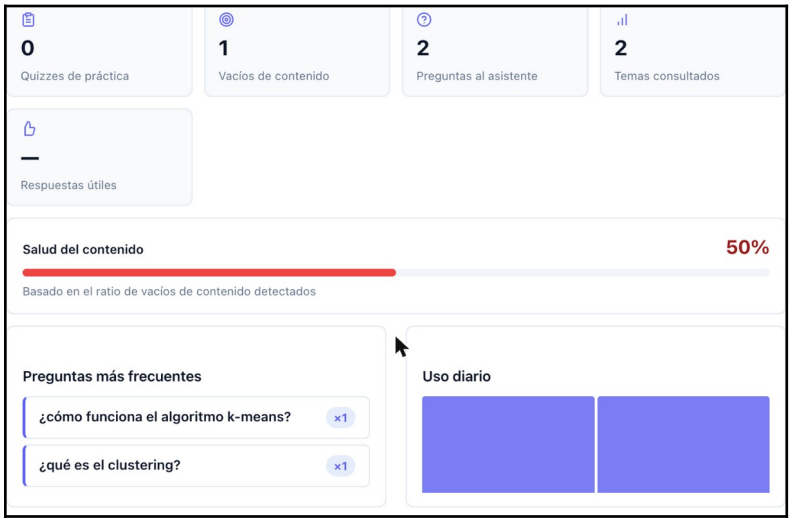


Figura 9. Panel docente: *Analytics*. *Fuente:* elaboración propia, captura de la versión actual desplegada localmente. *Texto alternativo:* métricas de uso del curso —*quizzes* de práctica, vacíos de contenido, preguntas al asistente, temas consultados—, un indicador de "salud del contenido" y la lista de preguntas más frecuentes.

El generador de exámenes completa el panel docente, como se observa en la Figura 10.

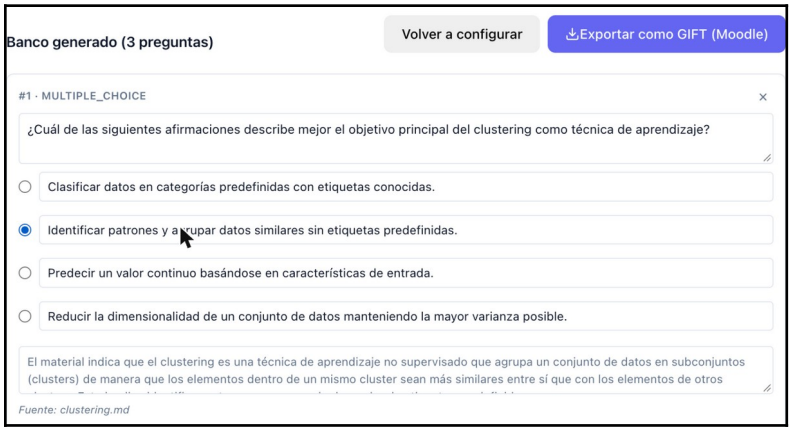


Figura 10. Panel docente: generador de exámenes. *Fuente:* elaboración propia, captura de la versión actual desplegada localmente. *Texto alternativo:* una pregunta de opción múltiple generada a partir del material del curso, con la fuente citada debajo y un botón para exportar el banco completo en formato GIFT de Moodle.

Las preguntas frecuentes de la Figura 8 comparten el backend de analítica de la Figura 9 (DOC-D01): el texto de cada pregunta se normaliza y se agrupa primero, y recién después un LLM asigna un nombre al tema resultante. El conteo de cuántas veces se preguntó cada tema se calcula siempre en Python y nunca se delega al LLM, con el mismo criterio de no inventar información que gobierna al asistente. El generador de

exámenes (EVAL-01/#235) pide al docente que elija los archivos de origen en lugar de usar todo el material del curso, y resuelve la exportación al formato GIFT en el *frontend*.

8.4 Arquitectura técnica del backend

Las Figuras 1 y 4 muestran la arquitectura y el flujo de una consulta a nivel de componentes y de secuencia; esta sección describe el mecanismo implementado detrás de cada pieza.

8.4.1 Autenticación HMAC en tres capas

El plugin y el backend son dos servicios independientes, y la clave del proveedor de IA reside solo en el backend. Por eso el navegador nunca se comunica directamente con FastAPI: todas las solicitudes pasan por el *proxy* PHP del plugin, lo que evita exponer la clave en el cliente, elimina la necesidad de CORS y reutiliza la sesión ya autenticada de Moodle. Del lado PHP se usa la clase `\curl` de Moodle en lugar de `curl_init()` o `file_get_contents()`: respeta la configuración de *proxy* de la institución y sus listas de *hosts* y puertos bloqueados, valida el certificado TLS del backend y no sigue redirecciones. Como esas listas suelen habilitar solo los puertos 80 y 443, el backend se publica únicamente en el puerto 443.

Del lado del servidor, la verificación se implementa como una dependencia de FastAPI (`verify_hmac`) que cada *endpoint* declara explícitamente, en lugar de un *middleware* global; así, `/health` y la documentación de la API quedan públicas sin excepciones especiales. La dependencia aplica cuatro controles en orden: (1) la comparación de la clave `Bearer`; (2) la ventana temporal, que rechaza solicitudes cuyo `X-Timestamp` difiera en más de 300 segundos del reloj del servidor; (3) la firma del mensaje, calculada como un HMAC (Krawczyk et al., 1997) sobre la concatenación de la marca de tiempo, el *nonce* y el cuerpo de la solicitud (ecuación 4):

$$\text{X-Signature} = \text{HMAC-SHA256}\big(K,\ timestamp \,\|\, nonce \,\|\, body\big) \tag{4}$$

donde K es el secreto compartido; y (4) el *nonce*, que se registra en Redis con `SET NX` y una expiración igual a la ventana, de modo que una segunda solicitud con el mismo *nonce* se rechaza con HTTP 401 aunque su firma sea válida. Las comparaciones de la clave y de la firma usan `hmac.compare_digest`, de tiempo constante, para evitar ataques de temporización. La firma no incluye el método ni la ruta HTTP; el *nonce* de

un solo uso impide igualmente reutilizar una solicitud capturada, y agregar ambos campos a la cadena firmada queda como mejora posible. Además de JWT y mTLS —descartados en ADR-005—, se descartó exponer el backend directamente al navegador, por el riesgo de que la clave del proveedor de IA quedara visible en el cliente.

8.4.2 Esquema de base de datos: alcance vigente y migración de dimensión

El Anexo B documenta el modelo de datos vigente. Por ADR-002 y ADR-014, PostgreSQL con pgvector almacena el índice del material (`documents` y `chunks`), las copias seudonimizadas de los *posts* de foro (`forum_post_embeddings`), los resúmenes en caché y el registro de consumo; los datos personales del alumno residen en tablas nativas de Moodle. La columna `embedding` se declara como `vector(n)`, con *n* tomado de la configuración (768 por defecto), y se indexa con HNSW sobre distancia coseno con *m* = 16 y *ef_construction* = 64. Como los vectores de modelos distintos no son comparables, un cambio de modelo de *embeddings* —por ejemplo, hacia uno de 1.536 dimensiones— requiere un procedimiento explícito: eliminar el índice HNSW, modificar la dimensión de la columna, recrear el índice y volver a encolar todos los documentos para que el pipeline de la Figura 5 recalcule sus vectores.

8.4.3 Streaming de respuestas: *Server-Sent Events*

La Figura 4 muestra el flujo de una consulta en tiempo real. Se eligió *Server-Sent Events* en lugar de *WebSockets* porque el canal es unidireccional (servidor → cliente), suficiente para un chat; funciona sobre HTTPS en el puerto 443 sin cambio de protocolo; y varios *proxies* y *firewalls* universitarios bloquean o manejan mal el *handshake* de WebSocket (WHATWG, s.f.). El *endpoint* `POST /api/v1/chat/stream` devuelve una `StreamingResponse` de FastAPI con tipo `text/event-stream`, alimentada por un generador asíncrono que emite eventos JSON con un campo `type`: `meta` (una vez, con la sesión y la cantidad de fragmentos recuperados), `token` (por cada porción de texto generada), `answer_meta` (las fuentes citadas), `done` (con los *tokens* consumidos) y `error`. Del lado de Moodle, `chat_stream.php` reenvía el flujo sin acumularlo, mediante la clase `\curl` con una función de escritura (`CURLOPT_WRITEFUNCTION`) que lee el código HTTP dentro del propio *callback*; así, la

respuesta llega token por token y errores como un HTTP 429 también alcanzan al *widget*. La latencia medida se informa en la sección 8.6.4.

8.4.4 Contratos de datos: *schemas* Pydantic

Las solicitudes y respuestas del backend se describen con modelos de Pydantic V2 (Pydantic, s.f.), que FastAPI usa para validar tipos, rangos y longitudes de cada campo y para generar el esquema OpenAPI. Los modelos usan la configuración por defecto, que ignora los campos no declarados en lugar de rechazarlos; la variante estricta descrita en el marco teórico (`extra='forbid'`) queda como endurecimiento posible del contrato. La salida de los modelos de lenguaje que debe tener estructura —preguntas de *quiz*, exámenes y temas de analítica— se solicita en modo JSON (`response_format={"type": "json_object"}`), que garantiza un JSON sintácticamente válido pero no su forma; por eso el backend valida esa salida contra modelos Pydantic (`QuizQuestion`, `QuizResponse`) y, si no coincide, responde con HTTP 503 en lugar de presentar un cuestionario incompleto o inventado. La alternativa más estricta, *Structured Outputs*, restringe la decodificación del modelo al esquema, pero no está disponible de la misma forma en todos los proveedores compatibles con la API de OpenAI que el sistema admite.

8.4.5 Ciclo de vida y estado de la aplicación

FastAPI reemplazó los *hooks* `@app.on_event("startup")` y `@app.on_event("shutdown")` por un administrador de contexto asíncrono `lifespan` (FastAPI, s.f.). En NexusAI, el arranque registra la versión y el modelo configurado y ejecuta `_recover_interrupted_documents()`, que marca con error los documentos que quedaron en `pending` o `indexing` si el contenedor se reinició durante una indexación; sin esa limpieza, esos registros bloquearían una nueva carga del mismo archivo y figurarían en cola indefinidamente en la interfaz docente. Al apagarse, la aplicación cierra la conexión con Redis y libera el *pool* de conexiones de SQLAlchemy. Las sesiones de base de datos y el cliente de Redis se obtienen por inyección de dependencias (`get_db`, `get_redis`), y un `RequestIDMiddleware` asigna a cada solicitud un identificador que se propaga a los registros y al consumo de *tokens*.

8.5 Arquitectura técnica del plugin Moodle

Esta sección completa, del lado del plugin, el mecanismo descrito para el backend: cómo se inyecta el *widget* en la página, cómo se controla quién puede usarlo, cómo se expone la capa de integración con Moodle y cómo convive React con el sistema de módulos AMD de Moodle.

8.5.1 Inyección del widget: Hook API y *callback* legado

Moodle 4.3 incorporó una API de *hooks* tipada, inspirada en PSR-14, que reemplaza progresivamente a los *callbacks* por convención de nombre; en Moodle 4.4 se migraron a esa API los *callbacks* de salida, como `before_footer` (Moodle HQ, s.f.-d). Durante el Sprint 1, sobre una instalación real de Moodle 4.4 (versión 2024042200), el equipo verificó que el *callback* legado se seguía ejecutando, pero su valor de retorno ya no se imprimía: el *widget* desaparecía sin error visible, solo con una advertencia de deprecación. Mientras el plugin soportó Moodle 4.1, ambos mecanismos convivieron, protegidos por una condición de versión para no inyectar el *widget* dos veces. Desde que la versión mínima pasó a Moodle 4.5 LTS (`$plugin->requires = 2024100700`), el *callback* legado se eliminó y `db/hooks.php` registra tres *listeners*: `before_footer_html_generation` inyecta el *widget*, `primary_extend` agrega el acceso a NexusAI en la navegación principal y `before_course_deleted` dispara la purga de los datos del curso en el backend. Antes de inyectar el *widget*, el *listener* verifica que el usuario esté autenticado y no sea invitado, que la página pertenezca a un curso con NexusAI activado, que el usuario tenga `local/nexusai:use` y que la conexión con el backend esté configurada; además, omite los *layouts* donde no puede funcionar (incrustado, ventana emergente, impresión o mantenimiento, entre otros). Finalmente, invoca `$PAGE->requires->js_call_amd()` sobre `local_nexusai/chatwidget-lazy` con el curso, el usuario, el `sesskey`, la URL del sitio y el idioma.

8.5.2 Capabilities y control de acceso

El acceso al plugin se controla con dos *capabilities* declaradas en `db/access.php` (Moodle HQ, s.f.-a): `local/nexusai:use` (lectura, contexto de curso; habilitada para estudiante, profesor, profesor editor y gestor) y `local/nexusai:manage` (escritura, contexto de curso; solo profesor editor y gestor), que protege la gestión del material, la analítica y la configuración del curso. Una tercera *capability*, `local/nexusai:viewanalytics`, se eliminó a pedido de la revisión del Moodle Plugins

Directory porque duplicaba a `:manage`. Toda función externa sigue el patrón `require_login($course)` → `context_course::instance()` → `require_capability(...)`, y el chat multicurso verifica `local/nexusai:use` en el contexto de cada curso consultado. El secreto compartido con el backend se almacena con `admin_setting_configpasswordunmask` y se lee con `get_config()`. La Privacy API (`classes/privacy/provider.php`) es obligatoria para publicar el plugin: declara las 16 tablas propias, las preferencias de usuario y tres ubicaciones externas —el backend de NexusAI, el proveedor de IA que este utiliza (al que no se envía el identificador del usuario) y el *webhook* opcional del resumen de foros—, e implementa la exportación y el borrado de datos, incluida la purga de las copias que conserva el backend.

8.5.3 *Web services*: el patrón de tres métodos

Moodle expone sus funciones externas en tres capas: una interfaz de servidor externo (REST, SOAP o AJAX), una API pública en PHP que extiende `external_api` y una API interna que confía en que la capa pública ya validó los datos (Moodle HQ, s.f.-c). Toda función externa sigue un patrón obligatorio de tres métodos: `execute_parameters()` define un contrato tipado que bloquea parámetros no esperados; `execute()` comienza con `validate_parameters()`, `validate_context()` y `require_capability()` antes de acceder a cualquier dato; y `execute_returns()` describe la salida, que Moodle depura con `external_api::clean_return_value()`. El plugin declara 56 funciones externas en `db/services.php`, todas invocables por AJAX desde el *widget* mediante `core/ajax`. En sentido inverso, el plugin no habilita servicios web REST para el backend: es el plugin quien envía el material, desde tareas *ad hoc* que se encolan cuando un *observer* detecta la creación, modificación o eliminación de una actividad Archivo, de modo que el backend nunca decide por sí mismo qué material puede ver cada persona.

8.5.4 React como módulo AMD

Moodle organiza su JavaScript en módulos AMD (RequireJS) que se compilan con Grunt, mientras que React está pensado para empaquetadores modernos con módulos ES (Moodle HQ, s.f.-e). La solución adoptada compila el *frontend* con Webpack en dos *bundles* autocontenidos con salida AMD (`libraryTarget: 'amd'`): `chatwidget-lazy`, el asistente, y `documents-manager-lazy`, la gestión del material.

Cada uno exporta una función `init()` que monta la aplicación con `createRoot` (React, s.f.) y se carga con `js_call_amd()`. Los parámetros de `js_call_amd()` se pasan como un arreglo anidado (`[[...]]`): el arreglo externo se expande como argumentos posicionales de `init()`, de modo que el interno llega como un único objeto. El sufijo `-lazy` indica a Moodle que no incluya esos módulos, de gran tamaño, en el paquete de JavaScript que se descarga en cada página, sino que los cargue solo cuando se usan. React se comunica con Moodle únicamente a través de `core/ajax`, que agrega el `sesskey` (protección CSRF), reutiliza la sesión autenticada y mantiene el mismo origen.

Durante el Sprint 1 el equipo encontró y documentó un defecto de producción: los `import()` dinámicos de Webpack (Webpack, s.f.) generaban *chunks* que, sin `output.publicPath`, se resolvían en relación con la URL del último script ejecutado por el navegador —en ese caso, el CDN de MathJax—, lo que producía un `ChunkLoadError` al intentar descargarlos desde `cdn.jsdelivr.net`. La corrección consistió en fijar `publicPath` a la ruta real del plugin, desactivar la división de código (`splitChunks: false`, `runtimeChunk: false`) y reemplazar los `import()` dinámicos del punto de entrada por *imports* estáticos.

8.5.5 Empaquetado con Webpack

La configuración de Webpack 5 declara como *externals* a `core/ajax`, `core/str`, `core/notification`, `core/templates` y `jquery`: no se empaquetan porque Moodle ya los provee en tiempo de ejecución, lo que permite reutilizar el `sesskey` de `core/ajax`, las cadenas de idioma de `core/str` y el sistema de notificaciones, y evita una segunda copia de jQuery. Los estilos se inyectan con `style-loader` y `css-loader` en un elemento `<style>` al cargar el *bundle*, con clases propias del plugin para no colisionar con el tema Boost/Bootstrap. Para cumplir con la revisión del Moodle Plugins Directory, Webpack escribe los *bundles* sin minificar en `amd/src/` —declarados en `thirdpartylibs.xml`—, y el comando `grunt amd` de Moodle genera `amd/build/` con sus mapas de código fuente; el CI ejecuta `moodle-plugin-ci grunt` para verificar que `amd/build/` coincide con las fuentes. El paquete distribuido incluye además el código fuente de React y las instrucciones de compilación. El *bundle* del asistente pesa unos 820 KB minificado y unos 200 KB comprimido con gzip, y solo se descarga en las páginas de cursos donde NexusAI está activo.

8.6 Pruebas

La estrategia de pruebas del MVP se organizó en nueve ciclos documentados como *issues* `TEST-01` a `TEST-09`, cerradas durante el Sprint 4, cada una con un responsable y un alcance específico. Después del MVP, el trabajo práctico de la cátedra de Calidad de Software (`TPC-01` a `TPC-11`) incorporó métricas de cobertura con SonarCloud, pruebas de carga con k6 y despliegue continuo con pruebas de humo hacia el ambiente de *staging*.

El plan de pruebas vigente (versión 1.2, 19/10/2026) define cinco niveles, con herramientas y criterios de entrada y de salida para cada uno, una matriz de riesgos y la correspondencia con las características de calidad de ISO/IEC 25010 (International Organization for Standardization, 2023). La tabla siguiente resume los niveles y su criterio de salida.

Tabla 12. *Niveles del plan de pruebas*

Nivel	Alcance	Herramientas	Criterio de salida
Unidad	Funciones y clases del backend, del plugin y del *widget*	pytest, PHPUnit, Vitest	Todas las pruebas pasan; cobertura del código nuevo ≥ 80 %
Integración	*Routers* con PostgreSQL y Redis reales; funciones externas con la base de Moodle	pytest, PHPUnit	Todas las pruebas pasan en las 7 combinaciones de Moodle
Integración externa	Proveedor de LLM real; contratos entre el plugin y el backend	pytest, ejecución diaria	Pruebas en verde
Sistema	API desplegada en *staging*, con solicitudes firmadas como las del plugin	Prueba de humo en el *pipeline*; k6	Prueba de humo en verde; carga dentro de los umbrales
Aceptación	Escenarios de las historias de usuario del MVP	Guiones manuales	Criterios de aceptación de cada historia

Los casos de prueba de caja negra de las tres funcionalidades de mayor prioridad se diseñaron con partición en clases de equivalencia (CE), análisis de valores límite (VL) y tablas de decisión (TD): 26 casos para el chat (HU-001), 14 para la carga de documentos (HU-010 y HU-011) y 24 para la generación de cuestionarios (HU-030 y HU-031), todos automatizados y en verde. La tabla siguiente muestra una selección representativa.

Tabla 13. *Selección de casos de prueba de caja negra*

ID	Técnica	Entrada	Resultado esperado
CP-CHAT-03	VL (máximo)	Pregunta de 2.000 caracteres	Se acepta
CP-CHAT-04	VL (máximo + 1)	Pregunta de 2.001 caracteres	Se rechaza
CP-CHAT-23	CE	Turno del historial con rol `system`	Se rechaza: el cliente no puede inyectar instrucciones de sistema
CP-CHAT-25	VL (límite + 1)	Pedido que supera el límite diario	HTTP 429
CP-DOC-02	TD	El mismo PDF ya indexado	HTTP 200 con el documento existente, sin duplicarlo
CP-DOC-09	TD, CE	Se declara PDF, pero el contenido es un ejecutable (firma `MZ`)	HTTP 400, sin crear el documento
CP-DOC-12	VL (máximo)	Archivo de 20 MB exactos	Se acepta: el límite es inclusivo
CP-DOC-13	VL (máximo + 1)	Archivo de 20 MB + 1 byte	HTTP 413
CP-QUIZ-12	CE	Dificultad `HARD`, fuera de los valores admitidos	HTTP 422
CP-QUIZ-17	TD	Falla la recuperación de fragmentos	HTTP 503, sin invocar al LLM

Medidos con cobertura de ramas, esos casos llevaron las funciones bajo prueba al 100 % de cobertura de sentencia y de decisión; por ejemplo, la función que genera cuestionarios pasó del 44,4 % al 100 % de sus sentencias, y al recorrer su tabla de decisión apareció un defecto real (DEF-08). El registro de defectos documenta diez defectos, DEF-01 a DEF-10 —cuatro de severidad alta, cuatro de severidad media y dos de severidad baja—, todos corregidos y con su prueba de regresión. Las pruebas de aceptación con Behat no tienen todavía escenarios; la aceptación se verificó con los guiones manuales de cada historia y con la prueba de uso de `TEST-06`.

8.6.1 Pruebas unitarias y de integración (backend)

`TEST-02` cubrió la *suite* unitaria del backend en Python con **pytest** (pytest, s.f.), sobre RAG, *embeddings*, fragmentación y los *endpoints* de FastAPI. Al 22/10/2026 esa *suite* reúne 539 funciones de prueba en 43 archivos; el Anexo C conserva, como referencia, las 37 pruebas del cierre del MVP. Usa `@pytest.mark.asyncio` para los *endpoints* asíncronos, `@pytest.mark.parametrize` para cubrir variantes de un mismo caso y *fixtures* de `conftest.py` (`fake_redis`, `fake_openai_chat_response`, `fake_chat_stream`) con `AsyncMock` y `MagicMock`, junto con la sustitución de dependencias de FastAPI, para simular Redis y al proveedor de LLM sin llamadas de red. Un archivo aparte,

`tests/integration/test_llm_real.py`, se ejecuta contra el proveedor real para no depender únicamente de respuestas simuladas.

La cobertura se mide en cada *pull request*, y SonarCloud aplica una compuerta de calidad (*quality gate*) que rechaza el cambio si la cobertura del código nuevo es menor al 80 % o si introduce errores o vulnerabilidades (SonarSource, s.f.). A partir de la línea base del 14/10/2026, la cobertura total ponderada de los tres componentes —backend, plugin y *widget*— pasó de 56,6 % a 83,6 % (10.812 de 12.931 líneas).

Las pruebas de integración restantes fueron manuales: `TEST-03` verificó el flujo completo React → *plugin* PHP → backend Python → proveedor de LLM con datos reales; `TEST-04` probó el *plugin* en un *sandbox* real de Moodle con usuarios de prueba; `TEST-05` encadenó ambas en una sesión *end-to-end* completa, desde que un alumno abre el chat hasta que recibe una respuesta sobre material real de curso.

8.6.2 Pruebas automatizadas del frontend y del plugin

Antes de la *issue* #311 (QA-01), el *widget* de React no tenía pruebas automatizadas. Esa primera iteración priorizó la lógica pura y la capa de API: `gift.js` (conversión al formato GIFT) alcanzó el 100 % de cobertura, y se sumaron pruebas de los módulos `api/*.js` y de un componente de manejo de errores, mientras que los paneles con estado quedaron pendientes. El trabajo práctico de Calidad de Software extendió luego la *suite* de Vitest (Vitest, s.f.) a 236 casos en 32 archivos, que incluyen esos paneles; al subir la cobertura del *widget* se detectó un defecto real que ninguna prueba cubría (DEF-01). Del lado del plugin, 257 pruebas PHPUnit en 41 archivos verifican, entre otros aspectos, el aislamiento entre cursos, la visibilidad del material, el control de acceso a los foros, la Privacy API y la migración de datos. Las tres *suites* corren en GitHub Actions; la del plugin lo hace con `moodle-plugin-ci` sobre siete combinaciones de versión de Moodle (4.5 a 5.3) y motor de base de datos (PostgreSQL y MySQL).

8.6.3 Evaluación de calidad del asistente: dos campañas

`TEST-08` produjo un *golden set*: una rúbrica manual de doce preguntas sobre contenido de bases de datos, evaluadas con tres criterios de 1 a 5 (relevancia, completitud y honestidad). Esa corrida, ejecutada en el MVP, se resume en el Anexo C: 8 de 10 preguntas respondidas correctamente y con citación adecuada, 2 de 2 casos en los

que el asistente admitió correctamente no poder responder, y ningún falso positivo ni falso negativo.

La segunda campaña, automatizada (11/09/2026), reutiliza las doce preguntas del *golden set* con palabras clave de referencia para calcular las métricas, pero las ejecuta contra un corpus sintético bajo un curso dedicado (`course_id` 990001), en lugar de las 30 a 50 preguntas anotadas por un docente sobre material real que preveía el plan original. Mide tres niveles. En la recuperación, sobre las ocho preguntas con respuesta en el material, obtuvo **Recall@5 = 1,00** (objetivo ≥ 0,85), **MRR = 0,9375** (objetivo ≥ 0,70) y **Precision@5 = 0,275** (objetivo ≥ 0,50, no alcanzado): con un corpus de unos nueve fragmentos, casi todo el *top-5* está formado por vecinos legítimos debido al solapamiento entre fragmentos, por lo que la métrica queda acotada por el tamaño del corpus más que por errores de recuperación. La fidelidad de la respuesta generada se evalúa con un segundo LLM como juez, de un proveedor distinto del que genera la respuesta, para que el modelo no se autoevalúe; y la honestidad del *fallback*, con cuatro preguntas deliberadamente ajenas al material. En esa corrida, la cuota gratuita de Gemini se agotó a mitad de la ejecución y el modelo de respaldo configurado estaba discontinuado: de las ocho preguntas de fidelidad solo se evaluaron dos (una fiel y una parcial, por corte de la respuesta) y las cuatro de *fallback* no pudieron medirse. El reporte distingue "no medible" de "0 % de honestidad", y el hallazgo —un modelo de respaldo discontinuado que convierte una degradación esperable en un error— constituye en sí mismo un resultado relevante de la evaluación.

8.6.4 Pruebas de aceptación, usuarios reales y rendimiento

`TEST-06` incorporó la opinión de entre tres y cinco estudiantes de la UCC que usaron el asistente con contenido real de un curso, y `TEST-07` cerró el ciclo del MVP con la corrección de los defectos encontrados y una verificación de regresión. `TEST-09` fijó el objetivo de una latencia promedio menor a 10 segundos, que luego se formalizó como acuerdo de nivel de servicio (p95 < 8 s por respuesta completa) y se midió con pruebas de carga en k6 (Grafana Labs, s.f.). Los escenarios firman cada solicitud con HMAC igual que el plugin; los de capacidad usan un LLM simulado con 1,5 s de latencia, para medir al backend y no la cuota del proveedor, y el escenario `real` consulta al modelo de *staging*. La tabla siguiente resume la corrida del 19/10/2026.

Tabla 14. *Resultados de las pruebas de carga con k6 (corrida del 19/10/2026)*

Escenario	Carga	Resultado	p50	p95	Criterio
`smoke`	1 usuario virtual, 30 s	5 consultas, 100 % correctas	1,95 s	2,45 s	p95 < 8 s: cumple
`sostenido`	10 usuarios virtuales, 10 min	1.008 consultas, 100 % correctas	2,01 s	2,45 s	p95 < 8 s: cumple
`stress`	Rampa hasta 100 usuarios virtuales	6.215 consultas, 99,95 % correctas, sin errores 5xx	2,04 s	2,52 s	Sin punto de quiebre
`rate_limit`	30 preguntas simultáneas de un alumno	20 respondidas y 10 rechazadas con HTTP 429	—	—	Cumple
`dedup`	20 cargas del mismo archivo	100 % reutilizan el documento existente	0,00 s	0,03 s	p95 < 1 s: cumple
`real` (*staging*, `gpt-4o-mini`)	2 usuarios virtuales, 2 min	35 consultas, 100 % correctas	2,99 s	4,39 s	p95 < 8 s: cumple

En el escenario sostenido, el p95 por minuto se mantuvo entre 2,41 y 2,53 s durante los diez minutos, sin tendencia creciente ni conexiones retenidas al finalizar. Las tres consultas fallidas del escenario de estrés (0,05 %) fueron errores de conexión del generador de carga, sin errores registrados en el backend; ese mismo escenario detectó además un defecto real (DEF-09), corregido y verificado con una nueva corrida. Por último, desde el 21/10/2026 cada *push* a la rama `staging` despliega el backend de forma automática y ejecuta una prueba de humo que verifica el estado del servicio, una consulta firmada de extremo a extremo (firma HMAC, base de datos, *embeddings* y LLM) y el rechazo de una solicitud repetida (anti-*replay*); si alguna verificación falla, el *pipeline* queda en rojo.

8.6.5 Síntesis de resultados frente a los objetivos

La tabla siguiente contrasta los indicadores medidos con los objetivos y los requerimientos no funcionales del proyecto.

Tabla 15. *Síntesis de resultados frente a los objetivos*

Indicador	Objetivo	Resultado	Estado
Recall@5 de la recuperación	≥ 0,85	1,00	Cumple
MRR de la recuperación	≥ 0,70	0,94	Cumple
Precision@5 de la recuperación	≥ 0,50	0,275 (corpus de unos 9 fragmentos)	No cumple
Latencia p95 de la respuesta completa con el LLM real (RNF-07)	< 8 s	4,39 s	Cumple

Indicador	Objetivo	Resultado	Estado
Latencia p95 con carga sostenida (10 usuarios, LLM simulado)	< 8 s	2,45 s	Cumple
Errores 5xx con 100 usuarios concurrentes	0	0	Cumple
Nueva carga de un archivo ya indexado	p95 < 1 s	0,03 s	Cumple
Latencia hasta el primer *token* (RNF-06)	< 2 s	No medida por separado	Pendiente
Cobertura de pruebas automatizadas (RNF-16)	≥ 70 %	83,6 % (total ponderado de los tres componentes)	Cumple
Vulnerabilidades en dependencias del *widget*	0	0 (desde 6 antes de la auditoría)	Cumple
Observaciones de la revisión del Moodle Plugins Directory	16 resueltas	16 de 16; plugin aprobado y publicado	Cumple
Fidelidad y honestidad del *fallback* (campaña automatizada)	Medir	2 de 8 y 0 de 4 casos medibles (cuota agotada)	Pendiente

9 Seguridad y cumplimiento

Las secciones anteriores describieron la arquitectura de seguridad desde el diseño (autenticación HMAC, *capabilities*, Privacy API). Este capítulo da respaldo concreto a esas decisiones: una auditoría de seguridad real ejecutada sobre el proyecto, la estrategia de privacidad bajo la que opera, y las prácticas del plugin que todavía no se explicaron en profundidad.

9.1 Auditoría de seguridad: OWASP Top 10 y análisis de dependencias

En agosto de 2026 el equipo ejecutó una auditoría de seguridad formal sobre el proyecto, combinando análisis automatizado de dependencias (`pip-audit` en el backend, `npm audit` en el *widget* de React) con un *checklist* manual del OWASP Top 10 (OWASP, s.f.-a) aplicado específicamente a NexusAI. El escaneo de dependencias del backend pasó de 43 CVEs en 11 paquetes a 12 CVEs en 3: dos paquetes sin uso real en el código (`python-jose[cryptography]`, `python-multipart`) se eliminaron por completo, junto con sus dependencias transitivas, —verificado por `grep` en todo `app/`, ya que la autenticación usa el módulo `hmac` de la librería estándar, no JWT, y las subidas de archivo son JSON+base64, no *multipart*—, `Pillow` se actualizó de 10.4.0 a 12.3.0, y las 12 CVEs restantes quedan **aceptadas y documentadas por CI, no ignoradas en bloque**: `starlette` está fijado por la versión de FastAPI en uso (una corrección completa implicaría migrar cada *router*), `pdfminer-six` está fijado transitivamente por `pdfplumber`, el núcleo del *pipeline* de extracción de PDF del sistema RAG (actualizarlo sin pruebas dedicadas arriesga romper la función central del producto), y `pytest` es una dependencia de desarrollo que nunca llega a producción. El frontend pasó de 6 vulnerabilidades (1 baja, 1 moderada, 4 altas) a 0, resueltas con `npm audit fix` sin forzar actualizaciones mayores; la auditoría destaca en particular `dompurify`, que sanitiza las respuestas del asistente antes de insertarlas en el DOM (`MessageBubble.jsx`) — la única superficie real de XSS del *widget*. Como hallazgo colateral, la auditoría encontró que el *workflow* de CI del frontend apuntaba a un `package.json` que no existe (la ruta real está un nivel más abajo, en `react/`), por lo que ese *job* nunca había corrido realmente en ningún PR anterior; se corrigió como parte de esta misma issue.

El *checklist* OWASP Top 10 aplicado al proyecto confirma, entre otros puntos: **A01 (control de acceso roto)**, mitigado porque todo *endpoint* exige `verify_hmac` y toda función externa de Moodle valida contexto y *capability* antes de delegar, con el `user_id` tomado siempre de `$USER->id` y nunca del navegador; **A02 (fallas criptográficas)**, mitigado por el esquema HMAC de tres capas; **A03 (inyección y XSS)**, mitigado por consultas parametrizadas y por el saneamiento de las respuestas con DOMPurify; **A07 (fallas de autenticación)**, mitigado por el *nonce* anti-*replay* en Redis y la comparación de secretos en tiempo constante; y **A09 (fallas de registro y monitoreo)**, cubierto parcialmente: existe un registro estructurado de accesos y la auditoría confirmó, tras revisar el código de chat, cuestionarios y foros, que en ningún lugar se registra el contenido de los mensajes en texto plano. Al momento de la auditoría quedaban dos brechas abiertas: el límite de 50 consultas diarias por alumno, documentado pero no implementado, y la ausencia de moderación de contenido. Ambas se resolvieron después: el backend aplica hoy límites de 20 consultas por minuto y 50 por día por usuario; presupuestos de *tokens* por rol —8.000 por hora y 40.000 por día para alumnos, y 20.000 y 100.000 para docentes— que se reservan de forma atómica antes de cada llamada; y un módulo de moderación que filtra las entradas antes de que lleguen al LLM, con comportamiento *fail-open* configurable. Una nueva revisión de dependencias del 21/10/2026 (DEF-10) actualizó Pillow a la versión 13.0.0 y Vite a la 8.3.5 por vulnerabilidades publicadas ese mismo día. Esta auditoría respalda **RNF-01** (la clave del proveedor de IA nunca llega al navegador), **RNF-02** y **RNF-03** (HMAC en tres capas y rechazo de firmas, marcas de tiempo o *nonces* inválidos) y **RNF-05** (un alumno no puede leer sesiones de chat ajenas); **RNF-04** queda respaldado por el punto A01 y por la sección 8.5.2.

9.2 Estrategia de privacidad y cumplimiento normativo

El Anexo E resume las decisiones ADR-006 y ADR-014; esta sección desarrolla el marco de cumplimiento que las sustenta. ADR-006 encuadra la estrategia de privacidad en dos marcos normativos: la Ley 25.326 de Protección de los Datos Personales de la República Argentina (Ley N.º 25.326, 2000) y el Reglamento General de Protección de Datos de la Unión Europea (Reglamento [UE] 2016/679, 2016). Las categorías de datos personales en juego son los mensajes que el alumno envía al chat, el historial de conversación, el vínculo entre el identificador de usuario de Moodle y sus sesiones, y las métricas de uso. La Privacy API de Moodle permite a un

administrador exportar o borrar los datos que residen en tablas de Moodle, pero no alcanza por sí sola a un servicio externo: mientras los datos del alumno residieron en el backend (antes de ADR-014), un pedido de supresión requería coordinación manual con quien lo administra. Por eso ADR-006 establece como condición previa a un despliegue con alumnos reales la firma de un Acuerdo de Tratamiento de Datos (DPA) con la UCC, cuya coordinación con el área legal de la universidad quedó asignada a Belén Zarazaga (Administración de Proyectos). Tras ADR-014, la exportación y el borrado de los datos del alumno se resuelven con la Privacy API, que además dispara la purga de las copias de *posts* de foro que conserva el backend; en esas copias, los autores se envían seudonimizados ("Participante 1", "Participante 2", numerados por hilo).

Al revisar las métricas de uso para la migración de ADR-014, el equipo detectó que el identificador de usuario de esas métricas era un *hash* SHA-256 sin sal de un número entero (el `user_id` de Moodle): un esquema de seudonimización ineficaz, porque el espacio de identificadores es lo suficientemente pequeño como para calcular el *hash* de cada valor posible y revertirlo por fuerza bruta. El procedimiento de corte definido para esa migración prevé un respaldo previo de la base del backend, un ensayo completo sobre una copia de producción en *staging*, la verificación de conteos y de consumo por curso y por mes antes de dar la migración por válida, y el archivo de las tablas anteriores del backend durante 60 días antes de su eliminación definitiva. El requerimiento RNF-14 del Anexo A refleja este modelo híbrido.

9.3 Prácticas de seguridad del plugin

Además de las *capabilities* y del patrón de autorización ya descritos, el plugin aplica las prácticas de seguridad que exige el desarrollo para Moodle. Nunca lee `$_GET` o `$_POST` directamente: todo parámetro pasa por `required_param()` u `optional_param()` con una constante `PARAM_*` (`PARAM_INT`, `PARAM_TEXT`, `PARAM_ALPHANUMEXT`, entre otras), que valida y depura el tipo antes de que el valor llegue a la lógica de negocio. El código se verifica contra el estándar de codificación de Moodle con `phpcs` y las comprobaciones de `moodle-plugin-ci`. Los errores del backend o del proveedor de IA se informan con `moodle_exception` y un mensaje traducido, nunca con `die()` o `exit()`, y las interacciones relevantes se registran mediante la Events API de Moodle. Las pruebas del plugin se automatizan con PHPUnit (257 pruebas), con el proveedor de LLM

simulado para obtener resultados deterministas. Por último, `thirdpartylibs.xml` declara cada librería de terceros incluida en el paquete —React, sus dependencias y los *bundles* generados— con su versión y licencia, requisito de la revisión del Moodle Plugins Directory.

La tabla siguiente resume los riesgos de seguridad específicos de un asistente de IA integrado a un LMS y la mitigación implementada para cada uno.

Tabla 16. *Riesgos de seguridad específicos de NexusAI y mitigaciones*

Riesgo	Mitigación implementada
Exposición de la clave del proveedor de IA	Patrón *Hybrid PHP Proxy*: la clave reside solo en el backend.
Acceso a material de otro curso	Filtro por `course_id` en cada consulta vectorial y verificación de la *capability* en el contexto del curso.
Uso de material oculto o restringido	El plugin calcula `visible_cmids` con `cm_info::uservisible` en cada solicitud, y el backend rechaza con HTTP 422 las solicitudes que no incluyen esa lista.
Lectura de conversaciones ajenas	Mensajes asociados al usuario de la sesión de Moodle y filtrados por él en cada consulta.
Exposición de respuestas privadas de foros	`forum_access_guard` reutiliza `forum_user_can_see_discussion()` y `forum_user_can_see_post()` de `mod_forum` para cada *post*.
Repetición de solicitudes capturadas	*Nonce* de un solo uso en Redis y ventana temporal de 300 s.
Abuso de consumo	Límites por minuto y por día, presupuestos de *tokens* por rol y alertas por cuota agotada.
Inyección de *prompt* a través del material	Delimitadores explícitos del contexto e instrucción de sistema que el contenido del curso no puede reemplazar.
Contenido inapropiado	Moderación de las entradas antes de invocar al LLM.

La inyección de *prompt* es el riesgo más específico de la lista y el que un *checklist* genérico como el OWASP Top 10 no contempla: un documento subido por un docente podría contener texto diseñado para manipular al modelo de lenguaje. La mitigación actual reduce el riesgo, pero no lo elimina; una evaluación sistemática con documentos adversarios queda como trabajo futuro.

10 Despliegue e infraestructura

10.1 Topología: del plan original a las dos VMs reales

El backend pasó por tres destinos de despliegue a lo largo del proyecto hasta quedar en Oracle Cloud, según se detalla en ADR-011 (Anexo E). Como se observa en la Figura 11, el plan original ubicaba el backend junto a la infraestructura de la UCC.

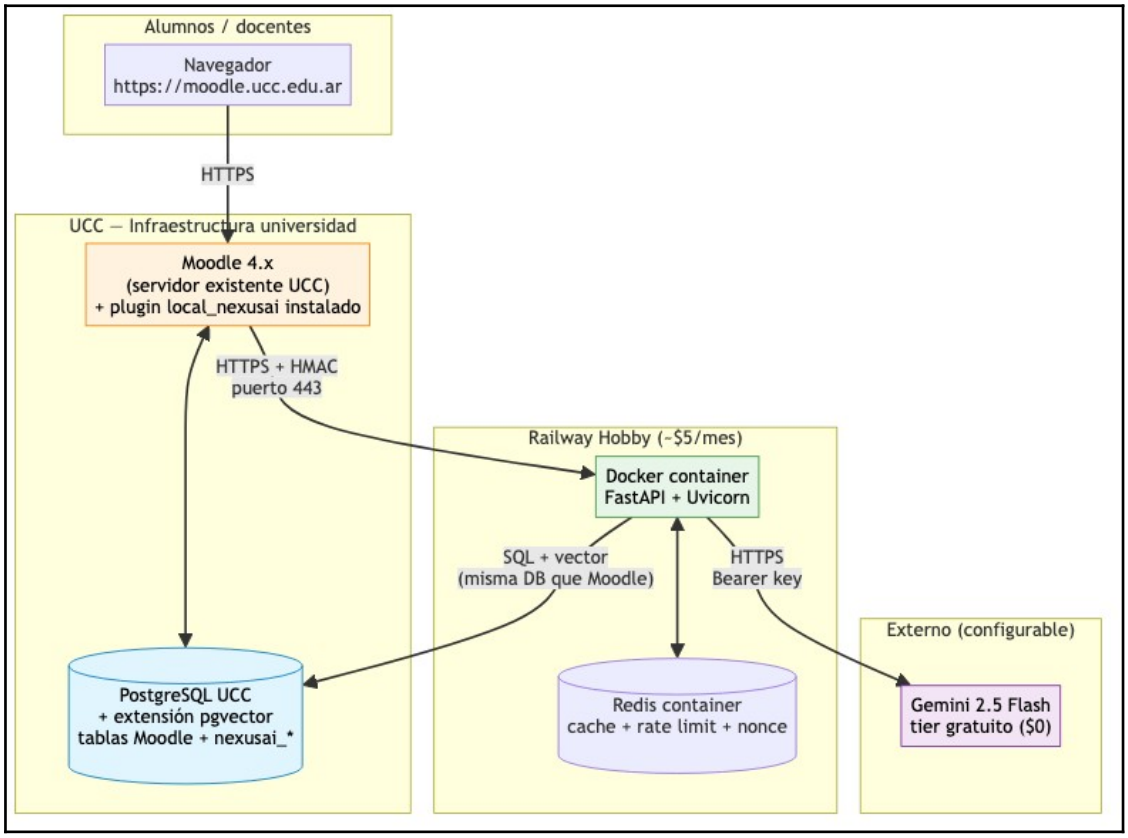


Figura 11. Arquitectura de despliegue planificada en mayo de 2026, actualmente no vigente. *Fuente:* elaboración propia. Se conserva como registro histórico: el despliegue real reemplazó Railway y la base de datos de la UCC por infraestructura autogestionada en Oracle Cloud. *Texto alternativo:* el navegador accede al Moodle de la universidad, que se comunica por HTTPS con un contenedor del backend en Railway; el backend usa la misma base PostgreSQL de Moodle y un Redis propio, y consulta a Gemini.

La Figura 12 muestra, en cambio, la topología vigente desde agosto de 2026.

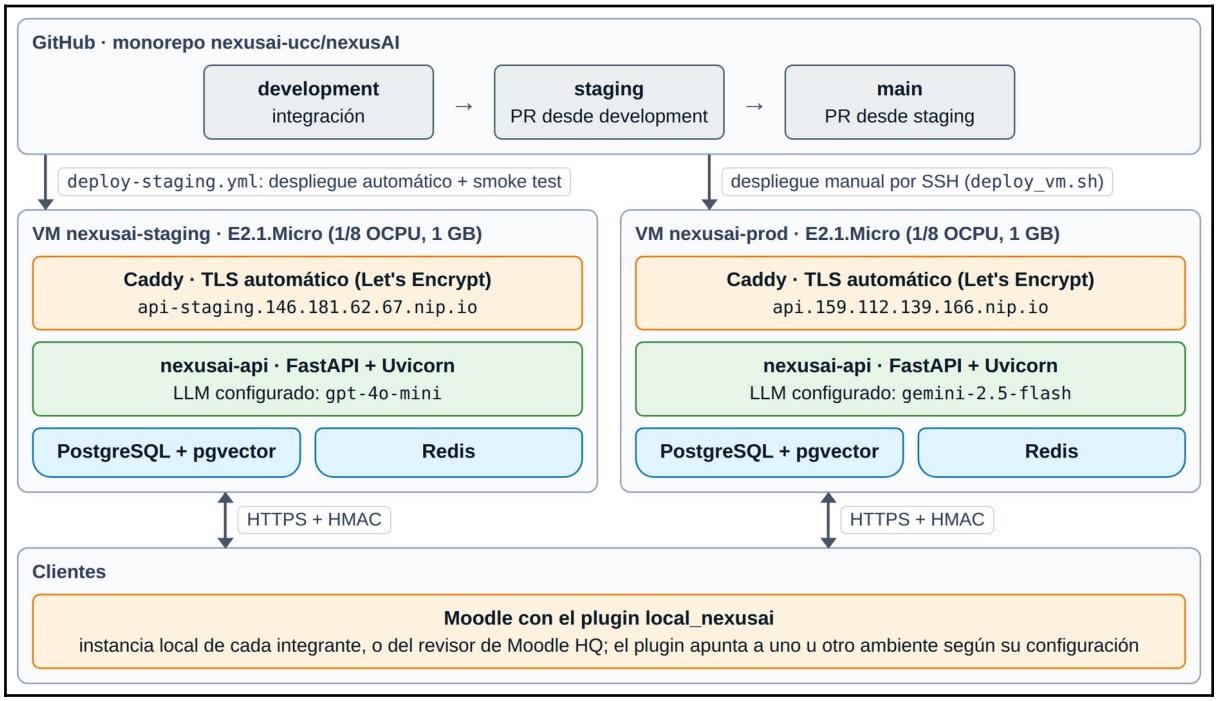


Figura 12. Topología de despliegue vigente al 22/10/2026. *Fuente:* elaboración propia. *Texto alternativo:* dos máquinas virtuales *Always Free* de Oracle Cloud (Oracle, s.f.), una por ambiente, con los mismos cuatro contenedores (Caddy, API, PostgreSQL y Redis); la rama `staging` se despliega automáticamente y la rama `main`, en forma manual; Moodle no se ejecuta en ninguna de las dos máquinas y se conecta a ellas por HTTPS con firma HMAC.

Cada VM es una instancia `VM.Standard.E2.1.Micro` (1/8 de OCPU con capacidad de ráfaga, 1 GB de RAM y 4 GB de *swap*) en la región Chile Central (Oracle, s.f.). La de *staging* (`nexusai-staging`, creada el 08/08/2026) sigue la rama `staging`, y la de producción (`nexusai-prod`, creada el 05/08/2026), la rama `main`. Cada VM sigue exclusivamente la rama de su ambiente: no se actualiza una VM desde `development` para pruebas puntuales, porque el ambiente quedaría en un estado que no corresponde a ninguna rama y el siguiente despliegue lo sobrescribiría sin aviso.

10.2 El proceso de despliegue

Hasta el 21/10/2026 el despliegue en ambas VMs era manual. Desde esa fecha, el *workflow* `deploy-staging.yml` despliega automáticamente cada *push* a la rama `staging`: se conecta por SSH con una clave que solo puede ejecutar el despliegue, corre `deploy_vm.sh` —que verifica la rama, aplica `git merge --ff-only`, reconstruye los contenedores y espera hasta cinco minutos a que la API informe un estado saludable—, compara el *commit* desplegado con el enviado y ejecuta la prueba de

humo descrita en la sección 8.6.4. Producción sigue un procedimiento manual equivalente: integrar la *pull request* en `main`, conectarse por SSH, confirmar con `git branch --show-current` que la VM está en la rama esperada, actualizar el código y levantar nuevamente el *stack* con `docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d --build` (Docker, s.f.). La reconstrucción solo es necesaria cuando cambia el código del backend; un cambio que afecta únicamente al plugin de Moodle no requiere intervenir la VM. Luego se verifica con `docker ps` que los cuatro contenedores estén activos y que el *health check* informe el ambiente correcto. Las migraciones de Alembic se aplican automáticamente: `entrypoint.sh` ejecuta `alembic upgrade head` antes de iniciar Uvicorn y, si una migración falla, el contenedor de la API no se inicia.

Un único `Caddyfile` (Caddy, s.f.) sirve a ambos ambientes porque el dominio no está fijado en el repositorio: se toma de la variable de entorno `NEXUSAI_DOMAIN`, distinta en el `.env` de cada VM, que Caddy usa para obtener automáticamente el certificado TLS de Let's Encrypt (Let's Encrypt, s.f.). Si ese valor no coincide con el dominio real, Caddy se inicia igualmente pero no obtiene el certificado: el sitio responde por HTTP con una redirección 308 y el acceso por HTTPS falla con un error de TLS; la corrección consiste en ajustar `NEXUSAI_DOMAIN` y reiniciar el *stack*. Los dominios utilizados (`api-staging.146.181.62.67.nip.io` y `api.159.112.139.166.nip.io`) pertenecen a `nip.io`, un servicio gratuito que resuelve ese formato de nombre a la dirección IP que contiene, sin necesidad de registrar un dominio; como contrapartida, si cambia la IP pública de una VM, cambia también su dominio, y deben actualizarse el `.env`, la documentación y la configuración del plugin en cada Moodle.

Las credenciales se dividen en dos grupos: el `.env` completo de cada VM y la clave SSH privada, que solo necesita el administrador de despliegue —el único integrante con cuenta en Oracle Cloud—, y los tres valores de configuración del plugin (URL del backend, clave de API y secreto compartido), que necesita cualquier integrante para probar el flujo completo contra un backend publicado. Estos últimos se ingresan en la pantalla de configuración del plugin y se comparten por un canal directo, nunca a través del repositorio, de *issues* ni de *pull requests*. La clave de API y el secreto compartido difieren entre *staging* y producción: cambiar la URL del backend sin cambiar esos dos valores hace fallar la validación HMAC.

10.3 Operación, mantenimiento y brechas pendientes

La limitación de 1 GB de RAM por VM —motivo, según ADR-012, para no sumar un *stack* de observabilidad— se controla manualmente con `free -h` y `docker stats --no-stream` cuando hace falta. Dos tareas mantienen auditable el consumo de IA: un *cron* diario (`30 3 * * *`) ejecuta `scripts/rollup_llm_usage.py` dentro del contenedor de la API, que consolida por día en `llm_usage_daily` el detalle de `llm_usage` con más de 13 meses de antigüedad y lo elimina de la tabla de detalle; y un *script* de carga retroactiva (`scripts/backfill_llm_usage.py`), ejecutado una vez y de forma idempotente, incorporó los *tokens* históricos del chat después de que la migración 023 creara las tablas `llm_usage`, `model_prices` y `llm_usage_daily`.

Quedan brechas abiertas en este despliegue. La más crítica es que **no hay respaldos automáticos de PostgreSQL** en ningún ambiente: si se pierde una VM, se pierde su base de datos, y no existe un `pg_dump` programado que lo evite. Le siguen la dependencia de dominios `nip.io` en lugar de un dominio propio; el despliegue todavía manual de producción —el de *staging* ya es automático—; la diferencia de modelo de lenguaje entre ambientes (`gpt-4o-mini` en *staging* y `gemini-2.5-flash` en producción), que impide que las pruebas de *staging* reproduzcan exactamente el comportamiento de producción; y la falta de un Moodle de *staging* conectado al backend, reemplazado hoy por pruebas manuales desde el Moodle local de cada integrante. Además, el backend identifica los cursos solo por su número: si dos instalaciones de Moodle distintas usan el mismo backend, el curso 2 de una y el curso 2 de la otra se tratan como el mismo curso, por lo que cada institución requiere hoy su propio backend.

11 Publicación en Moodle Marketplace

Más allá de las pruebas internas descritas en la sección anterior, el proyecto fue sometido a una validación externa concreta: la revisión de código y funcionalidad que exige el Moodle Plugins Directory (*Moodle Marketplace*) para publicar un plugin de terceros. Esta sección documenta ese proceso, los hallazgos de la revisión y cómo se resolvieron, por tratarse de la evaluación independiente más rigurosa que atravesó el plugin hasta el momento de este informe.

11.1 Por qué separar el repositorio

El formulario de registro de un plugin en moodle.org no admite indicar una subcarpeta: recibe una URL de repositorio y asume que su raíz es la raíz del plugin (Moodle HQ, s.f.-h). El código de NexusAI, sin embargo, vive dentro de un monorepo (`nexusAI`) junto al backend, la documentación y el material de investigación del proyecto, cuatro niveles por debajo de la raíz real del repositorio git. Un validador automático que clona el repo y busca `version.php` en el nivel superior no lo encuentra, y el envío se rechaza antes de llegar a un revisor humano.

De las alternativas evaluadas para resolverlo —pasar la URL del monorepo tal cual, usar un *submodule* de git o copiar la carpeta a mano a un repositorio nuevo— ninguna resultó viable sin perder el historial de commits del plugin o sin dejar de resolver el problema de fondo. La solución adoptada fue `git subtree split`: extrae únicamente los commits que modificaron la carpeta del plugin, con su historial intacto, hacia un repositorio nuevo y vacío (`nexusai-ucc/moodle-local_nexusai`), cuyo nombre sigue además la convención `moodle-{tipo}_{nombre}` que exige Moodle. Desde entonces, el monorepo sigue siendo el lugar donde se desarrolla, y un workflow de GitHub Actions (`sync-derived-repos.yml`) repite el *subtree split* de forma automática en cada *push* a `development`, manteniendo al repositorio público sincronizado sin intervención manual.

11.2 Proceso de envío y primera revisión

El plugin se envió al Moodle Plugins Directory el 21 de septiembre de 2026 en su versión `0.18.1` (*beta*), tras validar un checklist de requisitos de envío: metadatos completos en `version.php`, cabecera de licencia GPL v3+, cadenas de idioma en inglés y español, instalación desde ZIP sin pasos de compilación adicionales, CI real

corriendo sobre Moodle 4.5 a 5.2 con PostgreSQL y MySQL, `LICENSE` dual (MIT en el repositorio raíz, GPL v3 en el plugin), ícono de 256×256 px y capturas de pantalla en inglés.

El flujo de revisión de Moodle es público y sigue un circuito estándar: *Upload initial version* → *Waiting for approval* → *Needs more work* → *Upload fixed version*, hasta la aprobación. Recibir cambios solicitados en la primera ronda es habitual. El *ticket* de revisión MMR-301, a cargo del revisor Volodymyr Dovhan, devolvió el plugin el 11/10/2026 con el estado *Changes needed* y 16 observaciones sobre el código de la versión enviada.

11.3 Correcciones de la primera ronda

La tabla siguiente resume las 16 observaciones del revisor y la corrección aplicada a cada una en la versión 0.19.0.

Tabla 17. *Observaciones de la revisión MMR-301 y correcciones aplicadas en la versión 0.19.0*

N.º	Observación del revisor	Corrección en la versión 0.19.0
1	Usar el usuario actual en las alertas de calendario, no un `userid` enviado por el navegador.	`calendar_alert_save` y `calendar_alerts_list` usan siempre `$USER->id`; el parámetro `userid` se acepta por compatibilidad, pero se ignora.
2	Verificar el acceso al foro antes de enviar *posts* al modelo o devolverlos.	`forum_access_guard` reutiliza `forum_user_can_see_discussion()` y `forum_user_can_see_post()` de `mod_forum` (grupos separados, discusiones programadas, respuestas privadas y foros de preguntas y respuestas) al resumir hilos, sugerir respuestas y buscar *posts* similares.
3	Declarar y exportar las preferencias de usuario en la Privacy API.	El *provider* implementa `user_preference_provider`: el estado del *onboarding*, las cargas pendientes y el *token* del calendario (sin su valor) se declaran y exportan.
4	Declarar todos los datos personales enviados al backend y a los proveedores de IA, y permitir borrar las copias de los foros.	`get_metadata()` declara tres ubicaciones externas (backend, proveedor de IA —sin el identificador del usuario— y *webhook* opcional); los autores de los foros se envían seudonimizados y las copias se purgan al borrar los datos de un usuario, un foro o un curso.
5	Agregar los encabezados de licencia faltantes.	Todos los archivos —PHP, `amd/src`, fuentes de React, CSS y plantillas Mustache— incluyen el encabezado GPL estándar de Moodle.
6	No invocar `curl_init()` directamente.	`chat_stream.php` usa `\curl` con `CURLOPT_WRITEFUNCTION`: se mantiene el *streaming*, se verifica el certificado TLS y no se siguen redirecciones.
7	Mostrar un mensaje traducido cuando la pregunta es demasiado larga.	Toda validación de los servicios web lanza `moodle_exception` con una cadena de idioma, y `chat_stream.php` devuelve mensajes traducidos.

N.º	Observación del revisor	Corrección en la versión 0.19.0
8	Incluir en la versión publicada el código fuente de React.	El ZIP incluye `react/src`, `package.json`, `package-lock.json`, la configuración de Webpack y las instrucciones de compilación; `amd/build` se genera con `grunt amd`.
9	Cargar el *widget* solo donde puede funcionar.	No se carga sin backend configurado, durante instalaciones o actualizaciones, ni en *layouts* incrustados, emergentes, de impresión o de mantenimiento; además requiere NexusAI activo en el curso y `local/nexusai:use`.
10	Enviar los *posts* al backend desde una tarea *ad hoc* y no desde el *observer*.	Los *observers* solo encolan `sync_forum_post`; lo mismo ocurre con el material (`sync_material`) y con la purga de datos (`purge_backend_data`).
11	Generar la página de estado con una plantilla Mustache.	`admin.php` arma un contexto y lo presenta con la plantilla `local_nexusai/admin_status`.
12	Declarar y probar el soporte de las versiones actuales de Moodle.	`$plugin->supported = [405, 503]`; el CI se ejecuta sobre Moodle 4.5 a 5.3 con PostgreSQL y MySQL.
13	Excluir del *feed* de calendario los eventos de actividades que el alumno no puede ver.	Se omiten los eventos de actividades no visibles para el dueño del *feed* (`cm_info::uservisible`), y el *feed* queda vacío para usuarios suspendidos o sin `local/nexusai:use`.
14	Verificar `local/nexusai:use` en cada curso del chat multicurso.	`chat_turn::resolve_courses()` verifica la *capability* en el contexto de cada curso.
15	Omitir la tarea de alertas de calendario si el backend no está configurado.	Las alertas pasaron a tablas propias de Moodle y la tarea ya no invoca al backend.
16	Usar o eliminar la *capability* `local/nexusai:viewanalytics`.	Se eliminó; los servicios de analítica requieren `local/nexusai:manage`.

Además de los 16 puntos, se ajustó la página de estado de la administración, que marcaba como bloqueada cualquier URL en `localhost`, en una red privada o en el puerto 8000, aun cuando el plugin funcionaba contra un backend de desarrollo legítimo; ahora solo valida que el *endpoint* responda por HTTP o HTTPS y muestra los campos de estado del backend.

11.4 Auditoría propia y cierre de los últimos gaps

Antes de responder al ticket, el equipo hizo una auditoría interna cruzando cada punto marcado como resuelto contra el código real de `development`, y encontró una discrepancia: borrar un **curso completo** seguía sin purgar los posts de foro que el backend tenía indexados de ese curso. `data_cleanup::delete_course()` —la función que corre cuando Moodle borra un curso de verdad— solo encolaba la purga de documentos; la purga de posts de foro solo era alcanzable desde los tres métodos de la API de Privacidad de Moodle, que **no se ejecuta** en un borrado de curso real (a

diferencia de un pedido de eliminación de datos de un usuario). El método que faltaba conectar (`provider::delete_backend_forum_posts()`) ya existía y ya soportaba el caso de "todos los usuarios del curso"; solo hacía falta exponerlo como público y llamarlo desde el lugar correcto.

Una revisión posterior, más allá de los 16 puntos originales, encontró un segundo caso del mismo tipo de bug: borrar un **foro puntual**, dejando el resto del curso intacto, tampoco purgaba nada —siempre, no solo en el caso excepcional que el equipo tenía anotado. La causa resultó ser distinta a la del curso: el observador del evento `course_module_deleted` de Moodle filtra a propósito cualquier actividad que no sea de tipo "Archivo", y aunque se hubiera manejado ahí, no habría funcionado: `forum_delete_instance()` —la función que borra las discusiones y los posts de un foro — se ejecuta **antes** de que Moodle dispare ese evento. Para cuando un observador llega a correr, los posts ya no existen en las tablas del foro y no hay nada que consultar. La solución usa en cambio `pre_course_module_delete($cm)`, un *callback* que Moodle descubre por convención de nombre de archivo y ejecuta **antes** de borrar el módulo, pensado exactamente para este tipo de caso —el mismo patrón que usan `tool_recyclebin` y otros complementos del núcleo de Moodle.

Ambos hallazgos se corrigieron, con pruebas automatizadas nuevas que reproducen cada caso, y la corrección quedó incluida en la versión `0.19.0` enviada de vuelta al revisor.

11.5 Sincronización automática y una lección operativa

Como el repositorio que revisa Moodle deriva del monorepo, cada corrección depende de que el *workflow* de sincronización (`sync-derived-repos.yml`) la traslade. Al integrar las correcciones del último hallazgo, el *workflow* falló de forma silenciosa en las dos ejecuciones siguientes: el *token* de GitHub (`SYNC_REPOS_TOKEN`) con el que publica los cambios en el repositorio público había vencido, por lo que ese repositorio seguía mostrando una versión anterior del plugin aunque el monorepo ya tuviera la corrección. El incidente se detectó revisando el estado real de las ejecuciones de CI —y no infiriéndolo del estado de las *pull requests*—, se resolvió generando un nuevo *token* de acceso de alcance limitado (permisos de escritura solo sobre los dos repositorios derivados) y quedó documentado como recordatorio de verificar el vencimiento del *token* en cada renovación.

11.6 Estado actual

El revisor aprobó la versión 0.19.0 (`MATURITY_BETA`) el 19/10/2026, y el plugin quedó publicado en el Moodle Plugins Directory con las correcciones de los 16 puntos de MMR-301 y de los dos hallazgos adicionales descritos. La ficha pública registra como último *release* el 15/10/2026, fecha de carga de esa versión. Como se observa en la Figura 13, la ficha ya refleja esa versión.

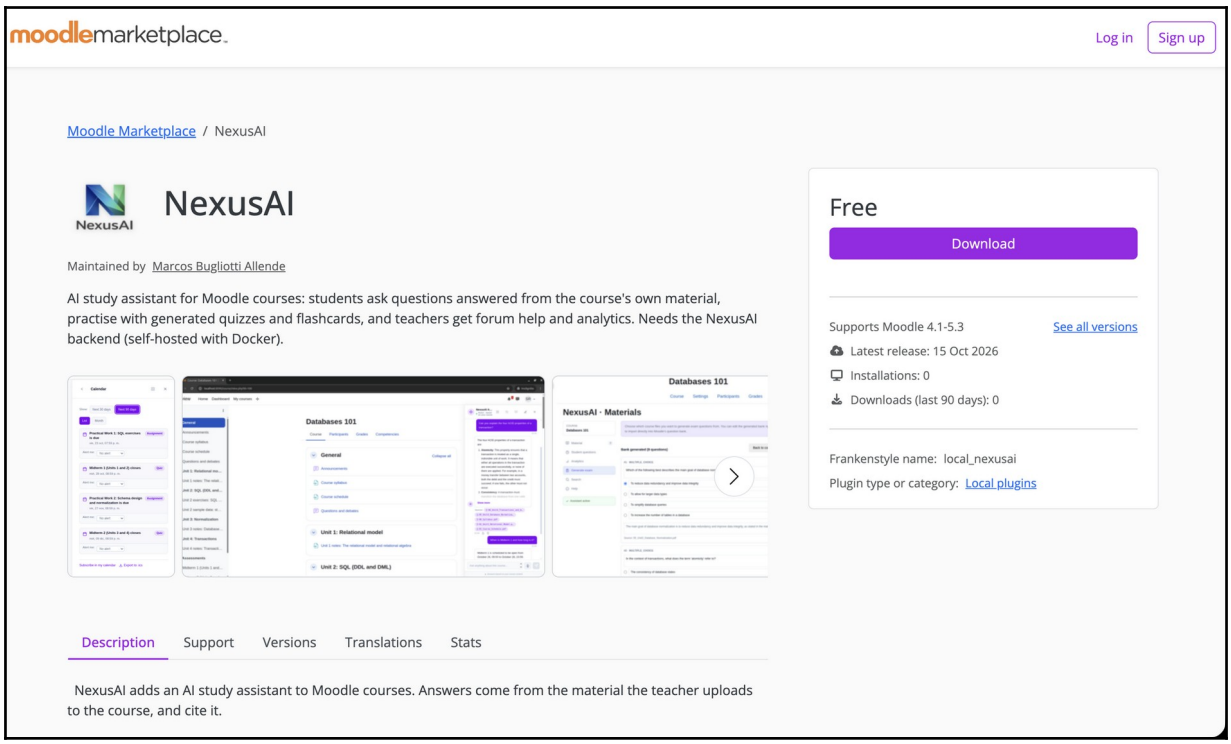


Figura 13. Ficha pública de NexusAI en el Moodle Plugins Directory. *Fuente:* Moodle (s.f.-e), captura del 21/10/2026. *Texto alternativo:* la ficha del plugin NexusAI, con la etiqueta "Free", el botón de descarga, la fecha del último *release* y tres capturas de la interfaz (calendario, chat en un curso y panel docente). El rango "Supports Moodle 4.1–5.3" acumula todas las versiones publicadas; la 0.19.0 declara soporte para Moodle 4.5 a 5.3.

12 Gestión del proyecto y metodología ágil

Esta sección documenta cómo se gestionó el proyecto puertas adentro: la metodología adoptada, el cierre formal de dos de sus instancias —Fase 1 y Sprint 2— con sus métricas de valor ganado, y un ejemplo concreto de cómo la planificación se ajustó frente a la evidencia durante el desarrollo.

12.1 Metodología adoptada

El equipo trabajó con **Scrum**, en sprints de aproximadamente dos semanas, con roles diferenciados: Santiago Tricherri como *Project Manager* y responsable del backend de IA, Delfina Salinas como *Scrum Master* y responsable del frontend, y Marcos Bugliotti a cargo de la base de datos y la integración con Moodle. Cada fase y cada sprint cierran con un **acta formal**, firmada por los tres integrantes, que documenta entregables planificados contra entregados, desviaciones justificadas, métricas y una retrospectiva — el mismo formato que exige la cátedra de Administración de Proyectos de Software, de la que estas actas son entregable directo.

El seguimiento de avance usa **EVM** (*Earned Value Management*): valor planificado (PV), valor ganado (EV) y costo real (AC) por fase, de los que se derivan las variaciones de cronograma y costo (SV, CV) y los índices de desempeño SPI (`EV/PV`) y CPI (`EV/AC`) — una práctica de gestión de proyectos formal, no solo una tabla de velocity informal.

12.2 Cierre de Fase 1 — Setup e Investigación

La Fase 1 (9 al 22 de abril de 2026) cerró con sus entregables técnicos 100% completados y en plazo, con un **sobre-cumplimiento deliberado** en investigación y formalización: 47 documentos en `investigacion/` contra los ~20 esperados originalmente, y 6 ADRs aceptados contra los 4 planificados (se sumaron HMAC PHP-Python y la estrategia de Privacidad, ambos necesarios antes de lo previsto por decisiones técnicas tempranas).

Las métricas de valor ganado al cierre de la fase fueron **SPI = 1,00** (en línea con el cronograma) y **CPI = 0,77** (el costo real superó en un 30 % al valor ganado, ya que 1/0,77 ≈ 1,30). El equipo interpretó ese sobrecosto como una decisión consciente, no como un desvío no controlado: invertir más horas en documentación y ADRs amortizó

tiempo de desarrollo en los sprints siguientes, hipótesis que las mediciones acumuladas del proyecto al 4 de mayo (SPI = 1,10, CPI = 0,85) respaldaron.

De las lecciones aprendidas documentadas en el acta de cierre, dos resultan especialmente relevantes para entender decisiones posteriores: el equipo subestimó el tiempo de coordinación con actores externos a la universidad (la reunión con el técnico de Moodle de la UCC quedó pendiente al cierre de la fase y nunca se concretó, lo que terminó reforzando la estrategia de demo autohospedada descrita en *Relevamiento y metodología de investigación*), y reconoció que hubiera convenido empezar la primera prueba funcional antes de cerrar toda la documentación, ya que varios problemas de integración (migración del Hook API de Moodle, *chunks* de Webpack rotos, *symlinks* no funcionando en Docker) solo aparecieron al implementar.

El registro de riesgos abierto al cierre de la fase incluía, entre otros: el riesgo de que la aprobación institucional de la UCC demorara más de 4-6 semanas (mitigación: Plan B de demo local, que terminó siendo la estrategia efectiva), el riesgo de que la calidad del RAG con material real quedara por debajo de lo esperado (mitigación: dataset de evaluación propio con preguntas reales del Proyecto Integrador) y el riesgo de restricciones de privacidad del proveedor de embeddings sobre datos de alumnos (mitigación: términos a confirmar antes del piloto, con un proveedor local como plan de contingencia).

12.3 Cierre de Sprint 2 — RAG completo

El Sprint 2 (7 al 20 de mayo de 2026) cerró en plazo con el núcleo funcional del MVP: el sistema ya respondía preguntas usando el material real del curso, citando la fuente, con una vista docente para gestionar ese material. Tres desviaciones de alcance quedaron documentadas con su justificación técnica: el modelo de *embeddings* planificado (`text-embedding-004`) dejó de responder por la vía compatible con OpenAI que expone Google y se reemplazó por `gemini-embedding-001`; el formato de subida de archivos pasó de *multipart* con HMAC (frágil de firmar correctamente en PHP) a JSON con el archivo en base64; y el *streaming* de respuestas, considerado para este sprint, se postergó al Sprint 3 al estimarse en 3 días adicionales de trabajo frente a una latencia sin *streaming* ya aceptable (3 a 6 segundos) para el MVP.

Las métricas de cierre: 25 *issues* completados, una velocidad acumulada de ~12,5 *issues* por sprint, SPI = 1,10 (10% adelantado), CPI = 0,85 (18 % de sobrecosto, ya que 1/0,85 ≈ 1,18, ya esperado al tratarse de los primeros sprints), cerca de 5.860 líneas de código entre backend, plugin y frontend, y una suite de tests que creció de 18 a aproximadamente 35 casos. La retrospectiva registró como punto fuerte que el *polling* de estado de indexación en la vista docente resultó una solución de UX simple y efectiva, y como punto débil la documentación opaca de la API de *embeddings* de Google, cuyo modelo aceptado cambió sin aviso previo.

12.4 Un pivote real de infraestructura

La planificación original del Sprint 3 contemplaba desplegar el backend en Fly.io, con un procedimiento documentado paso a paso. Ese despliegue se ejecutó, pero luego fue reemplazado por el esquema actual en Oracle Cloud (ADR-011). El documento de planificación de Fly.io se conserva sin modificaciones, marcado como histórico, como registro de que la infraestructura finalmente utilizada no fue la primera planificada y de los motivos del cambio.

12.5 Cierre

El Anexo C de este informe conserva la tabla de *velocity* consolidada del proyecto completo (SP comprometidos y completados por sprint) como referencia numérica rápida; este capítulo documenta, en cambio, el relato de gestión detrás de dos de esas instancias de cierre formal —con sus métricas de valor ganado, sus desviaciones justificadas y sus lecciones aprendidas— tal como quedaron asentadas en las actas firmadas por el equipo.

13 Beneficios post-implementación

Tras la implementación del proyecto se esperan los siguientes beneficios, tanto tangibles como intangibles:

- **Acceso a asistencia académica disponible de forma permanente (24/7)** sobre el material real del curso, sin incorporar personal de tutoría adicional, con una latencia p95 de 4,4 s por respuesta y una cobertura de pruebas automatizadas del 83,6 % (sección 8.6).
- **Reducción de la dispersión de la información**, al concentrar las consultas dentro del campus virtual donde reside el material validado: el relevamiento propio mostró que el 80,7 % de los estudiantes percibía esa dispersión antes de la implementación.
- **Mejora en la toma de decisiones del docente**, mediante el reporte de vacíos de contenido que evidencia qué temas no quedan cubiertos por el material.
- **Mitigación del riesgo de respuestas incorrectas**, gracias a la trazabilidad de cada respuesta hacia su fragmento de origen: en el *golden set* del MVP no hubo respuestas seguras pero incorrectas (Anexo C).
- **Preservación de la soberanía de los datos académicos**: los datos personales del alumno residen en la base de datos de Moodle de la institución (ADR-014), y el backend solo conserva el índice del material y copias seudonimizadas de los foros.
- **Costo operativo acotado y transparente**: unos USD 0,48 por alumno activo por mes en el escenario bajo, a cargo de la institución, con registro del consumo real de cada llamada a un modelo (sección 14).
- **Software libre y reutilizable**: el código se publica de forma gratuita, sin costo de licencias para la institución ni dependencia de un proveedor.

14 Impacto económico (estudio de costos)

NexusAI se distribuye como software libre y gratuito: el plugin se publica en el Moodle Plugins Directory bajo la licencia GPL v3, y el backend, en un repositorio público de GitHub bajo la licencia MIT, junto con una imagen Docker lista para instalar (sección 7.3.18). El equipo no cobra licencias ni servicios: la decisión responde al propósito de que futuros estudiantes y otras instituciones puedan estudiar, reutilizar y continuar el proyecto. En consecuencia, el proyecto no persigue ingresos, y los costos de operación —infraestructura, consumo de modelos de IA y mantenimiento— quedan a cargo de la institución educativa que lo adopta. Este capítulo estima esos costos en tres escenarios, valora el aporte del equipo y analiza el retorno para la institución. Todos los importes se expresan en dólares estadounidenses (USD).

14.1 Costos de operación para la institución

Los escenarios parten de tres supuestos documentados: (a) un consumo de IA de USD 108 por mes cada 500 alumnos activos —USD 0,216 por alumno y por mes—, estimado en ADR-004 con los precios de GPT-4o-mini (OpenAI, s.f.-c); (b) USD 170 anuales de infraestructura para 500 alumnos, según el estudio de costos del proyecto (Anexo F); y (c) 10 horas mensuales de mantenimiento a USD 12 por hora, la misma tarifa de la sección 14.2. Para los escenarios medio y alto se supone que la infraestructura y las horas de mantenimiento crecen con la escala, aunque en menor proporción que la cantidad de alumnos. El piloto actual corre sobre el nivel gratuito de Oracle Cloud (Oracle, s.f.), por lo que su costo de infraestructura es nulo, y el backend registra el costo real de cada llamada a un modelo en la tabla `llm_usage`, lo que permite reemplazar la estimación de IA por datos medidos una vez que haya uso sostenido.

Tabla 18. *Costos anuales de operación para la institución por escenario (USD)*

Concepto	Piloto actual	Bajo: 500 alumnos	Medio: 2.000 alumnos	Alto: 10.000 alumnos
Infraestructura	0 (Oracle Cloud *Always Free*)	170	340	1.000
Servicios de IA (LLM y *embeddings*)	Variable, registrado en `llm_usage`	1.296	5.184	25.920
Mantenimiento	—	1.440 (10 h/mes)	2.880 (20 h/mes)	5.760 (40 h/mes)
Total anual	**Solo consumo de IA**	**2.906**	**8.404**	**32.680**
Costo por alumno activo (año / mes)	—	5,81 / 0,48	4,20 / 0,35	3,27 / 0,27

El costo por alumno disminuye con la escala porque la infraestructura y el mantenimiento se reparten entre más usuarios; el componente de IA, en cambio, crece en proporción al uso y representa entre el 45 % y el 79 % del total según el escenario (1.296 / 2.906 y 25.920 / 32.680).

14.2 Costos de personal (valoración teórica)

El desarrollo no constituye un costo para la institución, porque el equipo de tres estudiantes de Ingeniería en Sistemas lo aporta como parte de su formación y lo publica como software libre. Como referencia del valor aportado, a una tarifa de referencia de USD 12 por hora para un desarrollador junior (supuesto del equipo), el MVP insumió unas 840 horas-persona, equivalentes a USD 10.080 (840 × 12); esa cifra no incluye el desarrollo posterior al MVP.

14.3 Análisis costo-beneficio para la institución

Sin ingresos por licencias, el análisis relevante es el de la institución que adopta el sistema: el costo de operación frente al valor del acompañamiento académico que brinda. Si una hora de tutoría cuesta USD 20 (supuesto del estudio de costos del proyecto, Anexo F) y NexusAI reemplaza una hora de tutoría por alumno y por año, en el escenario bajo el ahorro es de USD 10.000 (500 × 20) frente a un costo de USD 2.906, lo que equivale a un retorno sobre la inversión (ROI) de (10.000 − 2.906) / 2.906 ≈ 244 %. El punto de equilibrio se alcanza cuando el sistema reemplaza 0,29 horas de tutoría por alumno y por año, unos 17 minutos. La tabla siguiente resume los indicadores para los tres escenarios.

Tabla 19. *Indicadores de costo-beneficio para la institución por escenario*

Indicador	Bajo: 500 alumnos	Medio: 2.000 alumnos	Alto: 10.000 alumnos
Costo anual de operación (USD)	2.906	8.404	32.680
Costo por alumno activo por mes (USD)	0,48	0,35	0,27
Ahorro con 1 h de tutoría por alumno y año (USD)	10.000	40.000	200.000
Retorno sobre la inversión (ROI)	244 %	376 %	512 %
Punto de equilibrio (horas de tutoría por alumno y año)	0,29 (17 min)	0,21 (13 min)	0,16 (10 min)

Al tratarse de software libre, la institución tampoco depende del equipo ni de un proveedor para seguir usando, modificando o alojando el sistema. Los supuestos más sensibles son el costo de la hora de tutoría y el consumo real de IA por alumno; ambos pueden reemplazarse por valores propios de la institución y por el consumo registrado en `llm_usage`.

Como la tarifa de tutoría y el consumo de IA son los supuestos más inciertos, la tabla siguiente muestra la sensibilidad del ROI del escenario bajo a ambos. El retorno se mantiene positivo en todas las combinaciones, incluso con la tutoría más barata y un consumo de IA un 50 % mayor al estimado.

Tabla 20. *Sensibilidad del ROI del escenario bajo (500 alumnos)*

Costo de IA	Tutoría a USD 10/h	Tutoría a USD 20/h	Tutoría a USD 30/h
Estimado −50 % (costo total USD 2.258)	121 %	343 %	564 %
Estimado (ADR-004) (costo total USD 2.906)	72 %	244 %	416 %
Estimado +50 % (costo total USD 3.554)	41 %	181 %	322 %

14.4 Alternativa: clave de IA propia del alumno

Una variante para distribuir el costo de IA es que cada alumno conecte su propia clave de API de un proveedor (modalidad BYOK, sección 7.3.10). El equipo estudió su viabilidad en septiembre de 2026: es técnicamente posible, porque el adaptador `LLMProvider` ya acepta la clave y la dirección del proveedor como parámetros y el identificador del usuario llega al manejador del chat, pero **no está implementada** y requiere tres cambios: crear el proveedor por solicitud en lugar de usar una instancia

compartida; almacenar la clave de cada alumno cifrada, ya que el proyecto no cuenta hoy con un mecanismo de cifrado de secretos por usuario; y separar de esa clave los límites de uso, la moderación y el respaldo institucional, para que el alumno no consuma su cuota en pasos de seguridad ni la institución pague sin advertirlo cuando la clave del alumno se agota. Antes de implementarla deben revisarse, además, los términos de uso de cada proveedor respecto de claves de usuarios finales.

Si una proporción *p* de los alumnos usara su propia clave, el costo anual de IA de la institución se reduciría en esa misma proporción: en el escenario bajo, con *p* = 30 %, pasaría de USD 1.296 a USD 907 (1.296 × 0,7). Sin embargo, exigir la clave propia contradiría el objetivo de equidad del proyecto (sección 15), porque dejaría sin asistente a quien no puede pagar un plan de IA, y agregaría fricción para alumnos sin conocimientos técnicos. Por eso, la opción recomendada es ofrecerla como complemento voluntario, con la clave institucional como opción por defecto; queda planteada como trabajo futuro (sección 17).

15 Impacto social

El impacto social del proyecto se relaciona con la mejora del acceso al conocimiento y la equidad educativa, más allá de los aspectos técnicos o económicos.

- **Beneficio general:** el sistema democratiza el acceso a un acompañamiento académico permanente, históricamente limitado a quienes disponen de tiempo, recursos o redes de apoyo para resolver dudas fuera del aula.
- **Segmentos beneficiados:** estudiantes que cursan a distancia, que trabajan, o que por distintos motivos no pueden asistir a consultas presenciales, encuentran un canal de apoyo disponible en todo momento.
- **Inclusión y reducción de brechas:** al responder sobre el material validado por el docente y citar la fuente, el sistema ofrece una alternativa más confiable y equitativa que el uso desigual de herramientas de IA generales.
- **Apoyo al rol docente:** la solución amplifica la tarea del docente en lugar de reemplazarla, y le brinda información para mejorar su material en beneficio del conjunto del estudiantado.
- **Conocimiento abierto:** el código se publica de forma gratuita y bajo licencias libres para que futuros estudiantes y otras instituciones puedan estudiarlo, reutilizarlo y continuarlo; como la institución asume los costos de operación, ningún estudiante necesita pagar para acceder al asistente.
- **Accesibilidad:** el *widget* declara roles y atributos ARIA, muestra el foco del teclado y respeta la preferencia de movimiento reducido del sistema; la consulta por voz y la vista adaptada a teléfonos facilitan el acceso a quienes tienen dificultades para escribir o no disponen de computadora. No se realizó todavía una auditoría formal según las pautas WCAG 2.2 (World Wide Web Consortium, 2023).
- **Brecha digital:** el sistema requiere conexión a internet, pero funciona en el navegador del teléfono, sin instalar aplicaciones ni contratar planes de IA.

El relevamiento propio del equipo (166 estudiantes y 17 docentes de la UCC) respalda estos puntos: los 88 estudiantes que respondieron la pregunta sobre IA declararon usar herramientas generalistas para estudiar —con la salvedad metodológica de la sección 5.2—, lo que evidencia una necesidad de acompañamiento académico que hoy queda fuera del ámbito institucional y sin trazabilidad hacia el material de la materia. Del lado docente, el 88,2 % delegaría tareas a una IA integrada al campus —la mayoría con la condición de revisar el resultado antes de publicarlo—, lo que indica una disposición amplia a que la tecnología amplifique su rol sin reemplazarlo, en línea con el criterio de diseño "asistir, no sustituir" adoptado por NexusAI.

16 Impacto medioambiental

Si bien se trata de una solución de software, se consideran sus efectos sobre el consumo de recursos de cómputo:

- **Uso eficiente de la energía:** la arquitectura de un único servicio (monolito modular) y el uso de una sola base de datos reducen la huella de cómputo frente a arquitecturas distribuidas más complejas.
- **Aprovechamiento de niveles gratuitos y recursos compartidos:** el despliegue real corre sobre dos VMs `VM.Standard.E2.1.Micro` (1/8 de OCPU con capacidad de ráfaga y 1 GB de RAM cada una) del nivel *Always Free* de Oracle Cloud (Oracle, s.f.) —una de *staging* y otra de producción—, sin aprovisionar hardware dedicado adicional.
- **Sostenibilidad a largo plazo:** la abstracción multi-proveedor permite migrar a modelos de IA más eficientes a medida que estén disponibles, sin rediseñar el sistema.

Como estimación de orden de magnitud, el consumo de inferencia puede acotarse con la medición publicada por Google para su asistente Gemini: 0,24 Wh de energía y 0,26 mL de agua por consulta de texto mediana (Elsworth et al., 2025). Si se suponen 40 consultas por alumno y por mes —supuesto del equipo—, el escenario bajo implicaría unas 240.000 consultas al año (500 × 40 × 12), es decir, alrededor de 58 kWh y 62 litros de agua anuales, unos 0,12 kWh por alumno y por año. El valor es solo indicativo: varía según el modelo, el proveedor y la longitud de las respuestas, y no incluye el consumo de los servidores del backend ni el cálculo de *embeddings*.

17 Conclusión

El proyecto cumplió cinco de sus seis objetivos específicos y cumplió parcialmente el sexto, como resume la tabla siguiente. Se desarrolló un plugin de Moodle funcional con un asistente de IA que aplica RAG sobre el material del curso, con citas trazables, despliegue autohospedado y respuesta progresiva; la versión 0.19.0 fue aprobada y publicada en el Moodle Plugins Directory.

Tabla 21. *Cumplimiento de los objetivos específicos*

Objetivo específico	Estado	Evidencia
1. Relevar las problemáticas del campus virtual	Cumplido	Encuestas a 166 estudiantes y 17 docentes (sección 5).
2. Plugin `local` integrado a Moodle	Cumplido	Instalable en Moodle 4.5 a 5.3; CI en 7 combinaciones; aprobado en el Moodle Plugins Directory.
3. Asistente conversacional RAG	Cumplido	Recall@5 = 1,00 (meta ≥ 0,85), MRR = 0,94 (meta ≥ 0,70) y p95 = 4,39 s (meta < 8 s). Precision@5 = 0,275 quedó por debajo de su referencia de 0,50.
4. Generador de cuestionarios	Cumplido	RF-10 a RF-14 en producción.
5. Analítica y vacíos de contenido	Cumplido	RF-20 a RF-22 en producción.
6. Evaluación de la solución	Parcial	Cobertura del 83,6 % (meta ≥ 70 %) y pruebas de carga dentro del acuerdo de latencia; la prueba de uso se limitó a 3 a 5 estudiantes y no se midieron por separado la latencia hasta el primer *token* ni la fidelidad de las respuestas.

Desde el punto de vista académico, el trabajo aporta respuestas concretas a tres cuestiones vigentes en la aplicación de IA a la educación: la mitigación de la alucinación mediante RAG con citas trazables; el cierre del circuito de retroalimentación entre estudiante y docente mediante la detección automática de vacíos de contenido; y la preservación de la privacidad y la soberanía de los datos académicos mediante una arquitectura autohospedada.

Entre las **limitaciones** actuales se reconocen: el backend no distingue entre instalaciones de Moodle, por lo que cada institución requiere su propio backend; no existen respaldos automáticos de la base de datos del backend; el despliegue en producción sigue siendo manual; la evaluación automatizada del RAG se ejecutó sobre un corpus sintético pequeño y no pudo medir la fidelidad ni la honestidad del *fallback* por falta de cuota del proveedor; la latencia hasta el primer *token* (RNF-06) no

se midió por separado; la evaluación de usabilidad se limitó a un grupo de tres a cinco estudiantes; y la calidad del generador de cuestionarios depende del modelo de lenguaje utilizado.

Como **trabajo futuro** se plantean: incorporar respaldos programados de PostgreSQL y extender el despliegue continuo a producción; identificar a cada instalación de Moodle en el backend para atender a varias instituciones con una misma instancia; repetir la evaluación del RAG con 30 a 50 preguntas anotadas por docentes sobre material real e integrarla al CI con `ragas`; medir la latencia hasta el primer *token* y realizar pruebas de usabilidad con una muestra amplia de estudiantes; realizar una auditoría de accesibilidad según WCAG 2.2; evaluar sistemáticamente la resistencia a la inyección de *prompt*; agrupar semánticamente, mediante *embeddings*, los vacíos de contenido detectados; e implementar, como complemento voluntario, la opción de que cada alumno conecte su propia clave de IA (sección 14.4).

En síntesis, el proyecto demuestra que es posible integrar asistencia académica contextualizada y auditable dentro del LMS institucional, amplificando el rol del docente en lugar de sustituirlo.

17.1 Reflexiones personales del equipo

Delfina Salinas. Como *Scrum Master*, el aprendizaje de gestión más concreto fue coordinar la documentación de un proyecto que seguía creciendo en paralelo, sin que eso bloqueara al resto del equipo —dividir la expansión de este mismo informe en issues pequeñas y acotadas (`PI-08` a `PI-22`), cada una con su fuente real ya identificada, resultó más sostenible que plantear una reescritura grande de una sola vez. En lo técnico, investigar la *Hook API* de Moodle me dejó un hallazgo que no esperaba: un *callback* que funcionaba sin problema en versiones viejas de la plataforma deja de funcionar en Moodle 4.4 en adelante sin ningún error visible, solo una advertencia de deprecación fácil de pasar por alto. Si lo volviera a hacer, documentaría las decisiones de diseño e infraestructura en el momento en que se toman —el rediseño de la interfaz de agosto, por ejemplo, quedó registrado solo en mensajes de *commit*— en lugar de tener que reconstruirlas meses después para este informe.

Santiago Tricherri. Ejecutar una auditoría de seguridad real sobre el propio proyecto, en vez de aplicar una lista genérica de buenas prácticas, me obligó a mirar código que ya daba por “terminado” con otra exigencia, y encontré algo concreto que no habría aparecido de otra forma: una métrica de usuario que usaba un hash sin sal, reversible por fuerza bruta. En gestión, ser el único integrante con acceso de *deploy* a los dos ambientes de producción me mostró que esa concentración es una responsabilidad operativa real, no solo una formalidad en un documento. Si lo volviera a hacer, repartiría ese acceso antes de que se volviera un cuello de botella, en vez de esperar a que el equipo creciera para resolverlo.

Marcos Bugliotti. Integrar un *plugin* de Moodle con un backend propio —*Hook API*, *capabilities*, *web services*— resultó sustancialmente más profundo de lo que parece desde afuera del proyecto; cada capa (autenticación, autorización, empaquetado del *frontend* como módulo AMD) terminó teniendo su propia complejidad real, no solo de diseño. En gestión, mantener la consistencia entre documentos de investigación escritos en momentos distintos del proyecto es más difícil de lo esperado: la inconsistencia de nombres de *capabilities* que encontramos al redactar este mismo informe —dos convenciones distintas documentadas en dos archivos distintos— es un ejemplo real de cómo terminología técnica puede divergir en silencio si nadie la revisa de conjunto. Si lo volviera a hacer, congelaría nombres y contratos técnicos clave más temprano en el proyecto, en lugar de dejarlos evolucionar libremente entre documentos de diseño.

18 Bibliografía / Referencias

Las referencias siguen la norma APA v7. Toda fuente citada en el texto figura en esta sección, y toda referencia listada fue utilizada en el informe.

Artifex. (s.f.). *PyMuPDF documentation — About*. Recuperado el 23 de octubre de 2026, de https://pymupdf.readthedocs.io/en/latest/about.html

Caddy. (s.f.). *Caddy documentation — Automatic HTTPS*. Recuperado el 22 de octubre de 2026, de https://caddyserver.com/docs/automatic-https

ChromaDB. (s.f.). *Chroma documentation*. Recuperado el 22 de octubre de 2026, de https://docs.trychroma.com/

Cohn, M. (2009). *Succeeding with Agile: Software development using Scrum*. Addison-Wesley.

Coursera. (s.f.). *Coursera Coach: Leveraging GenAI to empower learners*. Coursera Blog. Recuperado el 22 de octubre de 2026, de https://blog.coursera.org/coursera-coach/

Cure53. (s.f.). *DOMPurify* [Software]. GitHub. Recuperado el 22 de octubre de 2026, de https://github.com/cure53/DOMPurify

Digital Education Council. (2024). *Global AI Student Survey 2024*. Recuperado el 23 de octubre de 2026, de https://www.digitaleducationcouncil.com/post/digital-education-council-global-ai-student-survey-2024

Django Software Foundation. (s.f.). *Django documentation*. Recuperado el 22 de octubre de 2026, de https://docs.djangoproject.com/

Docker. (s.f.). *Docker Compose documentation*. Recuperado el 22 de octubre de 2026, de https://docs.docker.com/compose/

Electronic Frontier Foundation. (s.f.). *Certbot*. Recuperado el 22 de octubre de 2026, de https://certbot.eff.org/

Elsworth, C., Huang, K., Patterson, D., Schneider, I., Sedivy, R., Goodman, S., Townsend, B., Ranganathan, P., Dean, J., Vahdat, A., Gomes, B., & Manyika, J. (2025). *Measuring the environmental impact of delivering AI at Google scale*. arXiv. https://doi.org/10.48550/arXiv.2508.15734

Es, S., James, J., Espinosa-Anke, L., & Schockaert, S. (2024). RAGAS: Automated evaluation of retrieval augmented generation. En *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations* (pp. 150–158). Association for Computational Linguistics.

FastAPI. (s.f.). *FastAPI documentation*. Recuperado el 22 de octubre de 2026, de https://fastapi.tiangolo.com/

Fette, I., & Melnikov, A. (2011). *The WebSocket protocol* (RFC 6455). IETF. https://doi.org/10.17487/RFC6455

Fly.io. (s.f.). *Fly.io documentation*. Recuperado el 23 de octubre de 2026, de https://fly.io/docs/

Free Software Foundation. (2007). *GNU General Public License, version 3*. Recuperado el 23 de octubre de 2026, de https://www.gnu.org/licenses/gpl-3.0.html

Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, H., & Wang, H. (2023). *Retrieval-augmented generation for large language models: A survey.* arXiv. https://arxiv.org/abs/2312.10997

Google. (s.f.-a). *Gemini API — Embeddings*. Recuperado el 22 de octubre de 2026, de https://ai.google.dev/gemini-api/docs/embeddings

Google. (s.f.-b). *Gemini API — OpenAI compatibility*. Recuperado el 22 de octubre de 2026, de https://ai.google.dev/gemini-api/docs/openai

Google. (s.f.-c). *NotebookLM*. Recuperado el 22 de octubre de 2026, de https://notebooklm.google/

Grafana Labs. (s.f.). *Grafana k6 documentation*. Recuperado el 22 de octubre de 2026, de https://grafana.com/docs/k6/latest/

Humble, J., & Farley, D. (2010). *Continuous delivery: Reliable software releases through build, test, and deployment automation*. Addison-Wesley.

International Organization for Standardization. (2023). *ISO/IEC 25010:2023 Systems and software engineering — Systems and software Quality Requirements and*

Evaluation (SQuaRE) — Product quality model. ISO. https://www.iso.org/standard/78176.html

Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y. J., Madotto, A., & Fung, P. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys, 55*(12), 1–38. https://doi.org/10.1145/3571730

Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). IETF. https://doi.org/10.17487/RFC7519

Khan Academy. (s.f.). *Khan Labs — Khanmigo*. Recuperado el 22 de octubre de 2026, de https://www.khanacademy.org/khan-labs

Krawczyk, H., Bellare, M., & Canetti, R. (1997). *HMAC: Keyed-hashing for message authentication* (RFC 2104). IETF. https://doi.org/10.17487/RFC2104

LangChain. (s.f.). *LangChain documentation*. Recuperado el 23 de octubre de 2026, de https://python.langchain.com/docs/introduction/

Leitner, S. (1972). *So lernt man lernen*. Herder.

Let's Encrypt. (s.f.). *Let's Encrypt — How it works*. Recuperado el 22 de octubre de 2026, de https://letsencrypt.org/how-it-works/

Lewis, J., & Fowler, M. (2014). *Microservices: A definition of this new architectural term*. martinfowler.com. https://martinfowler.com/articles/microservices.html

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems, 33*, 9459–9474.

Ley N.º 25.326. (2000). *Ley de Protección de los Datos Personales*. Boletín Oficial de la República Argentina, 2 de noviembre de 2000.

Malkov, Y. A., & Yashunin, D. A. (2020). Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence, 42*(4), 824–836. https://doi.org/10.1109/TPAMI.2018.2889473

Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to information retrieval*. Cambridge University Press.

Microsoft. (s.f.). *Microsoft Copilot for Education*. Recuperado el 22 de octubre de 2026, de https://www.microsoft.com/en-us/education/products/copilot

Moodle. (s.f.-a). *AI Chat* [Plugin]. Moodle Plugins Directory. Recuperado el 22 de octubre de 2026, de https://moodle.org/plugins/block_ai_chat

Moodle. (s.f.-b). *AI Chat Block* [Plugin]. Moodle Plugins Directory. Recuperado el 22 de octubre de 2026, de https://moodle.org/plugins/block_openai_chat

Moodle. (s.f.-c). *AI Manager* [Plugin]. Moodle Plugins Directory. Recuperado el 22 de octubre de 2026, de https://moodle.org/plugins/local_ai_manager

Moodle. (s.f.-d). *Moodle statistics*. Recuperado el 23 de octubre de 2026, de https://stats.moodle.org/

Moodle. (s.f.-e). *NexusAI* [Plugin]. Moodle Plugins Directory. Recuperado el 21 de octubre de 2026, de https://marketplace.moodle.com/plugins/4243

Moodle. (s.f.-f). *Terus RAG* [Plugin]. Moodle Plugins Directory. Recuperado el 22 de octubre de 2026, de https://moodle.org/plugins/block_terusrag

Moodle HQ. (2024). *Moodle Developer Documentation — Plugin types: local*. https://moodledev.io/docs/apis/plugintypes/local

Moodle HQ. (s.f.-a). *Moodle Developer Documentation — Access API*. Recuperado el 22 de octubre de 2026, de https://moodledev.io/docs/apis/subsystems/access

Moodle HQ. (s.f.-b). *Moodle Developer Documentation — AI subsystem*. Recuperado el 22 de octubre de 2026, de https://moodledev.io/docs/apis/subsystems/ai

Moodle HQ. (s.f.-c). *Moodle Developer Documentation — External functions and web services*. Recuperado el 22 de octubre de 2026, de https://moodledev.io/docs/apis/subsystems/external

Moodle HQ. (s.f.-d). *Moodle Developer Documentation — Hooks API*. Recuperado el 22 de octubre de 2026, de https://moodledev.io/docs/apis/core/hooks

Moodle HQ. (s.f.-e). *Moodle Developer Documentation — JavaScript modules*. Recuperado el 22 de octubre de 2026, de https://moodledev.io/docs/guides/javascript/modules

Moodle HQ. (s.f.-f). *Moodle Developer Documentation — Privacy API*. Recuperado el 22 de octubre de 2026, de https://moodledev.io/docs/apis/subsystems/privacy

Moodle HQ. (s.f.-g). *Moodle Developer Documentation — Templates*. Recuperado el 22 de octubre de 2026, de https://moodledev.io/docs/guides/templates

Moodle HQ. (s.f.-h). *Plugin contribution checklist*. Recuperado el 22 de octubre de 2026, de https://moodledev.io/general/community/plugincontribution/checklist

Myers, G. J., Sandler, C., & Badgett, T. (2011). *The art of software testing* (3.ª ed.). Wiley.

NGINX. (s.f.). *NGINX documentation*. Recuperado el 22 de octubre de 2026, de https://nginx.org/en/docs/

Open Source Initiative. (s.f.). *The MIT License*. Recuperado el 23 de octubre de 2026, de https://opensource.org/license/mit

OpenAI. (s.f.-a). *ChatGPT Edu*. Recuperado el 22 de octubre de 2026, de https://openai.com/chatgpt/education/

OpenAI. (s.f.-b). *Embeddings*. Recuperado el 22 de octubre de 2026, de https://platform.openai.com/docs/guides/embeddings

OpenAI. (s.f.-c). *GPT-4o pricing*. Recuperado el 22 de octubre de 2026, de https://openai.com/api/pricing/

OpenAI. (s.f.-d). *Moderation*. Recuperado el 22 de octubre de 2026, de https://platform.openai.com/docs/guides/moderation

OpenAI. (s.f.-e). *Structured outputs*. Recuperado el 22 de octubre de 2026, de https://platform.openai.com/docs/guides/structured-outputs

OpenJS Foundation. (s.f.). *Express: Fast, unopinionated, minimalist web framework for Node.js*. Recuperado el 22 de octubre de 2026, de https://expressjs.com/

Oracle. (s.f.). *Oracle Cloud Infrastructure — Always Free resources*. Recuperado el 22 de octubre de 2026, de https://www.oracle.com/cloud/free/

OWASP. (s.f.-a). *OWASP Top 10*. OWASP Foundation. Recuperado el 22 de octubre de 2026, de https://owasp.org/www-project-top-ten/

OWASP. (s.f.-b). *OWASP Top 10 for Large Language Model Applications*. OWASP Foundation. Recuperado el 22 de octubre de 2026, de https://owasp.org/www-project-top-10-for-large-language-model-applications/

pdfminer.six. (s.f.). *pdfminer.six documentation*. Recuperado el 23 de octubre de 2026, de https://pdfminersix.readthedocs.io/

pdfplumber. (s.f.). *pdfplumber* [Software]. GitHub. https://github.com/jsvine/pdfplumber

pgvector. (s.f.). *pgvector: Open-source vector similarity search for Postgres* [Software]. GitHub. https://github.com/pgvector/pgvector

Pinecone. (s.f.). *Pinecone documentation*. Recuperado el 23 de octubre de 2026, de https://docs.pinecone.io/

PostgreSQL Global Development Group. (s.f.). *PostgreSQL documentation — Full text search*. Recuperado el 22 de octubre de 2026, de https://www.postgresql.org/docs/current/textsearch.html

Prometheus Authors. (s.f.). *Prometheus overview*. Recuperado el 23 de octubre de 2026, de https://prometheus.io/docs/introduction/overview/

Pydantic. (s.f.). *Pydantic documentation*. Recuperado el 22 de octubre de 2026, de https://docs.pydantic.dev/

pypdf. (s.f.). *pypdf documentation*. Recuperado el 23 de octubre de 2026, de https://pypdf.readthedocs.io/

pytest. (s.f.). *pytest documentation*. Recuperado el 22 de octubre de 2026, de https://docs.pytest.org/

Qdrant. (s.f.). *Qdrant documentation*. Recuperado el 23 de octubre de 2026, de https://qdrant.tech/documentation/

Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. (2023). Robust speech recognition via large-scale weak supervision. En *Proceedings of the 40th International Conference on Machine Learning* (pp. 28492–28518). PMLR. https://proceedings.mlr.press/v202/radford23a.html

Railway. (s.f.). *Railway documentation*. Recuperado el 23 de octubre de 2026, de https://docs.railway.com/

React. (s.f.). *React documentation — createRoot*. Recuperado el 22 de octubre de 2026, de https://react.dev/reference/react-dom/client/createRoot

Redis. (s.f.). *SET command*. Redis documentation. Recuperado el 22 de octubre de 2026, de https://redis.io/docs/latest/commands/set/

Reglamento (UE) 2016/679. (2016). Reglamento del Parlamento Europeo y del Consejo relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales y a la libre circulación de estos datos (Reglamento general de protección de datos). *Diario Oficial de la Unión Europea, L 119*, 1–88.

Rescorla, E. (2018). *The Transport Layer Security (TLS) protocol version 1.3* (RFC 8446). IETF. https://doi.org/10.17487/RFC8446

Sandhu, R. S., Coyne, E. J., Feinstein, H. L., & Youman, C. E. (1996). Role-based access control models. *Computer, 29*(2), 38–47. https://doi.org/10.1109/2.485845

Saylor Academy. (s.f.). *SOLA — Saylor Online Learning Assistant: AI-powered learning coach Moodle plugin* [Software]. GitHub. Recuperado el 22 de octubre de 2026, de https://github.com/saylordotorg/moodle-local_ai_course_assistant

Smith, R. (2007). An overview of the Tesseract OCR engine. En *Ninth International Conference on Document Analysis and Recognition (ICDAR 2007)* (Vol. 2, pp. 629–633). IEEE. https://doi.org/10.1109/ICDAR.2007.4376991

SonarSource. (s.f.). *SonarQube Cloud documentation — Quality gates*. Recuperado el 22 de octubre de 2026, de https://docs.sonarsource.com/sonarqube-cloud/

Tanenbaum, A. S., & Wetherall, D. J. (2011). *Computer networks* (5.ª ed.). Pearson.

UNESCO. (2023). *Guidance for generative AI in education and research*. UNESCO. https://doi.org/10.54675/EWZM9535

v0. (s.f.). *v0 by Vercel — Generative UI* [Software]. Vercel. https://v0.dev/

Vitest. (s.f.). *Vitest documentation*. Recuperado el 22 de octubre de 2026, de https://vitest.dev/

Weaviate. (s.f.). *Weaviate documentation*. Recuperado el 23 de octubre de 2026, de https://weaviate.io/developers/weaviate

Webpack. (s.f.). *Webpack documentation — Output, externals y code splitting*. Recuperado el 22 de octubre de 2026, de https://webpack.js.org/configuration/output/

WHATWG. (s.f.). *HTML Living Standard — Server-sent events*. Recuperado el 22 de octubre de 2026, de https://html.spec.whatwg.org/multipage/server-sent-events.html

World Wide Web Consortium. (2023). *Web Content Accessibility Guidelines (WCAG) 2.2*. W3C. https://www.w3.org/TR/WCAG22/

Woźniak, P. A. (1990). *Optimization of learning* [Tesis de maestría]. University of Technology in Poznań.

Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., & Stoica, I. (2023). Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. *Advances in Neural Information Processing Systems, 36*.

19 Anexos

Los anexos reúnen material complementario que respalda el informe sin interrumpir su lectura principal.

19.1 Anexo A — Requerimientos detallados

Cada requerimiento tiene un ID único (RF-NN funcionales, RNF-NN no funcionales), una prioridad en el contexto del MVP y el sprint donde se implementó.

19.1.1 Requerimientos funcionales

Tabla 22. *Requerimientos funcionales: asistente conversacional RAG*

ID	Requerimiento	Prioridad	Sprint
RF-01	Consulta en lenguaje natural sobre el material del curso desde un widget flotante.	Alta	1
RF-02	El asistente responde usando exclusivamente el material indexado, citando los archivos fuente.	Alta	2
RF-03	Si la pregunta no se puede responder con el material disponible, el sistema lo admite explícitamente.	Alta	2
RF-04	El alumno ve el fragmento exacto usado para responder, con su porcentaje de similitud.	Alta	4
RF-05	La respuesta aparece token por token (*streaming*) para reducir la latencia percibida.	Media	4
RF-06	El alumno puede retomar conversaciones anteriores desde un historial.	Media	4
RF-07	Modo multi-curso opcional: el asistente consulta material de todos los cursos inscriptos.	Media	4

Tabla 23. *Requerimientos funcionales: buscador semántico*

ID	Requerimiento	Prioridad	Sprint
RF-08	Búsqueda de fragmentos por similitud semántica, sin pasar por el LLM.	Media	4
RF-09	Cada resultado muestra archivo, fragmento y porcentaje de similitud.	Media	4

Tabla 24. *Requerimientos funcionales: generador de cuestionarios*

ID	Requerimiento	Prioridad	Sprint
RF-10	Generación de un *quiz* de opción múltiple desde el material del curso.	Media	4
RF-11	El alumno elige un tema específico o deja el campo vacío para variedad de temas.	Media	4
RF-12	Si el tema pedido no está en el material, el sistema avisa (no genera un *quiz* engañoso).	Media	4
RF-13	Cada pregunta tiene 4 opciones: una correcta y tres distractores plausibles.	Media	4
RF-14	Feedback inmediato tras cada respuesta, con explicación y archivo fuente.	Media	4

Tabla 25. *Requerimientos funcionales: gestión de material (rol docente)*

ID	Requerimiento	Prioridad	Sprint
RF-15	El docente sube archivos PDF al curso para que el asistente los indexe.	Alta	2
RF-16	Extracción de texto, *chunking* y *embeddings* de cada archivo de forma asíncrona.	Alta	2
RF-17	El docente ve el estado de cada documento (`pending` / `indexing` / `indexed` / `error`).	Alta	2
RF-18	El docente puede borrar un documento; el borrado se propaga en cascada a sus *chunks*.	Alta	2
RF-19	Detección de *uploads* duplicados por hash SHA-256, evita reindexar el mismo archivo.	Media	4

Tabla 26. *Requerimientos funcionales: feedback al docente (detección de vacíos)*

ID	Requerimiento	Prioridad	Sprint
RF-20	Registro automático de las preguntas que el material no pudo responder.	Media	4
RF-21	Reporte agregado de vacíos por curso, ordenado por frecuencia.	Media	4
RF-22	El reporte de vacíos se filtra por ventana temporal (7/30/90/365 días).	Baja	4

19.1.2 Requerimientos no funcionales

Tabla 27. *Requerimientos no funcionales*

ID	Requerimiento	Categoría
RNF-01	La API key del LLM nunca llega al navegador del alumno.	Seguridad

ID	Requerimiento	Categoría
RNF-02	Toda comunicación entre el plugin y el backend se firma con HMAC SHA-256 en 3 capas (Bearer + firma + *nonce*).	Seguridad
RNF-03	El backend rechaza firmas inválidas, timestamps vencidos (>5 min) o *nonces* reusados.	Seguridad
RNF-04	El plugin valida la *capability* `local/nexusai:use` antes de llamar al backend.	Seguridad
RNF-05	El backend valida que un usuario no pueda leer sesiones de chat ajenas.	Seguridad
RNF-06	Latencia al primer token de respuesta (streaming) < 2 s.	Performance
RNF-07	Latencia de respuesta completa del chat (sin streaming) < 8 s.	Performance
RNF-08	Latencia del buscador semántico (retrieval puro) < 1 s.	Performance
RNF-09	Tiempo de indexación de un PDF de 50 páginas < 60 s.	Performance
RNF-10	Instalable en Moodle 4.5 LTS a 5.3 sin `composer install` ni dependencias externas (el MVP soportaba Moodle 4.1 a 4.5).	Compatibilidad
RNF-11	El widget funciona en Chrome, Firefox, Safari y Edge en versiones actuales.	Compatibilidad
RNF-12	El backend corre sobre Python 3.11+, PostgreSQL 16+ y pgvector 0.3.5+.	Compatibilidad
RNF-13	Los datos académicos viven en una base de datos controlada por la institución.	Privacidad
RNF-14	El plugin declara su Privacy API según el modelo híbrido de ADR-014 (datos del alumno en tablas nativas de Moodle, con retención/eliminación acorde).	Privacidad
RNF-15	El proveedor del LLM se cambia sin modificar código (variable de entorno).	Privacidad
RNF-16	Cobertura de tests automatizados del backend de al menos 70%.	Mantenibilidad
RNF-17	CI/CD en GitHub Actions para backend, frontend y plugin PHP.	Mantenibilidad
RNF-18	Decisiones arquitectónicas documentadas como ADRs en `docs/adr/`.	Mantenibilidad
RNF-19	Cada *feature* documentada con ADR (si aplica), comentarios en código e issue con criterios de aceptación.	Mantenibilidad
RNF-20	El plugin se distribuye como ZIP instalable vía *Site administration → Plugins → Install plugins*.	Distribución
RNF-21	Cada *release* etiquetada en GitHub con *changelog* y `version.php` actualizado.	Distribución

19.2 Anexo B — Modelo de datos

El backend de NexusAI usa una única base de datos PostgreSQL con la extensión pgvector, donde conviven los *embeddings* y los datos relacionales. Como se observa en la Figura 14, el núcleo del modelo vigente es el índice del material (`documents` y

`chunks`), al que se suman los resúmenes en caché y las copias seudonimizadas de los *posts* de foro.

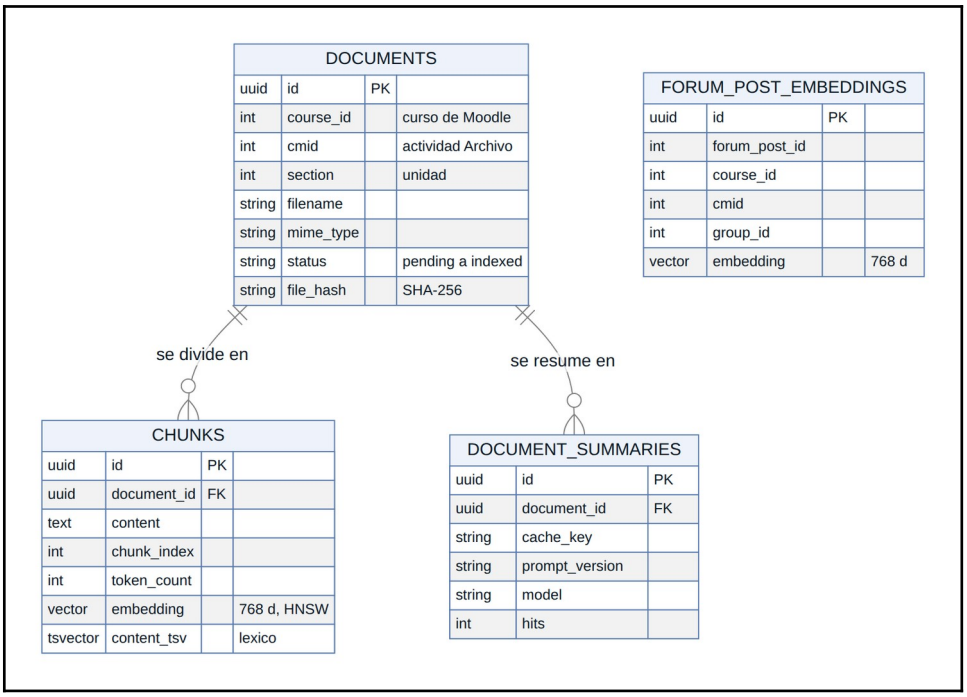


Figura 14. Diagrama entidad-relación del índice de material del backend. *Fuente:* elaboración propia a partir de los modelos de SQLAlchemy del backend. *Texto alternativo:* la entidad `documents` se relaciona de uno a muchos con `chunks` y con `document_summaries`; `forum_post_embeddings` es independiente y se vincula con Moodle mediante los identificadores de *post*, curso, actividad y grupo.

Notas de implementación: `chunks.embedding` usa un índice HNSW con distancia coseno (*m* = 16, *ef_construction* = 64) y `chunks.content_tsv` alimenta la búsqueda léxica del buscador híbrido. `documents` tiene un índice único parcial sobre (`course_id`, `filename`, `file_hash`) que evita indexar dos veces el mismo archivo, y la columna `cmid` vincula cada documento con su actividad Archivo de Moodle para el filtro de visibilidad. El esquema evolucionó en 26 migraciones de Alembic (001 a 026). El registro de consumo se compone de tres tablas adicionales: `llm_usage` (una fila por llamada, con funcionalidad, proveedor, modelo, *tokens*, costo y latencia), `model_prices` (precios por millón de *tokens* con fecha de vigencia) y `llm_usage_daily` (consolidación diaria). Por ADR-014, los datos personales del alumno —sesiones de chat, mensajes, vacíos de contenido, intentos y errores de *quiz*, *flashcards* y alertas de calendario— residen en dieciséis tablas nativas `local_nexusai_*` del plugin; las tablas equivalentes del

backend se conservan solo para migrar instalaciones existentes y archivarlas después.

19.3 Anexo C — Métricas del proyecto

Tabla 28. *Velocidad por sprint*

Sprint	SP comprometidos	SP completados	Velocity
Sprint 0 — Setup	60	60	100%
Sprint 1 — Core chat	50	45	90%
Sprint 2 — RAG	60	55	92%
Sprint 3 — Calidad	25	25	100%
Sprint 4 — MVP	70	70	100%
Promedio			**96%**

Tabla 29. *Métricas de desarrollo*

Métrica	Cierre del MVP (06/2026)	Estado al 22/10/2026
Commits en el repositorio	~290	738
Issues cerradas	160+	—
Pull requests integradas	85+	230
Líneas del backend en Python (sin pruebas)	~3.500	14.174
Líneas del plugin en PHP (sin pruebas)	~2.500	18.938
Líneas del *frontend* React (JS, JSX y CSS)	~3.200	19.461
Migraciones de base de datos	5	26
ADRs documentados	6	10
Funciones externas del plugin	—	56
Pruebas automatizadas	37 (backend)	539 pytest · 257 PHPUnit · 236 Vitest
Cobertura de pruebas	~80 % (backend)	83,6 % (total ponderado de los tres componentes)

Los valores al 22/10/2026 se obtuvieron del repositorio (`git rev-list --count`, `git log --merges` y conteo de líneas físicas con `wc -l`); los del MVP corresponden al cierre del Sprint 4.

Tabla 30. *Calidad de las respuestas del LLM en el golden set del MVP*

Métrica	Resultado en el MVP
Preguntas con respuesta correcta y citación adecuada	8/10
Preguntas donde el LLM admitió no poder responder (correcto)	2/2
Falsos positivos (respuesta segura pero incorrecta)	0
Falsos negativos (no responde teniendo material disponible)	0

19.4 Anexo D — Repositorio y evidencias

- **Repositorio:** github.com/nexusai-ucc/nexusAI
- **Issues:** github.com/nexusai-ucc/nexusAI/issues
- **Integración continua (GitHub Actions):** github.com/nexusai-ucc/nexusAI/actions
- **Versiones:** `v0.8.0-mvp`, cierre del MVP (github.com/nexusai-ucc/nexusAI/releases/tag/v0.8.0-mvp), y `v0.19.0`, versión aprobada y publicada en el Moodle Plugins Directory, etiquetada en el repositorio derivado github.com/nexusai-ucc/moodle-local_nexusai.

Nota: el backend está desplegado de forma permanente en dos ambientes sobre Oracle Cloud (*Always Free*): *staging*, para las pruebas del día a día (`https://api-staging.146.181.62.67.nip.io`), y producción, solo para funcionalidad ya validada (`https://api.159.112.139.166.nip.io`). Moodle no se despliega junto al backend: cada integrante lo ejecuta localmente y apunta el plugin al ambiente correspondiente. *Staging* se despliega automáticamente con cada cambio en su rama, y producción, en forma manual; para reducir el riesgo de acumular cambios sin probar, el equipo concentra las integraciones a `main` al cierre de cada sprint.

19.5 Anexo E — Registro de decisiones de arquitectura (ADRs)

El proyecto documenta sus decisiones de arquitectura significativas como *Architecture Decision Records* (ADR): diez decisiones aceptadas o propuestas, cada una con su contexto, las alternativas evaluadas y sus consecuencias. Las cuatro primeras se fundamentan en el marco teórico y se listan aquí como registro formal.

ADR-001 — Backend Python como monolito modular (Aceptada, 02/05/2026). Un único proceso FastAPI organizado por dominios (`chat/`, `documents/`, `analytics/`) en lugar de microservicios, decisión desarrollada en Marco Teórico. Revisar si la indexación bloquea el chat por minutos, o si el equipo crece a más de 6 desarrolladores.

ADR-002 — pgvector sobre PostgreSQL como única base de datos (Aceptada, 02/05/2026; acotada por ADR-014 a partir de septiembre: pgvector queda solo para el índice del material del curso). Revisar si una institución supera los 10 millones de vectores.

ADR-003 — Arquitectura agnóstica de proveedor LLM (Aceptada, 02/05/2026). Abstracción propia (`LLMProvider`/`EmbeddingProvider`) sobre el SDK de OpenAI, compatible con la mayoría de los proveedores relevantes cambiando solo la URL base. Revisar si aparece un proveedor incompatible con ese SDK.

ADR-004 — Gemini en MVP, OpenAI en producción (Aceptada, 02/05/2026). Costo de producción proyectado de ~$108/mes para 500 alumnos ($0,22/alumno/mes). Revisar si el piloto supera el límite gratuito de Gemini (1.500 solicitudes/día) antes de pasar a producción.

ADR-005 — Autenticación PHP/Python con HMAC SHA-256 en 3 capas (Aceptada, 04/05/2026). El plugin y el backend son dos servicios independientes, y la clave del proveedor de IA vive solo en el backend — nunca puede llegar al navegador. La autenticación se resuelve en tres capas: una clave `Bearer` identifica al cliente, una firma HMAC SHA-256 sobre `timestamp + nonce + body` garantiza integridad con una ventana de 5 minutos de tolerancia, y un *nonce* de un solo uso guardado en Redis (`SET NX EX 300`) impide repetir una solicitud capturada. Se descartó JWT por no aportar nada sobre HMAC+timestamp para autenticación servicio-a-servicio sin estado, y mTLS por la complejidad de distribuir y rotar certificados para dos servicios bajo el mismo control de despliegue. Revisar si el proyecto pasa a ser multi-institucional (cada universidad como cliente distinto).

ADR-006 — Estrategia de Privacy API: `null_provider` primero (Fecha 04/05/2026; **reemplazada por ADR-014**). Mientras los datos del alumno vivieran solo en el backend externo y no en tablas de Moodle, el plugin declaraba formalmente

`null_provider` en lugar de implementar el `metadata\provider` completo sin datos reales que exportar o borrar — una decisión deliberada (YAGNI), no una omisión, con triggers explícitos para disparar la migración (agregar cualquier tabla `local_nexusai_*` con `userid`, o decidir publicar el plugin en moodle.org). Esos triggers se cumplieron en septiembre y la decisión quedó reemplazada por ADR-014.

ADR-010 — Alcance del copiloto de onboarding al docente (**Propuesta, 30/08/2026 — pendiente de aprobación, no aceptada todavía**). Define que NexusAI **no escribe nada en Moodle**: cada paso del tutorial de "primeros pasos" de un curso es un link a la pantalla nativa de Moodle donde el propio docente hace la acción — se descartó explícitamente la alternativa de que NexusAI cree recursos (foros, grupos) en nombre del docente, por el riesgo de romper datos reales y por ampliar la superficie de la declaración de Privacidad sin necesidad.

ADR-011 — Retiro de las configuraciones de deploy de Railway y Fly.io (Aceptada, 07/09/2026). El backend pasó por tres destinos de despliegue a lo largo del proyecto: Railway durante el MVP y la defensa ante el jurado, Fly.io en el post-MVP temprano, y Oracle Cloud (*Always Free*) desde agosto de 2026, destino actual. Al llegar a septiembre, ninguno de los dos primeros seguía activo, pero el *workflow* de GitHub Actions que desplegaba a Fly.io seguía corriendo en cada *push* a `main` y fallando sistemáticamente — el repositorio mostraba una CI en rojo permanente por infraestructura que ya no existía, señal que competía con las fallas reales. El ADR formaliza eliminar esos archivos de configuración muerta (`deploy.yml`, `fly.toml`, `railway.json`, incluida una segunda copia divergente del *workflow* que nunca llegó a ejecutarse por estar en una ruta que GitHub Actions no lee) y deja explícitamente **sin modificar** la documentación histórica del proyecto (`entrega-final/`, `investigacion/`, el propio `CHANGELOG.md`) que describe Railway o Fly.io como entorno vigente, porque era cierto cuando se escribió — reescribirla sería falsificar el registro del proyecto, no limpiarlo.

ADR-012 — Alertas mínimas viables, sin infraestructura nueva de monitoreo (Aceptada, 11/09/2026). Las dos VMs de Oracle Cloud corren con 1/8 de OCPU y 1 GB de RAM cada una, compartidos entre el backend, PostgreSQL y Redis — un stack de observabilidad como Prometheus + Grafana + Alertmanager consumiría una porción desproporcionada de esa RAM para el tráfico de un piloto. Se implementaron

alertas por correo electrónico (SMTP, sin dependencias nuevas más allá de la librería estándar de Python) para los cuatro eventos con impacto directo en la experiencia del usuario: caída del backend, tasa elevada de errores 5xx, fallas o lentitud del proveedor de LLM, y cuota de IA agotada — cada uno con un umbral y una ventana en Redis para no inundar la casilla de correo mientras el problema persiste.

ADR-014 — Datos del alumno en tablas nativas de Moodle (opción C híbrida) (Aceptada, 26/09/2026; reemplaza a ADR-006 en dónde viven los datos personales, y acota a ADR-002). Es la decisión arquitectónica más relevante del proyecto en su última etapa. El diseño original —todo el dato del alumno en el backend externo— generaba privacidad incompleta (el borrado y la exportación no cubrían todas las tablas, y borrar un curso en Moodle dejaba datos huérfanos en el backend), quedaba fuera de las herramientas de Moodle (sin backup de curso, sin reportes nativos, sin backups del propio backend en ningún ambiente) y dependía de 48 de las 55 funciones externas que tenía el plugin en ese momento llamando al backend por HTTP. Mediciones del equipo mostraron el costo real de esa dependencia: una lectura nativa en la base de Moodle toma 0,23 ms de mediana, la misma lectura vía el backend en la misma máquina toma 4,7 ms, y una llamada al backend de producción toma ~135 ms. La decisión mueve los datos del alumno (sesiones de chat, mensajes, interacciones, intentos y errores de cuestionarios, flashcards, alertas de calendario) a dieciséis tablas nativas `local_nexusai_*`, propiedad del plugin PHP; el backend conserva únicamente el índice vectorial del material del curso y las cachés de contenido, sin datos personales. El trade-off aceptado es tener la misma lógica de negocio duplicada en dos lenguajes (PHP y Python) para lo que se portó, mitigado probando la versión portada contra los mismos casos que los tests de Python ya cubrían.

19.6 Anexo F — Documentación técnica del repositorio

La tabla siguiente indica dónde se encuentra, en el repositorio github.com/nexusai-ucc/nexusAI, la documentación técnica que respalda cada capítulo de este informe.

Tabla 31. *Documentación técnica del repositorio*

Tema	Documento o carpeta del repositorio
Arquitectura y diagramas	`NexusAI/docs/architecture.md` · `NexusAI/docs/diagrams/`
Decisiones de arquitectura	`NexusAI/docs/adr/`
Investigación previa (Moodle, RAG, backend, *frontend*)	`NexusAI/investigacion/`
Relevamiento con encuestas	`NexusAI/investigacion/09-relevamiento/`
Diseño de la interfaz	`NexusAI/docs/diseno/`
Evaluación del RAG	`NexusAI/investigacion/02-rag/evaluacion-rag.md` · `eval-report.json`
Auditoría de seguridad	`NexusAI/docs/security/owasp-audit-2026-08.md`
Despliegue y operación	`NexusAI/docs/DEPLOY_ORACLE.md` · `NexusAI/docs/calidad/CD_STAGING.md`
Migración de los datos del alumno (ADR-014)	`NexusAI/docs/CORTE_DATOS_MOODLE.md`
Calidad: métricas, pruebas de carga y defectos	`NexusAI/docs/calidad/`
Revisión del Moodle Plugins Directory	`NexusAI/docs/MARKETPLACE_SUBMISSION_STATUS.md` · `NexusAI/docs/REVIEWER_TESTING.md`
Estudio de costos (supuestos de infraestructura, mantenimiento y tutoría)	`NexusAI/entrega-final/09_costos.md`
Estimación del costo de IA (ADR-004)	`NexusAI/docs/adr/004-gemini-mvp-openai-prod.md`
Plan de pruebas, casos y registro de defectos	`NexusAI/docs/calidad/PLAN_DE_PRUEBAS.md` · `CASOS_DE_PRUEBA.md` · `REGISTRO_DEFECTOS.md`